

Sistem Informasi Arsip Berkas Digital

Fakultas Informatika dan Teknik Elektro (FITE)

Dokumen Spesifikasi Perancangan Sistem (*Software Requirements Specification*)

Platform REST API: **Go (Gin + GORM) ·
Next.js · MySQL 8 · Google Cloud
Storage**

Program studi	S1 Informatika · S1 Sistem Informasi · S1 Teknik Elektro
Cakupan dokumen	Bab 1 sampai Bab 6, Lampiran A sampai E
Status	Draf. Bab 7 (Screen Design) dan Bab 8 (Kesimpulan) belum disusun
Tanggal	8 Oktober 2026

Catatan status dokumen. Dokumen ini disusun ulang dari draf Laravel menjadi Go + Next.js. Lampiran A.1 (proses As-Is) adalah asumsi yang harus divalidasi ke staf BAAK. Kode pada Bab 5 belum dikompilasi atau diuji; jalankan `go build ./... && go vet ./...` terlebih dahulu. Lampiran A sampai D berisi deskripsi pengerjaan, bukan gambar.

Daftar Isi

BAB 1: PENDAHULUAN

- 1.1 Purpose of Document
- 1.2 Scope
- 1.3 Definition, Acronym, and Abbreviation
- 1.4 Reference Documents
- 1.5 Document Summary

BAB 2: WEBSITE OVERVIEW

- 2.1 Purpose & User Characteristics
- 2.2 Platform & Technical Stack
- 2.3 Operational Requirement & SLA
- 2.4 API Testing Environment (Postman Setup)

BAB 3: TEST CASE (API & SYSTEM)

- 3.0 Ketentuan Umum Pengujian
- 3.1 Autentikasi & Authorization
- 3.2 Upload Berkas
- 3.3 Filter & Multi-Criteria Search
- 3.4 Verifikasi Berkas Publik
- 3.5 Delete Authorization
- 3.6 File Retention Check
- 3.7 Ringkasan Cakupan

BAB 4: REST API SPECIFICATION

- 4.0 Konvensi Umum
- 4.1 Auth & Profile Routes (`/api/v1/auth/...`)
- 4.2 Document Management Routes (`/api/v1/documents/...`)
- 4.3 Verification & Approval Routes (`/api/v1/verifications/...`)
- 4.4 Notification Routes (`/api/v1/notifications/...`)
- 4.5 User & Role Management Routes (`/api/v1/users/...`)
- 4.6 Dashboard Statistics Routes (`/api/v1/dashboard/...`)
- 4.7 Ringkasan

BAB 5: IMPLEMENTASI REST API (GO + GIN + GORM)

- 5.1 Struktur Proyek, Dependensi, dan Konfigurasi
- 5.2 Fondasi: Galat dan Respons
- 5.3 Model Domain (GORM)
- 5.4 Abstraksi Penyimpanan (GCS)
- 5.5 Middleware
- 5.6 Service
- 5.7 Handler
- 5.8 Route Definition
- 5.9 Titik Masuk Aplikasi
- 5.10 Keputusan Implementasi, Pemetaan Uji, dan Hal yang Belum Ditulis

BAB 6: DATABASE DESIGN & ARCHITECTURE

- 6.1 Sitemap / Hierarchical Information Architecture
- 6.2 Table Relation
- 6.3 Table Specifications
- 6.4 Migrasi SQL (`golang-migrate`)

LAMPIRAN

LAMPIRAN A: PEMODELAN PROSES BISNIS (BPMN 2.0)

A.0 Ketentuan Pemodelan

A.1 Proses Saat Ini (Current / As-Is)

A.2 Proses Target (Target / To-Be)

A.3 Perbandingan As-Is dan To-Be

LAMPIRAN B: USE CASE DIAGRAM

B.1 Aktor

B.2 Daftar Use Case

B.3 Relasi

B.4 Tata Letak Diagram

B.5 Spesifikasi Use Case (empat yang paling penting)

LAMPIRAN C: ENTITY RELATIONSHIP DIAGRAM (ERD)

C.1 Pengaturan Diagram

C.2 Entitas dan Atribut

C.3 Relasi dan Kardinalitas

C.4 Catatan Perancangan (tuliskan di bawah ERD)

C.5 Cara Menggambar ERD

LAMPIRAN D: MOCKUP ANTARMUKA

D.1 Panduan Pembuatan di Figma

D.2 Daftar Layar

D.3 Perilaku yang Harus Terlihat pada Mockup

D.4 Alur Prototipe yang Dihubungkan di Figma

LAMPIRAN E: MATRIKS KETERLACAKAN DAN CEKLIS

E.1 Keterlacakan

E.2 Ceklis Penyelesaian

E.3 Keputusan yang Masih Terbuka

1.1 Purpose of Document

Dokumen ini adalah Spesifikasi Perancangan Sistem (*Software Requirements Specification*) untuk **Sistem Informasi Arsip Berkas Digital Fakultas Informatika dan Teknik Elektro (FITE)**. Dokumen ini menjadi acuan bersama bagi:

- **Tim pengembang** (4 akun Super Admin) dalam implementasi backend (Go), frontend (Next.js), dan basis data.
- **Penguji** dalam menyusun dan mengeksekusi pengujian API dan sistem.
- **Pemangku kepentingan fakultas** (Dekan, BAAK, Kaprodi) dalam memvalidasi kesesuaian sistem dengan tata kelola arsip.

Dokumen ini mencakup kebutuhan fungsional dan non-fungsional, spesifikasi REST API, rancangan basis data, rancangan antarmuka, dan skenario pengujian.

1.2 Scope

Dalam cakupan sistem:

1. Repositori terpusat untuk berkas digital tiga program studi: S1 Informatika, S1 Sistem Informasi, dan S1 Teknik Elektro.
2. Autentikasi dan otorisasi berbasis peran (*Role-Based Access Control*) untuk enam aktor.
3. Pengelolaan siklus hidup berkas: unggah, klasifikasi kategori, pengaturan visibilitas (Private/Public), verifikasi, unduh, dan hapus.
4. Penelusuran berkas dengan pencarian kata kunci dan filter multi-kriteria.
5. Notifikasi di dalam sistem untuk antrean verifikasi berkas publik.
6. Penyimpanan fisik berkas pada Google Cloud Storage / Google Workspace Pro melalui API, dengan metadata pada MySQL.
7. Pembersihan otomatis arsip berumur lebih dari 8 tahun oleh penjadwal (*scheduler*) pada layanan backend Go.
8. Dasbor statistik per aktor.

Di luar cakupan sistem:

1. Penyimpanan atau *streaming* video dan berkas besar. Tautan video wajib ditulis pada berkas teks, lalu di-*zip* dan diunggah.
2. Fitur penyuntingan dokumen daring (*collaborative editing*).
3. Integrasi dengan sistem akademik (SIKAD) atau sistem kepegawaian eksternal.
4. Aplikasi *mobile native*. Antarmuka web dirancang responsif.
5. Tanda tangan digital tersertifikasi.

1.3 Definition, Acronym, and Abbreviation

Istilah	Kepanjangan / Definisi
FITE	Fakultas Informatika dan Teknik Elektro
SRS	<i>Software Requirements Specification</i> , dokumen spesifikasi kebutuhan perangkat lunak
REST API	<i>Representational State Transfer Application Programming Interface</i> , gaya arsitektur layanan web berbasis HTTP dan sumber daya (<i>resource</i>)
Go (Golang)	Bahasa pemrograman terkompilasi yang digunakan untuk backend

Istilah	Kepanjangan / Definisi
Gin	<i>Framework</i> HTTP untuk Go yang menyediakan <i>routing</i> , <i>middleware</i> , dan <i>binding</i> JSON
GORM	<i>Object-Relational Mapper</i> untuk Go; memetakan <i>struct</i> Go ke tabel MySQL
Next.js	<i>Framework</i> React untuk frontend; dalam proyek ini memakai <i>App Router</i> dan TypeScript
SPA / CSR / SSR	<i>Single Page Application</i> / <i>Client-Side Rendering</i> / <i>Server-Side Rendering</i>
JWT	<i>JSON Web Token</i> (RFC 7519), <i>token</i> bertanda tangan yang memuat <i>claim</i> identitas
Access Token	JWT berumur pendek (15 menit) yang dikirim pada setiap permintaan API
Refresh Token	<i>Token</i> acak berumur lebih panjang (7 hari) untuk menerbitkan <i>access token</i> baru; disimpan sebagai <i>hash</i> di basis data
Bearer Token	Skema autentikasi HTTP: <code>Authorization: Bearer <token></code>
httpOnly Cookie	<i>Cookie</i> yang tidak dapat dibaca oleh JavaScript di peramban; dipakai untuk menyimpan <i>refresh token</i>
CORS	<i>Cross-Origin Resource Sharing</i>
CSRF	<i>Cross-Site Request Forgery</i>
RBAC	<i>Role-Based Access Control</i> , pengendalian akses berdasarkan peran pengguna
GCS	<i>Google Cloud Storage</i> , layanan penyimpanan objek Google
CRUD	<i>Create, Read, Update, Delete</i>
Soft Delete	Penghapusan logis: data ditandai terhapus (<code>deleted_at</code>) namun masih ada di basis data
Hard Delete	Penghapusan permanen data dan berkas fisik
Scheduler / Cron	Penjadwal tugas periodik (pustaka <code>robfig/cron</code> pada backend Go)
Middleware	Lapisan pemroses permintaan sebelum mencapai <i>handler</i> (autentikasi, otorisasi, pembatasan laju)
Handler	Fungsi Go yang menangani satu <i>endpoint</i> HTTP
Migration	Skrip versi perubahan skema basis data (<code>golang-migrate</code>)
MIME Sniffing	Deteksi tipe berkas dari isi (<i>byte</i> awal), bukan dari ekstensi
BAAK	Biro Administrasi Akademik dan Kemahasiswaan
Kaprodi	Ketua Program Studi
TA (aktor)	<i>Teaching Assistant</i> (Asisten Dosen), digabung dengan aktor Dosen
TA/PA (berkas)	Tugas Akhir / Proyek Akhir
NIM / NIP	Nomor Induk Mahasiswa / Nomor Induk Pegawai
BAP	Berita Acara Perkuliahan
RPS	Rencana Pembelajaran Semester
SK	Surat Keputusan
SLA	<i>Service Level Agreement</i>
Pending Verification	Status berkas publik yang menunggu keputusan verifikator
Approved / Rejected	Status hasil verifikasi berkas publik

1. IEEE Std 830-1998, *Recommended Practice for Software Requirements Specifications*, dan ISO/IEC/IEEE 29148:2018, *Requirements Engineering*.
2. Dokumentasi resmi Go (*The Go Programming Language Specification*, pustaka standar `net/http`, `log/slog`).
3. Dokumentasi Gin Web Framework dan GORM.
4. Dokumentasi Next.js (App Router) dan React.
5. RFC 7519, *JSON Web Token (JWT)*; RFC 6750, *OAuth 2.0 Bearer Token Usage*; RFC 9110, *HTTP Semantics*.
6. Dokumentasi Google Cloud Storage dan pustaka klien Go (`cloud.google.com/go/storage`).
7. Dokumentasi MySQL 8.x Reference Manual.
8. OWASP Application Security Verification Standard (ASVS), OWASP API Security Top 10, dan OWASP JWT Cheat Sheet.
9. Ketentuan retensi dan tata kelola arsip yang berlaku di institusi. *Nomor dan tahun peraturan diisi oleh tim setelah dikonfirmasi ke pihak kampus.*

1.5 Document Summary

Bab	Isi
Bab 1	Tujuan, cakupan, istilah, referensi, dan ringkasan dokumen
Bab 2	Gambaran umum sistem, karakteristik pengguna, kebutuhan fungsional dan non-fungsional, SLA, dan lingkungan pengujian API
Bab 3	Rancangan <i>test case</i> API dan sistem
Bab 4	Spesifikasi <i>endpoint</i> REST API
Bab 5	Implementasi REST API pada Go (Gin + GORM): struktur proyek, <i>handler</i> , DTO validasi, <i>middleware</i> , dan definisi rute
Bab 6	Arsitektur informasi, relasi tabel, spesifikasi tabel, dan <i>migration</i> basis data
Bab 7	Rancangan layar dan <i>wireframe</i> (Next.js) (<i>belum disusun</i>)
Bab 8	Kesimpulan dan rencana pengembangan lanjutan (<i>belum disusun</i>)
Lampiran A-E	Pemodelan proses (BPMN), use case, ERD, mockup, dan matriks keterlacakan (deskripsi pengerjaan)

2.1 Purpose & User Characteristics

Tujuan sistem. Menyediakan satu repositori digital terpusat bagi FITE untuk menggantikan penyimpanan berkas yang tersebar (folder lokal, aplikasi pesan, dan surel). Sistem menjamin bahwa setiap berkas tersimpan aman, terklasifikasi, dapat ditelusuri, dan hanya dapat diakses oleh pihak yang berwenang.

Tabel Hak Akses 6 Aktor

Keterangan: R = Read, C = Create/Upload, M = Manage (ubah metadata, kategori, visibilitas), D = Delete, V = Verify. “Prodi sendiri” berarti prodi yang tertaut pada akun pengguna.

Aktor	Jumlah / Karakteristik	Cakupan Baca (Read)	Unggah & Kategori Berkas	Hak Kelola	Hak Hapus	Hak Verifikasi
Super Admin	4 akun tim pengembang; kemampuan teknis tinggi	Seluruh berkas semua prodi dan aktor	Semua kategori	Penuh (CRUD berkas, user, role, sistem)	Semua berkas, termasuk <i>hard delete</i>	Ya (menyetujui/me nolak)
Dekan	Pimpinan fakultas; penggunaan periodik, butuh tampilan ringkas	Seluruh berkas fakultas (semua prodi, semua aktor)	SK Keputusan Dekan, Dokumen Akreditasi Fakultas, Arsip Khusus	Berkas miliknya	Hanya berkas yang diunggah sendiri	Tidak
BAAK / Staf	Admin operasional; pengguna paling aktif	Berkas per prodi (IF, SI, TE)	Legalisir, Transkrip Nilai Akademik, Dokumen BAAK, Berkas Mahasiswa/Dosen	Berkas per prodi (Read & Management)	Berkas miliknya dan berkas operasional prodi	Ya (Approved/Rejected) dan menerima notifikasi antrean
Kaprodi	Pimpinan prodi	Berkas dosen se-fakultas dan berkas mahasiswa di prodi sendiri	SK Keputusan Prodi, Dokumen Akreditasi Prodi, Kurikulum, dll.	Berkas miliknya	Hanya berkas miliknya	Tidak
Dosen / TA	Pengajar dan asisten; mengunggah dokumen perkuliahan	Berkas mahasiswa di prodi sendiri dan berkas pribadi	RPS, SK Mengajar, BAP, dll.	Berkas miliknya	Hanya berkas miliknya	Tidak
Mahasiswa	Jumlah terbanyak; literasi digital beragam	Hanya berkas pribadi (default)	Proposal TA/PA, Berkas Yudisium, Surat Keterangan, dll.	Berkas miliknya	Hanya berkas miliknya	Tidak

Catatan aturan tambahan:

- 1. Berkas **Public** yang berstatus *Approved* dapat diakses dan diunduh seluruh civitas akademika terautentikasi melalui *explorer* publik.
- 2. Berkas yang belum *Approved* tetap mengikuti aturan visibilitas Private.
- 3. Pemilik berkas dapat mengunduh berkas miliknya kapan pun.
- 4. Aktor yang berada di atas hierarki (sesuai matriks akses) dapat membaca berkas Private bawahannya sesuai cakupan pada tabel.

Lapisan	Teknologi	Peran
Frontend	Next.js (App Router) + React + TypeScript	Antarmuka pengguna; dijalankan sebagai aplikasi terpisah (<i>decoupled</i>) dan berkomunikasi dengan backend melalui REST API dan JSON
Backend	Go + Gin (RESTful API)	Logika bisnis, validasi, otorisasi, penjadwal
Akses data	GORM + driver MySQL	Pemetaan model ke tabel, <i>query</i> , transaksi, <i>soft delete</i>
Migrasi skema	golang-migrate (berkas SQL <i>up/down</i>)	Perubahan skema terkontrol dan dapat di- <i>rollback</i>
Autentikasi	JWT <i>access token</i> (Bearer) + <i>refresh token</i> (httpOnly cookie)	Identifikasi pengguna pada setiap permintaan API; sesi dapat dicabut
Validasi	go-playground/validator (terintegrasi di Gin)	Validasi input terstruktur
Basis Data	MySQL 8.x (relasional)	Metadata berkas, pengguna, peran, prodi, kategori, verifikasi, notifikasi, log penyimpanan, sesi
Penyimpanan Berkas	Google Cloud Storage / Google Workspace Pro (via API)	Penyimpanan fisik berkas; basis data hanya menyimpan referensi objek
Penjadwal	robfig/cron v3 dalam proses <i>worker</i>	Pembersihan otomatis arsip > 8 tahun
Log	log/slog (JSON terstruktur)	Pencatatan aplikasi dan galat
Pengujian API	Postman (dan Newman)	Uji fungsional dan koleksi dokumentasi API

Alur arsitektur.

1. Peramban memuat Next.js. Setelah *login*, *access token* disimpan **di memori aplikasi** (bukan `localStorage`), sedangkan *refresh token* disimpan di *cookie* httpOnly yang diatur oleh API Go.
2. Setiap permintaan ke API Go membawa `Authorization: Bearer <access token>`. *Middleware* Go memeriksa tanda tangan JWT, masa berlaku, status sesi, dan status akun, lalu memeriksa peran dan kepemilikan (*policy*).
3. Saat *access token* kedaluwarsa (401), frontend memanggil `POST /auth/refresh` secara otomatis, menerima pasangan *token* baru (*refresh token* diganti pada setiap pemakaian), lalu mengulang permintaan.
4. Untuk unggah, backend memvalidasi berkas, menyimpannya ke GCS, dan mencatat metadata ke MySQL dalam satu alur atomik. Untuk unduh, backend memeriksa hak akses lalu mengalirkan berkas dari GCS.
5. Proses *worker* (penjadwal) berjalan terpisah dari server HTTP namun memakai basis kode dan layanan yang sama.

2.2.1 Development Requirement

A. Kebutuhan Fungsional per Aktor

Umum (semua aktor)

ID	Kebutuhan
FR-AUTH-01	Sistem menyediakan <i>login</i> dengan kredensial dan menghasilkan <i>access token</i> (Bearer) serta <i>refresh token</i> sesuai peran pengguna
FR-AUTH-02	Sistem menyediakan <i>logout</i> yang mencabut sesi aktif (<i>access</i> dan <i>refresh token</i> tidak berlaku lagi)

ID	Kebutuhan
FR-AUTH-03	Sistem menampilkan dan memperbarui profil serta mengganti kata sandi
FR-AUTH-04	Sistem menyediakan pembaruan <i>access token</i> melalui <i>refresh token</i> dengan rotasi (<i>refresh token</i> lama otomatis tidak berlaku)
FR-DOC-01	Pengguna dapat mengunggah berkas pada kategori yang diizinkan bagi perannya
FR-DOC-02	Berkas baru bersifat Private secara default
FR-DOC-03	Pengguna dapat menandai berkas sebagai Public; status berkas menjadi <i>Pending Verification</i>
FR-DOC-04	Pengguna dapat melihat daftar, detail, dan mengunduh berkas sesuai matriks akses
FR-DOC-05	Pengguna dapat menghapus berkas sesuai hak hapus perannya
FR-SRC-01	Pencarian kata kunci nama berkas
FR-SRC-02	Filter multi-kriteria: Program Studi (IF, SI, TE), aktor pengunggah, kategori, NIM/NIP, ekstensi, dan status visibilitas
FR-NTF-01	Pengguna dapat melihat, menandai dibaca, dan menghapus notifikasi miliknya

Super Admin

ID	Kebutuhan
FR-SA-01	CRUD seluruh berkas fakultas, termasuk <i>hard delete</i>
FR-SA-02	Menyetujui atau menolak berkas publik
FR-SA-03	Manajemen pengguna (buat, ubah, nonaktifkan, atur ulang kata sandi)
FR-SA-04	Manajemen peran, kategori, dan prodi
FR-SA-05	Melihat log penyimpanan dan statistik sistem
FR-SA-06	Menerima notifikasi antrean verifikasi

Dekan

ID	Kebutuhan
FR-DKN-01	Membaca seluruh berkas fakultas
FR-DKN-02	Mengunggah SK Keputusan Dekan, Dokumen Akreditasi Fakultas, dan Arsip Khusus
FR-DKN-03	Menghapus hanya berkas yang diunggah sendiri
FR-DKN-04	Melihat dasbor statistik tingkat fakultas

BAAK / Staf

ID	Kebutuhan
FR-BAK-01	Membaca dan mengelola berkas per prodi
FR-BAK-02	Mengunggah Legalisir, Transkrip Nilai Akademik, Dokumen BAAK, dan Berkas Mahasiswa/Dosen
FR-BAK-03	Menerima notifikasi berkas publik baru pada antrean verifikasi
FR-BAK-04	Memverifikasi berkas publik (<i>Approved/Rejected</i>), dengan alasan wajib saat <i>Rejected</i>
FR-BAK-05	Menghapus berkas miliknya dan berkas operasional prodi

Kaprodi

ID	Kebutuhan
FR-KPR-01	Membaca berkas dosen se-fakultas dan berkas mahasiswa di prodi sendiri
FR-KPR-02	Mengunggah SK Keputusan Prodi, Dokumen Akreditasi Prodi, Kurikulum, dll.
FR-KPR-03	Menghapus hanya berkas miliknya

Dosen / TA

ID	Kebutuhan
FR-DSN-01	Membaca berkas mahasiswa di prodi sendiri dan berkas pribadi
FR-DSN-02	Mengunggah RPS, SK Mengajar, BAP, dll.
FR-DSN-03	Menghapus hanya berkas miliknya

Mahasiswa

ID	Kebutuhan
FR-MHS-01	Membaca hanya berkas pribadi
FR-MHS-02	Mengunggah Proposal TA/PA, Berkas Yudisium, Surat Keterangan, dll.
FR-MHS-03	Menghapus hanya berkas miliknya

Sistem otomatis

ID	Kebutuhan
FR-SYS-01	Penjadwal memeriksa metadata berkas secara periodik dan membersihkan berkas berumur > 8 tahun: <i>soft delete</i> terlebih dahulu, lalu <i>hard delete</i> setelah masa tenggang 3 bulan (90 hari)
FR-SYS-02	Sistem membuat notifikasi otomatis ke BAAK dan Super Admin saat berkas publik masuk antrean
FR-SYS-03	Sistem memberi tahu pengunggah saat berkasnya <i>Approved</i> atau <i>Rejected</i>
FR-SYS-04	Setiap operasi berkas (unggah, unduh, hapus, pembersihan) dicatat pada <code>storage_logs</code>

B. Kebutuhan Non-Fungsional

ID	Kategori	Kebutuhan
NFR-SEC-01	Keamanan	Seluruh lalu lintas menggunakan HTTPS (TLS 1.2 atau lebih baru)
NFR-SEC-02	Keamanan	Kata sandi disimpan sebagai <i>hash</i> (bcrypt cost $\geq$ 12 atau argon2id), tidak pernah dalam bentuk teks biasa
NFR-SEC-03	Keamanan	Otorisasi diberlakukan di sisi server (<i>middleware</i> dan <i>policy</i>) pada setiap <i>endpoint</i> ; pembatasan di antarmuka Next.js (menu, <i>route guard</i>) hanya untuk kenyamanan dan tidak dianggap kontrol keamanan

ID	Kategori	Kebutuhan
NFR-SEC-04	Keamanan	SRS Sistem Informasi Arsip Berkas Digital FITE <i>Access token</i> berumur 15 menit dan memuat <i>claim sid</i> (ID sesi). <i>Middleware</i> memeriksa bahwa sesi belum dicabut dan akun masih aktif pada setiap permintaan, sehingga <i>logout</i> , penonaktifan akun, dan reset sandi berlaku segera. Algoritma tanda tangan dikunci (HS256 atau lebih kuat); tanda tangan, <i>exp</i> , <i>iss</i> , dan <i>aud</i> selalu diverifikasi. Token tidak dikirim lewat URL
NFR-SEC-05	Keamanan	Pembatasan laju (<i>rate limiting</i>) pada <i>endpoint</i> autentikasi untuk mencegah <i>brute force</i>
NFR-SEC-06	Keamanan	Validasi berkas pada ekstensi dan isi: tipe dideteksi dengan <i>MIME sniffing</i> (<code>http.DetectContentType</code>) serta pemeriksaan struktur untuk berkas berbasis ZIP (<code>docx/xlsx/pptx/zip</code>). Nama berkas disanitasi dan disimpan dengan nama acak (UUID) di GCS
NFR-SEC-07	Keamanan	Bucket GCS bersifat privat; unduhan dialirkan melalui backend setelah pemeriksaan hak akses
NFR-SEC-08	Keamanan	Perlindungan terhadap SQL injection (<i>query</i> terparameter oleh GORM, tanpa penyambungan <i>string</i> SQL), XSS (Next.js melakukan <i>escape</i> otomatis), dan CSRF (<i>refresh token</i> hanya dikirim ke jalur <code>/auth/*</code> dengan <code>SameSite=Strict</code> , dan <i>endpoint</i> lain memakai Bearer di <i>header</i>); CORS dibatasi pada domain frontend dengan <code>Allow-Credentials</code> aktif hanya untuk domain tersebut
NFR-SEC-09	Keamanan	Rahasia (kunci JWT, kredensial basis data, kunci akun layanan GCS) dibaca dari variabel lingkungan atau <i>secret manager</i> , tidak pernah berada di repositori
NFR-SEC-10	Keamanan	Header keamanan HTTP diterapkan (<code>X-Content-Type-Options</code> , <code>Strict-Transport-Security</code> , <code>Content-Security-Policy</code> pada frontend)
NFR-FIL-01	Batas berkas	Ukuran maksimal 10 MB per berkas; batas ditegakkan di tingkat <i>request body</i> (<code>http.MaxBytesReader</code>) dan di validasi aplikasi
NFR-FIL-02	Batas berkas	Format <i>whitelist</i> : <code>.pdf</code> , <code>.docx</code> , <code>.xlsx</code> , <code>.pptx</code> , <code>.zip</code> , <code>.png</code> , <code>.jpg</code> . Video atau berkas besar dilarang; tautannya ditulis pada berkas teks lalu di- <i>zip</i>
NFR-RET-01	Retensi	Berkas dengan tanggal arsip lebih dari 8 tahun dibersihkan otomatis oleh penjadwal
NFR-RET-02	Retensi	Pembersihan menjalankan <i>soft delete</i> terlebih dahulu dan <i>hard delete</i> (basis data dan objek GCS) setelah masa tenggang 90 hari; setiap tindakan dicatat. Masa tenggang dikonfigurasi lewat variabel lingkungan

ID	Kategori	Kebutuhan
NFR-RET-03	Retensi	Penjadwal aman dijalankan pada lebih dari satu instans: hanya satu instans yang memproses pada satu waktu (kunci GET_LOCK MySQL) dan proses bersifat idempoten
NFR-PER-01	Kinerja	Sesuai SLA pada 2.3
NFR-AVL-01	Ketersediaan	Sesuai SLA pada 2.3
NFR-INT-01	Integritas data	Relasi antartabel dijaga oleh <i>foreign key</i> ; metadata berkas dan objek GCS harus konsisten (rekonsiliasi berkala)
NFR-USB-01	Kegunaan	Antarmuka responsif (desktop dan tablet/ponsel) dengan pesan galat berbahasa Indonesia yang jelas
NFR-MNT-01	Pemeliharaan	Kode Go mengikuti gofmt dan lolos golangci-lint; frontend lolos ESLint dan pemeriksaan tipe TypeScript; terdapat pengujian otomatis (go test); API diberi versi (/api/v1)
NFR-MNT-02	Pemeliharaan	Aplikasi mengikuti arsitektur berlapis (<i>handler</i> → <i>service</i> → <i>repository</i>) dan dapat dikemas sebagai kontainer Docker

2.2.2 Hardware & Software Development Tools

Perangkat keras (pengembangan)

Komponen	Spesifikasi Minimum
Prosesor	Setara Intel Core i5 generasi ke-8 / AMD Ryzen 5
Memori	8 GB RAM (disarankan 16 GB)
Penyimpanan	SSD dengan ruang kosong minimal 20 GB
Jaringan	Koneksi internet stabil untuk akses GCS dan repositori paket

Perangkat lunak dan alat bantu

Kategori	Alat	Keterangan
Bahasa/Runtime	Go (versi stabil terbaru, minimal 1.22), Node.js LTS, npm/pnpm	Backend dan build frontend
Framework	Gin, GORM, Next.js (App Router), React, TypeScript	Sesuai stack
Pustaka Go utama	golang-jwt/jwt/v5, golang.org/x/crypto/bcrypt, cloud.google.com/go/storage, robfig/cron/v3, go-playground/validator, golang-migrate/migrate, google/uuid	Autentikasi, hash sandi, GCS, penjadwal, validasi, migrasi
Basis data	MySQL 8.x, MySQL Workbench / DBeaver	Perancangan dan administrasi DB
Penyimpanan cloud	Google Cloud Console, Service Account GCS	Kredensial disimpan di variabel lingkungan, tidak di repositori
Pengujian API	Postman (dan Newman untuk otomatisasi)	Lihat 2.4
Pengujian kode	go test, testify, net/http/httpptest, Vitest/Playwright (frontend)	Uji unit, integrasi, dan E2E
Alat bantu Go	air (hot reload), golangci-lint, swaggo/swag (opsional, dokumentasi OpenAPI)	Produktivitas dan kualitas kode

Kategori	Alat	Keterangan
Kontrol versi	Git dan GitHub/GitLab	Kolaborasi 4 pengembang; <i>branching</i> dan <i>pull request</i>
Editor/IDE	VS Code (ekstensi Go) atau GoLand	
Lingkungan lokal	Docker dan Docker Compose (MySQL, API, frontend)	Lingkungan seragam antar anggota tim
Desain	Figma / draw.io	<i>Wireframe</i> dan ERD
Server target	Linux (Ubuntu LTS), Nginx sebagai <i>reverse proxy</i> dan TLS, <code>systemd</code> atau Docker untuk menjalankan API, <i>worker</i> , dan Next.js	<i>Deployment</i>

2.3 Operational Requirement & SLA

Angka berikut telah disetujui sebagai target rancangan awal.

ID	Aspek	Target	Cara Pengukuran
SLA-RT-01	<i>Response time</i> API non-berkas (login, daftar, detail, verifikasi)	≤ 500 ms pada persentil ke-95 dalam kondisi normal	Postman/Newman, log aplikasi
SLA-RT-02	Pencarian dan filter	≤ 1 detik untuk hingga 100.000 baris metadata (didukung indeks)	Uji beban dan <code>EXPLAIN</code> query
SLA-RT-03	Unggah berkas 10 MB	≤ 15 detik pada koneksi ≥ 5 Mbps	Uji fungsional
SLA-AV-01	Ketersediaan	≥ 99% per bulan pada jam layanan kampus (tidak termasuk pemeliharaan terjadwal)	Pemantauan <i>uptime</i> (<i>endpoint /healthz</i>)
SLA-AV-02	Pemeliharaan terjadwal	Dilakukan di luar jam kerja, dengan pemberitahuan sebelumnya	Kalender pemeliharaan
SLA-DI-01	Integritas data	Tidak ada metadata yatim (<i>orphan</i>) terhadap objek GCS; rekonsiliasi berkala	Skrip rekonsiliasi
SLA-DI-02	<i>Backup</i> basis data	Cadangan harian; RPO ≤ 24 jam; RTO ≤ 8 jam	Prosedur pemulihan diuji berkala
SLA-DI-03	Konsistensi transaksi	Operasi unggah bersifat atomik: bila penyimpanan GCS atau pencatatan DB gagal, tidak boleh tersisa data setengah jadi	Uji integrasi
SLA-SUP-01	Penanganan insiden	Insiden kritis (sistem tidak dapat diakses) direspons ≤ 4 jam kerja	Log insiden

2.4 API Testing Environment (Postman Setup)

A. Struktur Workspace

```
FITE Arsip Digital API
├── Environments
│   ├── Local      (base_url = http://localhost:8080/api/v1)
│   └── Staging    (base_url = https://staging.<domain>/api/v1)
├── Collection: FITE-Arsip-API
│   ├── 01 Auth      (login, refresh, logout, me, profile, password)
│   ├── 02 Documents
│   ├── 03 Verifications
│   └── 04 Notifications
```

13 / 132

Variabel	Contoh Nilai	Keterangan
base_url	http://localhost:8080/api/v1	URL dasar API
token_superadmin	(diisi otomatis)	Access token Super Admin
token_dekan	(diisi otomatis)	Access token Dekan
token_baak	(diisi otomatis)	Access token BAAK
token_kaprodi	(diisi otomatis)	Access token Kaprodi
token_dosen	(diisi otomatis)	Access token Dosen/TA
token_mahasiswa	(diisi otomatis)	Access token Mahasiswa
document_id	(diisi otomatis)	ID berkas hasil unggah
verification_id	(diisi otomatis)	ID antrean verifikasi

C. Header Standard

Header	Nilai
Accept	application/json
Content-Type	application/json (untuk unggah: multipart/form-data)
Authorization	Bearer {{token_<peran>}}

```
pm.test("Status code 200", () => pm.response.to.have.status(200));

const json = pm.response.json();
pm.expect(json.data).to.have.property("access_token");

// Sesuaikan nama variabel dengan akun yang digunakan
pm.environment.set("token_baak", json.data.access_token);

// Refresh token dikirim sebagai cookie httpOnly
pm.test("Refresh cookie terpasang", () => {
  pm.expect(pm.cookies.has("refresh_token")).to.be.true;
});
```

```
pm.test("Response time < 500 ms", () => {
  pm.expect(pm.response.responseTime).to.be.below(500);
});

pm.test("Content-Type JSON", () => {
  pm.response.to.have.header("Content-Type", /application\/json/);
});
```

Basis data pengujian diisi melalui perintah *seeder* (`go run ./cmd/seed`) yang membuat satu akun per aktor (dan minimal satu akun per prodi untuk Kaprodi, Dosen, dan Mahasiswa) agar pembatasan lintas prodi dapat diuji. Kredensial akun uji disimpan hanya pada lingkungan pengujian dan tidak dimasukkan ke repositori publik.

G. Menjalankan Pengujian Otomatis

```
newman run FITE-Arsip-API.postman_collection.json \
-e Local.postman_environment.json \
--reporters cli,htmlextra
```

Pengujian unit dan integrasi Go dijalankan terpisah dengan `go test ./... -race -cover`.

H. Data Uji Berkas

Berkas	Tujuan
valid-9mb.pdf	Batas bawah sukses (< 10 MB)
exact-10mb.pdf	Uji batas tepat 10 MB (10.485.760 <i>byte</i>)
over-10mb.pdf	Harus ditolak (> 10 MB)
script.exe / video.mp4	Harus ditolak (di luar <i>whitelist</i>)
fake.pdf (isi bukan PDF)	Harus ditolak (validasi isi/MIME)
fake.docx (ZIP biasa tanpa struktur Office)	Harus ditolak (validasi struktur berkas ZIP)
arsip-9tahun (data metadata buatan)	Uji retensi penjadwal

3.0 Ketentuan Umum Pengujian

Aspek	Ketentuan
Lingkungan	Postman/Newman (Local/Staging, lihat 2.4), go test untuk uji integrasi, Playwright untuk uji sistem Next.js; basis data diisi <i>seeder</i>
Format respons sukses	<code>{"success":true,"message":"...","data":{"..."},"meta":{"...}}</code>
Format respons galat	<code>{"success":false,"message":"...","errors":{"field":["..."]}}</code> (errors hanya pada 422)
Kode status	200, 201, 204; 401 (token tidak ada/tidak valid/kedaluwarsa/sesi dicabut, atau kredensial salah); 403 (terautentikasi tetapi tidak berwenang, atau akun nonaktif saat login); 404 ; 409 (konflik status); 413 (isi permintaan melebihi batas keras); 415 (tipe konten salah); 422 (validasi gagal); 429 ; 5xx (galat server, pesan generik)
Batas ukuran	10 MB = 10.485.760 byte . Batas keras <i>request body</i> = 11 MB (10 MB + ruang <i>multipart</i>); di atas itu ditolak 413 sebelum diproses
<i>Access token</i>	JWT, umur 15 menit (<code>expires_in</code> = 900 detik), dikirim sebagai <code>Authorization: Bearer</code>
<i>Refresh token</i>	<code>Cookie refresh_token: HttpOnly; Secure; SameSite=Strict; Path=/api/v1/auth</code> , umur 7 hari, dirotasi setiap dipakai
Tanggal arsip	Kolom <code>archived_at</code> pada tabel <code>documents</code> menjadi acuan retensi
Kolom <i>Actual Behavior</i> dan <i>Verdict</i>	Dikosongkan (“-”) pada tahap perancangan; diisi saat eksekusi. Verdict: Pass / Fail / Blocked

Akun uji (seeder):

Kode	Peran	Prodi
SA	Super Admin	(semua)
DKN	Dekan	(semua)
BAK	BAAK/Staf	(semua prodi)
KPR-IF, KPR-SI	Kaprodi	Informatika, Sistem Informasi
DSN-IF, DSN-SI	Dosen/TA	Informatika, Sistem Informasi
MHS-IF-1, MHS-IF-2, MHS-SI-1	Mahasiswa	Informatika, Informatika, Sistem Informasi

3.1 Autentikasi & Authorization

Test Specification

Atribut	Isi
Test ID	TS-AUTH
FR/NFR ID	FR-AUTH-01..04, NFR-SEC-03, NFR-SEC-04, NFR-SEC-05, NFR-SEC-08, NFR-SEC-10, FR-MHS-01, FR-DSN-01, FR-KPR-01, FR-DKN-01
Test Name	Login multi-role, Bearer Token, <i>refresh token</i> , pencabutan sesi, dan pembatasan akses baca berbasis peran

Atribut	Isi
Objective	Memastikan setiap aktor dapat masuk; <i>access</i> dan <i>refresh token</i> dikelola dengan aman (rotasi, deteksi pemakaian ulang, pencabutan segera); dan cakupan baca berkas sesuai matriks akses
Precondition	Seeder telah dijalankan; berkas uji tersedia pada beberapa prodi dan pengunggah berbeda; API Go dan Next.js aktif; konfigurasi umur <i>token</i> dapat diperpendek untuk uji kedaluwarsa
Testing Scenario	Login tiap peran; login gagal; akses <i>endpoint</i> terproteksi tanpa/dengan token tidak valid; <i>refresh</i> dan <i>logout</i> ; manipulasi JWT; perubahan status/peran akun saat sesi berjalan; akses berkas lintas peran dan prodi; perilaku frontend saat token kedaluwarsa
Evaluation Criteria	Kode status dan struktur JSON sesuai harapan; <i>refresh token</i> tidak pernah muncul di isi JSON; token yang dicabut ditolak pada permintaan berikutnya; tidak ada kebocoran data berkas di luar hak akses

Test Case

ID	Input	Expected Behavior	Actual Behavior	Verdict
TC-AUTH-01	POST /auth/login kredensial valid Super Admin	200; data.access_token terisi, token_type = "Bearer", expires_in = 900, data.user.role = super_admin; header Set-Cookie: refresh_token=...; HttpOnly; Secure; SameSite=Strict; Path=/api/v1/auth; isi JSON tidak memuat refresh token	-	-
TC-AUTH-02	Login valid untuk Dekan, BAAK, Kaprodi, Dosen, Mahasiswa (masing-masing)	200; token terbit; role dan prodi pada payload sesuai akun	-	-
TC-AUTH-03	Login dengan kata sandi salah	401; pesan galat generik (tidak menyebut apakah email atau sandi yang salah); tidak ada <i>cookie</i> terpasang	-	-
TC-AUTH-04	Login dengan email tidak terdaftar (≥ 30 percobaan, dibandingkan dengan sandi salah pada email terdaftar)	401 dengan pesan identik dengan TC-AUTH-03; selisih rata-rata waktu respons < 50 ms (perbandingan <i>hash</i> semu dijalankan agar tidak membocorkan keberadaan akun)	-	-
TC-AUTH-05	Login dengan field kosong atau email berformat salah	422; <i>errors</i> memuat field yang wajib/valid	-	-
TC-AUTH-06	Login akun berstatus nonaktif (kredensial benar)	403; token dan <i>cookie</i> tidak diterbitkan	-	-

ID	Input	Expected Behavior	Actual Behavior	Verdict
TC-AUTH-07	6 kali login gagal berturut-turut untuk kombinasi email+IP yang sama dalam 1 menit (ambang 5)	Permintaan ke-6 mendapat 429 dengan header <code>Retry-After</code> ; login benar setelah jeda berlalu berhasil	-	-
TC-AUTH-08	GET <code>/documents</code> tanpa header <code>Authorization</code>	401; tidak ada data berkas	-	-
TC-AUTH-09	GET <code>/documents</code> dengan Bearer rusak (<code>Bearer abc</code> , <code>Bearer</code> kosong, skema <code>Basic</code>)	401 untuk semuanya	-	-
TC-AUTH-10	POST <code>/auth/logout</code> dengan <code>access token</code> valid, lalu GET <code>/auth/me</code> dengan token yang sama, lalu POST <code>/auth/refresh</code> dengan <code>cookie</code> lama	Logout 200 dan <code>cookie</code> dihapus (<code>Max-Age=0</code>); permintaan GET <code>/auth/me</code> 401 (sesi dicabut walau token belum kedaluwarsa); <code>refresh</code> 401	-	-
TC-AUTH-11	Gunakan <code>access token</code> yang telah melewati masa berlaku (umur token diperpendek pada lingkungan uji)	401	-	-
TC-AUTH-12	MHS-IF-1 GET <code>/documents/{id}</code> milik sendiri	200; detail berkas tampil	-	-
TC-AUTH-13	MHS-IF-1 GET <code>/documents/{id}</code> milik MHS-IF-2 (Private)	403	-	-
TC-AUTH-14	MHS-IF-1 GET <code>/documents</code> (daftar)	200; hanya berkas milik sendiri (berkas publik hanya lewat GET <code>/documents/public</code>)	-	-
TC-AUTH-15	DSN-IF membuka berkas mahasiswa prodi IF	200	-	-
TC-AUTH-16	DSN-IF membuka berkas mahasiswa prodi SI	403	-	-
TC-AUTH-17	KPR-IF membuka berkas dosen prodi SI (berkas dosen se-fakultas)	200	-	-
TC-AUTH-18	KPR-IF membuka berkas mahasiswa prodi SI	403	-	-
TC-AUTH-19	DKN membuka berkas dari semua prodi dan semua aktor	200 untuk seluruh sampel	-	-
TC-AUTH-20	MHS mengakses GET <code>/users</code> (manajemen pengguna)	403	-	-
TC-AUTH-21	BAK mengakses GET <code>/users</code>	403 (hanya Super Admin)	-	-

ID	Input	Expected Behavior	Actual Behavior	Verdict
TC-AUTH-22	PUT /auth/password dengan sandi lama benar dan sandi baru valid	200; login dengan sandi baru berhasil, sandi lama gagal; sesi lain milik pengguna dicabut, sesi yang sedang dipakai tetap berlaku	-	-
TC-AUTH-23	Manipulasi ID pada URL (<i>IDOR</i>): MHS mengubah {id} secara berurutan	Setiap ID milik orang lain 403; tidak ada perbedaan pesan yang membocorkan keberadaan berkas	-	-
TC-AUTH-24	POST /auth/refresh dengan <i>cookie</i> valid	200; <i>access_token</i> baru; <i>cookie</i> <i>refresh_token</i> baru terpasang (rotasi); <i>refresh token</i> lama tidak dapat dipakai lagi	-	-
TC-AUTH-25	Setelah TC-AUTH-24, kirim kembali <i>refresh token</i> lama (deteksi pemakaian ulang)	401; seluruh rangkaian sesi dicabut: <i>refresh token</i> baru dan <i>access token</i> sesi tersebut juga 401	-	-
TC-AUTH-26	POST /auth/refresh tanpa <i>cookie</i>	401	-	-
TC-AUTH-27	POST /auth/refresh dengan <i>refresh token</i> yang telah melewati 7 hari (data uji diatur)	401	-	-
TC-AUTH-28	POST /auth/refresh setelah logout	401	-	-
TC-AUTH-29	Manipulasi JWT: (a) alg: none, (b) tanda tangan diubah, (c) <i>claim</i> role/sub diubah, (d) iss/aud salah	401 untuk semua varian	-	-
TC-AUTH-30	SA menonaktifkan akun DSN-IF saat DSN-IF masih memegang <i>access token</i> yang belum kedaluwarsa; DSN-IF GET /documents lalu POST /auth/refresh	Kedua permintaan 401 (sesi dicabut seketika)	-	-
TC-AUTH-31	SA mengubah peran KPR-IF menjadi Dosen; KPR-IF memakai <i>access token</i> lama untuk mengakses berkas dosen prodi SI	Hak akses mengikuti peran terbaru dari basis data (bukan <i>claim</i> di token): akses baru ditolak 403	-	-
TC-AUTH-32	SA mengatur ulang kata sandi sebuah akun yang sedang login	Seluruh sesi akun tersebut dicabut; token lama 401; <i>must_change_password</i> = true pada login berikutnya	-	-

ID	Input	Expected Behavior	Actual Behavior	Verdict
TC-AUTH-33	CORS: (a) OPTIONS <i>preflight</i> dari <i>origin</i> frontend yang diizinkan; (b) permintaan dari <i>origin</i> lain	(a) 204 dengan Access-Control-Allow-Origin tepat sama dengan <i>origin</i> frontend, Allow-Credentials: true, metode dan header yang diizinkan; (b) tanpa header Access-Control-Allow-Origin	-	-
TC-AUTH-34	Periksa header keamanan pada respons API dan halaman Next.js	X-Content-Type-Options: nosniff, Strict-Transport-Security (di lingkungan HTTPS), dan Content-Security-Policy pada halaman frontend ada	-	-
TC-AUTH-35	(Sistem, Next.js) <i>Access token</i> kedaluwarsa saat pengguna mengklik menu	Frontend memanggil /auth/refresh secara otomatis, mengulang permintaan sekali, dan halaman tampil tanpa pengguna diarahkan ke login	-	-
TC-AUTH-36	(Sistem, Next.js) <i>refresh</i> gagal (cookie hilang/dicabut)	Memori token dikosongkan dan pengguna diarahkan ke halaman login dengan pesan sesi berakhir; tidak terjadi perulangan <i>refresh</i> tanpa batas	-	-
TC-AUTH-37	(Sistem, Next.js) MHS membuka URL halaman admin (/admin/pengguna) secara manual	Frontend menampilkan halaman “tidak berwenang” atau mengalihkan; bila pembatas dilewati, API tetap 403 (TC-AUTH-20)	-	-
TC-AUTH-38	(Sistem, Next.js) Periksa localStorage, sessionStorage, dan <i>cookie</i> yang terbaca JavaScript setelah login	<i>Access token</i> tidak tersimpan pada penyimpanan web; <i>refresh token</i> tidak terlihat oleh JavaScript (httpOnly)	-	-

3.2 Upload Berkas

Test Specification

Atribut	Isi
Test ID	TS-UPL
FR/NFR ID	FR-DOC-01, FR-DOC-02, FR-DOC-03, NFR-FIL-01, NFR-FIL-02, NFR-SEC-06, SLA-DI-03, SLA-RT-03
Test Name	Validasi ukuran 10 MB, whitelist format (ekstensi dan isi), kategori per peran, dan visibilitas awal
Objective	Memastikan hanya berkas valid yang tersimpan, kategori sesuai peran, visibilitas default Private, dan operasi unggah atomik

Atribut	Isi
Precondition	Pengguna terautentikasi; berkas uji tersedia (lihat 2.4-H); GCS dapat diakses (atau di- <i>mock</i> untuk uji kegagalan)
Testing Scenario	Unggah berkas valid; unggah pada batas ukuran dan jauh di atas batas; unggah format terlarang; unggah dengan isi tidak sesuai ekstensi; unggah pada kategori yang bukan hak perannya; unggah dengan <code>visibility=public</code> ; simulasi kegagalan GCS/DB; unggah paralel
Evaluation Criteria	Berkas valid: 201, objek ada di GCS, baris metadata ada, log tercatat. Berkas tidak valid: ditolak (422/403/413/415) tanpa objek GCS maupun metadata tersisa; penggunaan memori server tetap stabil

Test Case

ID	Input	Expected Behavior	Actual Behavior	Verdict
TC-UPL-01	MHS mengunggah <code>valid-9mb.pdf</code> , kategori “Proposal TA/PA”, tanpa field visibilitas	201; <code>visibility = private</code> ; <code>verification_status = null</code> ; objek tersimpan di GCS dengan nama UUID	-	-
TC-UPL-02	Unggah <code>exact-10mb.pdf</code> (10.485.760 <i>byte</i> tepat)	201 (batas diterima)	-	-
TC-UPL-03	Unggah berkas 10 MB + 1 <i>byte</i>	422; <code>errors.file</code> menyebut batas 10 MB; tidak ada objek di GCS	-	-
TC-UPL-04	Kirim <i>body</i> 50 MB (jauh di atas batas keras 11 MB)	413; koneksi ditutup tanpa menampung seluruh <i>body</i> di memori; tidak ada objek di GCS	-	-
TC-UPL-05	Unggah tujuh berkas asli untuk tiap format whitelist: <code>.pdf</code> , <code>.docx</code> , <code>.xlsx</code> , <code>.pptx</code> , <code>.zip</code> , <code>.png</code> , <code>.jpg</code>	201 untuk ketujuhnya	-	-
TC-UPL-06	Unggah <code>script.exe</code>	422; format ditolak	-	-
TC-UPL-07	Unggah <code>video.mp4</code>	422; pesan menyarankan menaruh tautan di berkas teks lalu di- <i>zip</i>	-	-
TC-UPL-08	Unggah <code>fake.pdf</code> (ekstensi <code>.pdf</code> , isi bukan PDF)	422; ditolak oleh deteksi isi (<i>sniffing</i>)	-	-
TC-UPL-09	Unggah <code>fake.docx</code> (ZIP biasa tanpa struktur dokumen Office)	422; ditolak oleh pemeriksaan struktur berkas ZIP	-	-
TC-UPL-10	Unggah berkas PDF asli yang diubah namanya menjadi <code>.docx</code>	422; ketidaksesuaian ekstensi dan isi terdeteksi	-	-
TC-UPL-11	Unggah <code>laporan.pdf.exe</code> (ekstensi ganda)	422	-	-
TC-UPL-12	Unggah foto <code>.jpeg</code> asli	201; ekstensi disimpan sebagai <code>jpg</code> (dinormalisasi)	-	-

ID	Input	Expected Behavior	Actual Behavior	Verdict
TC-UPL-13	Unggah .gif, .txt, .rar, .svg, .html	422 untuk semuanya (SVG dan HTML ditolak karena berisiko XSS)	-	-
TC-UPL-14	Unggah tanpa lampiran berkas	422; errors.file wajib	-	-
TC-UPL-15	Unggah tanpa kategori atau dengan category_id tidak ada	422	-	-
TC-UPL-16	MHS mengunggah pada kategori “SK Keputusan Dekan”	403; berkas tidak tersimpan	-	-
TC-UPL-17	DSN mengunggah “RPS” dan “BAP”	201	-	-
TC-UPL-18	BAK mengunggah “Transkrip Nilai Akademik” dengan subject_user_id = MHS-SI-1	201; owner = MHS-SI-1; prodi = SI; pengunggah = BAK	-	-
TC-UPL-19	DKN mengunggah “Dokumen Akreditasi Fakultas”	201; prodi = null (berkas fakultas)	-	-
TC-UPL-20	MHS mengunggah dengan visibility=public	201; visibility = public; verification_status = pending; baris verifikasi terbentuk; notifikasi terkirim ke BAK dan SA	-	-
TC-UPL-21	DKN mengunggah dengan visibility=public	201; status <i>pending</i> dan masuk antrean (semua berkas Public wajib diverifikasi)	-	-
TC-UPL-22	Nama berkas berisi karakter berbahaya (../../etc/passwd.pdf, <script>.pdf, nama sangat panjang, karakter Unicode kontrol)	Nama disanitasi atau dipotong; berkas disimpan dengan UUID; tidak ada <i>path traversal</i> ; respons dan unduhan memakai nama aman	-	-
TC-UPL-23	Simulasikan GCS gagal (kredensial dicabut/di- <i>mock</i>) saat unggah	5xx dengan pesan galat umum; tidak ada baris metadata yatim	-	-
TC-UPL-24	Simulasikan penyimpanan metadata ke MySQL gagal setelah objek terunggah ke GCS	5xx; objek GCS yang sudah terunggah dihapus kembali (kompensasi); tidak ada objek yatim	-	-
TC-UPL-25	Unggah tanpa token	401	-	-
TC-UPL-26	Unggah dua berkas bernama sama oleh pengguna yang sama	Keduanya tersimpan sebagai dua rekaman berbeda (UUID berbeda)	-	-
TC-UPL-27	Non-BAK/SA mengirim field subject_user_id atau archived_at	422 (field dilarang untuk perannya); tidak ada berkas tersimpan	-	-

ID	Input	Expected Behavior	Actual Behavior	Verdict
TC-UPL-28	Unggah dengan Content-Type: application/json (bukan multipart/form-data)	415	-	-
TC-UPL-29	20 unggahan paralel berkas 9 MB oleh akun berbeda	Semua 201; setiap metadata menunjuk objek yang benar (tidak tertukar); penggunaan memori tidak tumbuh sebanding 20 × 9 MB (berkas dialirkan ke GCS, tidak seluruhnya dibaca ke memori)	-	-

3.3 Filter & Multi-Criteria Search

Test Specification

Atribut	Isi
Test ID	TS-SRC
FR/NFR ID	FR-SRC-01, FR-SRC-02, FR-DOC-04, NFR-SEC-08, SLA-RT-02
Test Name	Pencarian kata kunci dan filter multi-kriteria
Objective	Memastikan hasil pencarian akurat, terbatas pada hak akses pengguna, dan aman dari injeksi
Precondition	Basis data uji memuat ≥ 200 berkas campuran (prodi, kategori, ekstensi, visibilitas, pengunggah); indeks terpasang
Testing Scenario	Kirim GET /documents dengan berbagai parameter <i>query</i> tunggal dan kombinasi; verifikasi isi hasil dan waktu respons
Evaluation Criteria	Seluruh hasil memenuhi semua filter yang dikirim; tidak ada berkas di luar hak akses; respons ≤ 1 detik (SLA-RT-02)

Parameter query: q, prodi (IF/SI/TE), uploader_role, category_id, nim, nip, extension, visibility (private / public_approved), sort, page, per_page.

Test Case

ID	Input	Expected Behavior	Actual Behavior	Verdict
TC-SRC-01	BAK: ?q=transkrip	200; semua nama berkas memuat “transkrip” (tidak peka huruf besar-kecil)	-	-
TC-SRC-02	BAK: ?prodi=SI	200; seluruh hasil berprodi SI	-	-
TC-SRC-03	BAK: ?uploader_role=dosen	200; seluruh pengunggah berperan Dosen	-	-
TC-SRC-04	BAK: ?category_id={Legalisir}	200; seluruh hasil berkategori Legalisir	-	-
TC-SRC-05	BAK: ?nim={NIM valid}	200; hanya berkas milik mahasiswa dengan NIM tersebut (termasuk yang diunggah BAAK atas namanya)	-	-

ID	Input	Expected Behavior	Actual Behavior	Verdict
TC-SRC-06	KPR: ?nip={NIP dosen valid}	200; hanya berkas milik dosen tersebut	-	-
TC-SRC-07	BAK: ?extension=pdf dan ?extension=xlsx	200; hasil sesuai ekstensi	-	-
TC-SRC-08	BAK: ?visibility=public_approved	200; hanya berkas Public berstatus Approved (tidak ada <i>pending/rejected</i>)	-	-
TC-SRC-09	BAK: ?visibility=private	200; hanya berkas Private	-	-
TC-SRC-10	BAK: ?prodi=IF&category_id={Transkrip}&extension=pdf&q=2024	200; setiap baris memenuhi semua kriteria (logika AND)	-	-
TC-SRC-11	BAK: ?q=zzzzzz_tidak_ada	200; data = [] ; meta.total = 0	-	-
TC-SRC-12	MHS-IF-1: ?nim={NIM MHS-IF-2}	200 dengan hasil kosong; tidak ada berkas milik MHS-IF-2 yang bocor	-	-
TC-SRC-13	DSN-IF: ?prodi=SI	Tidak ada berkas mahasiswa SI dalam hasil (tetap dibatasi hak akses)	-	-
TC-SRC-14	DKN: ?prodi=TE	200; berkas TE tampil (Dekan berhak baca semua)	-	-
TC-SRC-15	BAK: ?page=2&per_page=20	200; meta memuat current_page, per_page, total, last_page; tidak ada duplikasi antarhalaman	-	-
TC-SRC-16	BAK: ?per_page=100000 dan ?per_page=0	422 (batas 1 sampai 100)	-	-
TC-SRC-17	BAK: ?q=' OR '1'='1 dan ?q=%27;DROP TABLE documents; --	200 atau 422; tidak ada galat SQL; tabel tidak berubah (<i>query</i> terparameter)	-	-
TC-SRC-18	BAK: ?prodi=XX (nilai di luar enum)	422	-	-
TC-SRC-19	Berkas yang di- <i>soft delete</i>	Tidak muncul pada hasil pencarian pengguna biasa	-	-
TC-SRC-20	Pencarian kombinasi pada 100.000 baris metadata (data uji beban)	Waktu respons $\leq$ 1 detik (persentil ke-95); bukti EXPLAIN dilampirkan	-	-
TC-SRC-21	BAK: ?q=100% dan ?q=a_b	Karakter % dan _ diperlakukan sebagai teks biasa (tidak menjadi <i>wildcard</i> SQL); hasil hanya berkas yang memuat teks tersebut	-	-

ID	Input	Expected Behavior	Actual Behavior	Verdict
TC-SRC-22	BAK: ?sort=-size_kb, lalu ?sort=password	Pertama: 200 dan terurut menurun menurut ukuran; kedua: 422 (kolom tidak ada dalam daftar urut yang diizinkan)	-	-

3.4 Verifikasi Berkas Publik

Test Specification

Atribut	Isi
Test ID	TS-VER
FR/NFR ID	FR-DOC-03, FR-BAK-03, FR-BAK-04, FR-SA-02, FR-SA-06, FR-SYS-02, FR-SYS-03
Test Name	Alur Pending Verification menjadi Approved/Rejected oleh Admin BAAK dan Super Admin
Objective	Memastikan transisi status berjalan benar, hanya verifikator berwenang yang dapat memutuskan, keputusan bersamaan tidak menghasilkan data ganda, dan notifikasi terkirim
Precondition	Berkas Public berstatus <i>pending</i> tersedia; akun BAK, SA, dan aktor non-verifikator siap
Testing Scenario	Unggah berkas Public; cek notifikasi verifikator; setuju/tolak; cek visibilitas di explorer publik; cek notifikasi ke pengunggah; coba verifikasi oleh aktor tidak berwenang; keputusan bersamaan
Evaluation Criteria	Status berubah sesuai keputusan; hanya berkas <i>Approved</i> tampil di explorer publik; alasan penolakan wajib; notifikasi tepat sasaran

Test Case

ID	Input	Expected Behavior	Actual Behavior	Verdict
TC-VER-01	MHS mengunggah berkas <code>visibility=public</code> ; kemudian BAK dan SA GET <code>/notifications</code>	Notifikasi “berkas publik baru” muncul pada BAK dan SA; tidak muncul pada DSN/KPR/DKN	-	-
TC-VER-02	BAK GET <code>/verifications?status=pending</code>	200; berkas dari TC-VER-01 ada dalam antrean	-	-
TC-VER-03	Sebelum diverifikasi: MHS lain GET <code>/documents/public</code>	Berkas <i>pending</i> tidak tampil	-	-
TC-VER-04	BAK PATCH <code>/verifications/{id}{ "status": "approved" }</code>	200; status <code>approved</code> ; <code>verified_by</code> dan <code>verified_at</code> terisi	-	-
TC-VER-05	Setelah TC-VER-04: pengguna terautentikasi lain (mis. MHS-SI-1) membuka explorer publik	Berkas tampil dan dapat diunduh	-	-

ID	Input	Expected Behavior	Actual Behavior	Verdict
TC-VER-06	Setelah TC-VER-04: pengunggah menerima notifikasi	Notifikasi “berkas disetujui” diterima pengunggah	-	-
TC-VER-07	BAK PATCH <code>{"status": "rejected", "reason": "Berkas tidak sesuai ketentuan"}</code>	200; status <code>rejected</code> ; alasan tersimpan	-	-
TC-VER-08	Setelah TC-VER-07: cek explorer publik dan notifikasi pengunggah	Berkas tidak tampil publik; pengunggah menerima notifikasi berisi alasan; berkas tetap dapat diakses pemilik dan yang berhak (Private)	-	-
TC-VER-09	BAK PATCH <code>{"status": "rejected"}</code> tanpa <code>reason</code>	422; <code>errors.reason</code> wajib	-	-
TC-VER-10	SA PATCH <code>{"status": "approved"}</code> pada berkas <i>pending</i>	200; berhasil (Super Admin berwenang)	-	-
TC-VER-11	DSN, KPR, DKN, dan MHS masing-masing PATCH <code>/verifications/{id}</code>	403 untuk semuanya	-	-
TC-VER-12	BAK menyetujui berkas yang sudah <i>approved</i>	409; status tidak berubah	-	-
TC-VER-13	PATCH dengan <code>status</code> tidak valid (<code>"maybe"</code>)	422	-	-
TC-VER-14	PATCH <code>/verifications/999999</code> (tidak ada)	404	-	-
TC-VER-15	BAK dan SA menyetujui/menolak berkas yang sama bersamaan (dua permintaan paralel dengan keputusan berbeda)	Hanya satu keputusan tersimpan; permintaan lain 409; tidak ada data ganda; status akhir konsisten dengan keputusan yang menang	-	-
TC-VER-16	Pengunggah mengembalikan visibilitas dari Public ke Private saat masih <i>pending</i>	Berkas keluar dari antrean; tidak ada verifikasi atau notifikasi menggantung	-	-
TC-VER-17	Berkas <i>approved</i> diubah menjadi Private oleh pemilik	Berkas hilang dari explorer publik; verifikasi lama dibersihkan	-	-
TC-VER-18	Berkas <i>rejected</i> diajukan Public kembali oleh pemilik	Verifikasi baru berstatus <i>pending</i> (alasan lama dikosongkan); notifikasi baru terkirim ke BAK dan SA	-	-
TC-VER-19	Menolak dengan <code>reason</code> kurang dari 5 karakter atau lebih dari 500 karakter	422	-	-

3.5 Delete Authorization

Test Specification

Atribut	Isi
Test ID	TS-DEL
FR/NFR ID	FR-DOC-05, FR-SA-01, FR-DKN-03, FR-BAK-05, FR-KPR-03, FR-DSN-03, FR-MHS-03, FR-SYS-04, NFR-SEC-03
Test Name	Otorisasi hapus: pengunggah versus aktor tidak berwenang
Objective	Memastikan hak hapus sesuai matriks: pemilik menghapus miliknya, BAAK menghapus berkas operasional prodi, Super Admin menghapus semua (termasuk <i>hard delete</i>)
Precondition	Berkas milik masing-masing aktor tersedia
Testing Scenario	Setiap aktor mencoba menghapus berkas miliknya dan berkas milik aktor lain
Evaluation Criteria	Penghapusan sah: 200/204, berkas hilang dari daftar, log tercatat. Tidak sah: 403, berkas utuh

Test Case

ID	Input	Expected Behavior	Actual Behavior	Verdict
TC-DEL-01	MHS-IF-1 DELETE /documents/{id} berkas miliknya	204; berkas tidak tampil lagi; log storage_logs mencatat penghapusan	-	-
TC-DEL-02	MHS-IF-1 menghapus berkas milik MHS-IF-2	403; berkas utuh	-	-
TC-DEL-03	MHS menghapus berkas milik dosen	403	-	-
TC-DEL-04	DSN-IF menghapus berkas miliknya	204	-	-
TC-DEL-05	DSN-IF menghapus berkas mahasiswa prodi IF (bisa dibaca, bukan miliknya)	403	-	-
TC-DEL-06	KPR-IF menghapus berkas miliknya	204	-	-
TC-DEL-07	KPR-IF menghapus berkas dosen atau mahasiswa	403	-	-
TC-DEL-08	DKN menghapus berkas yang diunggahnya sendiri	204	-	-
TC-DEL-09	DKN menghapus berkas milik aktor lain (walau hak baca semua)	403	-	-
TC-DEL-10	BAK menghapus berkas miliknya	204	-	-
TC-DEL-11	BAK menghapus berkas operasional prodi (Legalisir, Transkrip, Dokumen BAAK, Berkas Mahasiswa/Dosen)	204	-	-
TC-DEL-12	BAK menghapus berkas pribadi milik Dekan	403	-	-

ID	Input	Expected Behavior	Actual Behavior	Verdict
TC-DEL-13	SA menghapus berkas milik aktor mana pun (<i>soft delete</i>)	204	-	-
TC-DEL-14	SA menjalankan <i>hard delete</i> (DELETE /documents/{id}?force=true)	204; baris metadata dan objek GCS terhapus permanen; log tercatat	-	-
TC-DEL-15	Non-Super Admin memakai ?force=true pada berkas miliknya	403 (<i>hard delete</i> hanya Super Admin); berkas utuh	-	-
TC-DEL-16	DELETE tanpa token	401	-	-
TC-DEL-17	DELETE /documents/999999 (tidak ada)	404	-	-
TC-DEL-18	DELETE berkas yang sudah di- <i>soft delete</i> oleh pengguna biasa	404	-	-
TC-DEL-19	Hapus berkas <i>pending</i> verifikasi oleh pemiliknya	Berkas terhapus dan baris antrean verifikasi ikut dibersihkan; notifikasi antrean tidak menunjuk berkas hilang	-	-
TC-DEL-20	Simulasikan GCS gagal saat <i>hard delete</i>	Galat ditangani; metadata tidak hilang lebih dulu (tidak ada objek yatim di GCS); operasi dapat diulang	-	-

3.6 File Retention Check

Test Specification

Atribut	Isi
Test ID	TS-RET
FR/NFR ID	FR-SYS-01, FR-SYS-04, NFR-RET-01, NFR-RET-02, NFR-RET-03, SLA-DI-01
Test Name	Pembersihan otomatis arsip lebih dari 8 tahun (<i>soft delete</i> , masa tenggang 3 bulan, lalu <i>hard delete</i>) oleh proses <i>worker</i>
Objective	Memastikan penjadwal menandai berkas berumur > 8 tahun, menahannya selama masa tenggang 90 hari, lalu menghapusnya permanen beserta objek GCS, dengan aman dijalankan pada lebih dari satu instans
Precondition	Data uji dengan <code>archived_at</code> dan <code>deleted_at</code> diatur melalui <i>seeder</i> atau SQL (tidak perlu mengubah jam sistem): 7 tahun 11 bulan, tepat 8 tahun, 8 tahun 1 hari, serta berkas <i>soft-deleted</i> berumur 89 dan 91 hari; perintah <code>worker --once</code> (menjalankan satu putaran retensi lalu keluar) tersedia
Testing Scenario	Jalankan putaran retensi pada data uji; periksa status metadata, objek GCS, dan <code>storage_logs</code> ; jalankan dua <i>worker</i> bersamaan; simulasikan kegagalan
Evaluation Criteria	Hanya berkas yang memenuhi syarat yang diproses; proses idempoten; setiap tindakan tercatat; tidak ada objek yatim

Aturan yang diuji: berkas dengan `archived_at` lebih dari 8 tahun (dihitung menurut kalender) ditandai *soft delete* (`deleted_at` diisi, `retention_status = expired`). Setelah 90 hari sejak `deleted_at`, berkas dihapus

permanen (*hard delete*) dari basis data dan GCS.

Test Case

ID	Input	Expected Behavior	Actual Behavior	Verdict
TC-RET-01	Berkas <code>archived_at</code> = 7 tahun 11 bulan lalu; jalankan putaran retensi	Tidak diubah	-	-
TC-RET-02	Berkas <code>archived_at</code> = tepat 8 tahun lalu	Tidak diubah (batas: hanya yang <i>lebih dari</i> 8 tahun)	-	-
TC-RET-03	Berkas <code>archived_at</code> = 8 tahun 1 hari lalu	Ditandai <i>soft delete</i> ; <code>deleted_at</code> terisi; objek GCS masih ada; log <code>expired_soft_delete</code> tercatat	-	-
TC-RET-04	Setelah TC-RET-03: pengguna biasa mencari/membuka berkas tersebut	Tidak tampil (404 untuk akses langsung)	-	-
TC-RET-05	Berkas <i>soft-deleted</i> berumur 89 hari; jalankan putaran retensi	Tidak dihapus permanen; objek GCS masih ada	-	-
TC-RET-06	Berkas <i>soft-deleted</i> karena kedaluwarsa berumur 91 hari; jalankan putaran retensi	Dihapus permanen: baris metadata dan objek GCS hilang; log <code>expired_hard_delete</code> tercatat	-	-
TC-RET-07	Jalankan putaran retensi dua kali berturut-turut pada data yang sama	Hasil identik; tidak ada galat atau log ganda yang tidak semestinya (idempoten)	-	-
TC-RET-08	Batch 500 berkas kedaluwarsa	Semua diproses secara bertahap (per <i>chunk</i>); waktu proses wajar; penggunaan memori tidak membesar sebanding jumlah berkas	-	-
TC-RET-09	Objek GCS gagal dihapus pada berkas berumur 91 hari	Metadata tidak dihapus; kegagalan dicatat; dicoba lagi pada putaran berikutnya; tidak ada objek yatim	-	-
TC-RET-10	Berkas kedaluwarsa yang masih berstatus <i>pending verification</i>	Antrean verifikasi ikut dibersihkan; notifikasi antrean terkait tidak menunjuk berkas hilang	-	-
TC-RET-11	Jalankan <i>worker</i> dalam mode normal dan periksa log awal	Tugas retensi terdaftar pukul 02:00 WIB (zona waktu terkonfigurasi <code>Asia/Jakarta</code>); log mencatat waktu eksekusi berikutnya	-	-
TC-RET-12	Berkas dengan <code>archived_at</code> kosong (NULL)	Tidak diproses secara keliru; dicatat sebagai data anomali untuk ditinjau	-	-

ID	Input	Expected Behavior	Actual Behavior	Verdict
TC-RET-13	Setelah pembersihan: rekonsiliasi jumlah objek GCS versus baris metadata	Tidak ada selisih (SLA-DI-01)	-	-
TC-RET-14	Berkas <code>archived_at</code> = 29 Februari 2020; putaran dijalankan pada 1 Maret 2028 (waktu uji disuntikkan)	Dianggap lebih dari 8 tahun menurut kalender; diproses secara benar tanpa selisih hari kabisat	-	-
TC-RET-15	Dua proses <i>worker</i> dijalankan bersamaan pada data kedaluwarsa yang sama	Hanya satu proses yang bekerja (kunci <code>GET_LOCK MySQL</code>); proses lain keluar tanpa mengubah data; tidak ada log atau penghapusan ganda	-	-
TC-RET-16	<i>Worker</i> dihentikan paksa di tengah batch	Putaran berikutnya melanjutkan sisa data; tidak ada duplikasi, kehilangan data, atau objek yatim	-	-
TC-RET-17	Sesi (<i>refresh token</i>) yang telah kedaluwarsa atau dicabut lebih dari 30 hari	Dibersihkan oleh <i>worker</i> ; sesi aktif tidak terpengaruh	-	-
TC-RET-18	Berkas yang dihapus pengguna (bukan kedaluwarsa) berumur 91 hari sejak <code>deleted_at</code>	Dihapus permanen; log <code>deleted_hard_delete</code> tercatat (dibedakan dari <code>expired_hard_delete</code>)	-	-

3.7 Ringkasan Cakupan

Area	Jumlah Test Case	FR/NFR Utama
3.1 Autentikasi & Authorization	38	FR-AUTH-01..04, NFR-SEC-03/04/05/08/10
3.2 Upload Berkas	29	FR-DOC-01..03, NFR-FIL-01/02, NFR-SEC-06
3.3 Filter & Search	22	FR-SRC-01/02
3.4 Verifikasi Berkas Publik	19	FR-BAK-03/04, FR-SA-02
3.5 Delete Authorization	20	FR-DOC-05, FR-*-Delete
3.6 File Retention	18	FR-SYS-01, NFR-RET-01/02/03
Total	146	

Keputusan yang dikunci untuk Bab 4, 5, dan 6:

- Token dan sesi.** Respons login berisi `access_token`, `token_type`, `expires_in`. *Refresh token* hanya lewat *cookie*. Bab 4 menambah `POST /auth/refresh`; Bab 6 menambah tabel sesi.
- Peran dan status akun dibaca dari basis data pada setiap permintaan**, bukan dipercaya dari *claim* JWT (TC-AUTH-30 dan TC-AUTH-31).
- Batas unggah berlapis:** 413 untuk *body* > 11 MB, 422 untuk berkas > 10 MB, dan 415 untuk tipe konten salah (TC-UPL-03, 04, 28).
- Penolakan akses berkas memakai 403**, bukan 404. Bila tim memilih 404 untuk mencegah penebakan ID, ubah TC-AUTH-13, 16, 18, 23 secara konsisten.
- Penjadwal berjalan sebagai proses *worker*** dengan mode `--once` untuk pengujian, zona waktu Asia/Jakarta, dan kunci `GET_LOCK` agar aman multi-instans.

6. **Pembatasan laju login berada di memori per instans.** Bila API dijalankan lebih dari satu instans, TC-AUTH-07 hanya benar per instans; untuk lebih dari satu instans perlu penghitung bersama (Redis atau tabel MySQL).
7. *Endpoint* GET `/dashboard/storage-logs` untuk SA (FR-SA-05), dan penghapusan permanen setelah 90 hari berlaku untuk semua berkas *soft-deleted* (TC-RET-18).

Bab ini mengikuti keputusan Bab 3: pasangan *access* dan *refresh token*, batas unggah berlapis (413/415/422), peran dan status akun dibaca dari basis data pada setiap permintaan, dan 403 untuk akses tidak berwenang. Total **44 endpoint**.

4.0 Konvensi Umum

Aspek	Ketentuan
Base URL	{{base_url}} = https://<domain>/api/v1 (lokal: http://localhost:8080/api/v1)
Format data	JSON (application/json, UTF-8); unggah berkas memakai multipart/form-data. Isi JSON dengan <i>field</i> yang tidak dikenal ditolak dengan 422
Header wajib	Accept: application/json; Authorization: Bearer <access_token> untuk seluruh <i>endpoint</i> kecuali POST /auth/login dan POST /auth/refresh
Header respons	Setiap respons memuat X-Request-ID (dipakai untuk pelacakan log); respons berkas memuat X-Content-Type-Options: nosniff
Penamaan	Sumber daya jamak huruf kecil (/documents), kebab-case untuk aksi, snake_case untuk <i>field</i> JSON
Tipe ID	id numerik (int64) untuk semua tabel, kecuali notifications.id yang berupa UUID string
Pagination	?page=1&per_page=15 (default 15, minimum 1, maksimum 100); respons memuat meta
Waktu	ISO 8601 UTC (contoh 2026-10-07T08:15:00Z)
Rate limit	POST /auth/login: 5 percobaan gagal per menit per kombinasi email+IP (percobaan ke-6 mendapat 429 + Retry-After). <i>Endpoint</i> lain: 60 permintaan per menit per pengguna. Penghitung berada di memori per instans API
CORS	Hanya <i>origin</i> frontend yang terdaftar; Access-Control-Allow-Credentials: true hanya untuk <i>origin</i> tersebut
Syarat penempatan	Frontend dan API harus berada pada situs yang sama (domain terdaftar yang sama, mis. arsip.fite.example dan api.arsip.fite.example) agar cookie SameSite=Strict terkirim saat <i>refresh</i>

Singkatan peran: SA Super Admin, DKN Dekan, BAK BAAK/Staf, KPR Kaprodi, DSN Dosen/TA, MHS Mahasiswa. Semua berarti seluruh pengguna terautentikasi.

Amplop respons sukses:

```
{
  "success": true,
  "message": "Deskripsi singkat hasil",
  "data": {},
  "meta": {}
}
```

meta hanya muncul pada respons terdaftar (paginasi): {"current_page":1,"per_page":15,"total":42,"last_page":3}.

Amplop respons galat:

```
{
  "success": false,
  "code": "validation_error",
```

```
"message": "Data yang dikirim tidak valid",
"errors": { "field": ["Pesan validasi"] }
}
```

`code` adalah penanda yang dapat dibaca mesin untuk frontend; `errors` hanya muncul pada kode 422.

Katalog kode status dan `code`:

Status	code	Arti
200 / 201 / 204	-	Berhasil / Berhasil membuat sumber daya / Berhasil tanpa isi
401	unauthenticated	Token tidak ada, tidak valid, kedaluwarsa, atau sesi dicabut; kredensial login salah
403	forbidden	Terautentikasi tetapi tidak berwenang
403	account_disabled	Akun nonaktif (saat login)
403	password_change_required	Wajib mengganti kata sandi awal sebelum memakai <i>endpoint</i> lain
404	not_found	Sumber daya tidak ada (termasuk berkas <i>soft-deleted</i> bagi pengguna biasa)
409	conflict	Konflik status (mis. berkas sudah diverifikasi, kategori masih dipakai)
413	payload_too_large	<i>Request body</i> melebihi batas keras 11 MB
415	unsupported_media_type	Content-Type tidak sesuai (mis. JSON pada <i>endpoint</i> unggah)
422	validation_error	Validasi gagal
429	rate_limited	Terlalu banyak permintaan (disertai Retry-After)
500	server_error	Galat server (pesan generik; detail hanya di log)

Catatan urutan rute: rute statis (`/documents/public`, `/documents/categories`, `/users/roles`, `/users/prodi`) harus didefinisikan **sebelum** rute berparameter (`/{{id}}`).

Endpoint operasional (di luar `/api/v1`, tidak dihitung dalam 44): `GET /healthz` (publik; 200 bila proses hidup) dan `GET /readyz` (200 bila MySQL dan GCS terjangkau; dipakai pemantau *uptime* SLA-AV-01).

4.1 Auth & Profile Routes (/api/v1/auth/...)

No	Method	Endpoint	Deskripsi	Request (Header/Body)	Response (Code & Payload)	Hak Akses
1	POST	/auth/login	Login; menerbitkan <i>access token</i> dan memasang <i>cookie refresh token</i>	Body: <pre>{ "email": "string", "password": "string" }</pre>	200 <pre>{ "success": true, "message": "Login berhasil", "data": { "access_token": "...", "token_type": "Bearer", "expires_in": 900, "user": {...} } }</pre> dan header Set-Cookie: <pre>refresh_token=...; 401</pre> kredensial salah; 403 <pre>account_disabled; 422</pre> field kosong/format salah; 429 terlalu banyak percobaan	Publik
2	POST	/auth/refresh	Menerbitkan <i>access token</i> baru dan mengganti <i>refresh token</i> (rotasi)	Tanpa body. Wajib <i>cookie refresh_token</i> ; header <i>Origin</i> harus <i>origin</i> frontend yang terdaftar	200 payload sama dengan login (termasuk <i>user</i>) dan <i>cookie refresh_token</i> baru; 401 <i>cookie</i> tidak ada/kedaluwarsa /dicabut/ pernah dipakai (deteksi pemakaian ulang: sesi dicabut); 403 <i>Origin</i> tidak sah	Publik (dibatasi <i>cookie</i>)
3	POST	/auth/logout	Mencabut sesi saat ini dan menghapus <i>cookie</i>	Header: Bearer (sesi dari <i>claim sid</i>). Bila <i>access token</i> sudah kedaluwarsa, sesi ditentukan dari <i>cookie refresh_token</i>	200 <pre>{ "success": true, "message": "Logout berhasil", "data": null } </pre> dan Set-Cookie: <pre>refresh_token=; Max-Age=0; 401</pre> bila tidak ada token maupun <i>cookie</i> yang sah	Semua
4	GET	/auth/me	Data pengguna yang sedang login	Header: Bearer	200 <pre>data: objek pengguna; 401</pre>	Semua

SRS Sistem Informasi Arsip Berkas Digital FITE

No	Method	Endpoint	Deskripsi	Request (Header/Body)	Response (Code & Payload)	Hak Akses
5	PUT	/auth/profile	Memperbarui profil (nama, telepon)	Body: {"name":"string","phone":"string\\ null"}	200 data: pengguna terbaru; 422 ; 401	Semua
6	PUT	/auth/password	Mengganti kata sandi sendiri	Body: {"current_password":"...","password":"...","password_confirmation":"..."}.	200 {"success":true,"message":"Kata sandi diperbarui","data":null}; sesi lain milik pengguna dicabut, sesi yang sedang dipakai tetap berlaku; must_change_password menjadi false ; 422 sandi lama salah atau sandi baru lemah; 401	Semua

Aturan kata sandi baru: minimal 8 karakter, memuat huruf besar, huruf kecil, dan angka, serta berbeda dari sandi lama.

Atribut *cookie* refresh_token: HttpOnly; Secure; SameSite=Strict; Path=/api/v1/auth; Max-Age=604800 (7 hari). Nilainya acak 32 *byte* (*base64url*); basis data hanya menyimpan *hash*-nya. *Refresh token* **tidak pernah** muncul di isi JSON.

Klaim JWT *access token* (HS256 atau lebih kuat):

Klaim	Isi
sub	ID pengguna
sid	ID sesi (dipakai untuk pencabutan seketika)
iss	fite-arsip-api
aud	fite-arsip-web
iat, exp	Waktu terbit dan kedaluwarsa (15 menit)

Peran, prodi, dan status akun **tidak** dimasukkan ke klaim. *Middleware* membacanya dari basis data pada setiap permintaan, sehingga perubahan peran atau penonaktifan akun berlaku seketika (TC-AUTH-30, TC-AUTH-31).

Contoh respons POST /auth/login (200):

```
{
  "success": true,
  "message": "Login berhasil",
  "data": {
    "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "token_type": "Bearer",
    "expires_in": 900,
    "user": {
      "id": 12,
      "name": "Nama Pengguna",
      "email": "user@contoh.ac.id",
      "role": "baak",
      "role_label": "BAAK / Staf",
      "nim": null,
      "nip": "198001012005011001",
      "phone": null,
```

```

"prodi": null,
"is_active": true,
"must_change_password": false
}
}
}

```

Untuk peran lintas prodi (SA, DKN, BAK), `prodi` bernilai `null`; selain itu berbentuk `{"id":2,"code":"SI","name":"S1 Sistem Informasi"}`. Untuk MHS, `nim` terisi dan `nip` bernilai `null`; untuk DSN, KPR, dan BAK sebaliknya.

4.2 Document Management Routes (`/api/v1/documents/...`)

4.2.1 Tabel Endpoint

No	Method	Endpoint	Deskripsi	Request (Header/Body)	Response (Code & Payload)	Hak Akses
1	GET	<code>/documents</code>	Daftar berkas sesuai hak baca pengguna, dengan pencarian dan filter	Query: <code>q</code> , <code>prodi</code> , <code>uploader_role</code> , <code>category_id</code> , <code>nim</code> , <code>nip</code> , <code>extension</code> , <code>visibility</code> , <code>sort</code> , <code>page</code> , <code>per_page</code>	200 <code>data</code> : array Document, <code>meta</code> ; 401 ; 422 parameter tidak valid	Semua (cakupan sesuai 4.2.2)
2	GET	<code>/documents/public</code>	Explorer publik: hanya berkas Public berstatus Approved	Query: <code>q</code> , <code>prodi</code> , <code>category_id</code> , <code>extension</code> , <code>sort</code> , <code>page</code> , <code>per_page</code>	200 array Document, <code>meta</code> ; 401	Semua
3	GET	<code>/documents/categories</code>	Daftar kategori yang boleh dipakai perannya (untuk <i>dropdown</i> unggah/filter)	Header: Bearer	200 <code>data</code> : <code>[{"id":1,"name":"RPS","allowed_roles":["dosen","super_admin"],"is_operational":false}]</code> ; 401	Semua
4	POST	<code>/documents/categories</code>	Menambah kategori	Body: <code>{"name":"string","allowed_roles":["dosen","kaprodi"],"is_operational":false}</code>	201 kategori baru; 403 ; 422 nama duplikat atau peran tidak dikenal	SA
5	PUT	<code>/documents/categories/{id}</code>	Mengubah kategori	Body seperti no. 4	200 ; 403 ; 404 ; 422	SA
6	DELETE	<code>/documents/categories/{id}</code>	Menghapus kategori	Header: Bearer	204 ; 403 ; 404 ; 409 kategori masih dipakai berkas	SA

No	Method	Endpoint	Deskripsi	Request (Header/Body)	Response (Code & Payload)	Hak Akses
7	POST	/documents	Mengunggah berkas	Body multipart/form-data: file (wajib), title (wajib), category_id (wajib), visibility (private public, default private), description, prodi_id (wajib bagi BAK/SA yang mengunggah tanpa subject_user_id), subject_user_id (hanya BAK/SA), archived_at (hanya BAK/SA)	201 data: Document (visibility=private, atau public dengan verification_status=pending); 401; 403 kategori bukan hak perannya; 413 body > 11 MB; 415 bukan multipart/form-data; 422 berkas > 10 MB, format terlarang, isi tidak sesuai ekstensi, field dilarang untuk perannya, atau field kurang	Semua (kategori dibatasi peran)
8	GET	/documents/{id}	Detail berkas	Header: Bearer	200 data: Document; 401; 403 di luar hak baca; 404	Semua (sesuai hak baca)
9	GET	/documents/{id}/download	Mengunduh berkas (dialirkan dari GCS)	Header: Bearer	200 stream dengan Content-Disposition: attachment; filename="<nama asli>" dan Content-Type sesuai; 401; 403; 404; unduhan dicatat di storage_logs	Semua (sesuai hak baca)
10	PATCH	/documents/{id}	Mengubah metadata (judul, deskripsi, kategori)	Body: { "title": "...", "description": "...", "category_id": 1 }	200 Document terbaru; 403; 404; 422 (termasuk kategori yang tidak sesuai peran pengunggah)	Pengunggah; BAK (berkas yang berprodi); SA

No	Method	Endpoint	Deskripsi	Request (Header/Body)	Response (Code & Payload)	Hak Akses
11	PATCH	/documents/{id}/visibility	Mengubah visibilitas	Body: {"visibility": "public"} atau "private"	200 Document terbaru: ke public → verification_status=pending dan notifikasi antrean; ke private → keluar dari antrean/explorer publik; 403 ; 404 ; 409 sudah publik; 422	Pengunggah; SA
12	DELETE	/documents/{id}	Menghapus berkas (<i>soft delete</i>); ? force=true untuk <i>hard delete</i>	Query opsional: force=true	204 ; 401 ; 403 (termasuk force=true oleh non-SA); 404	Pengunggah; BAK (berkas kategori operasional); SA (semua, termasuk <i>hard delete</i>). DKN/KPR/DSN/MHS hanya berkas sendiri

4.2.2 Cakupan Baca GET /documents per Peran

Peran	Berkas yang dikembalikan
SA	Semua berkas (semua prodi, semua aktor)
DKN	Semua berkas fakultas
BAK	Berkas seluruh prodi (Read & Management)
KPR	Berkas dosen se-fakultas, berkas mahasiswa di prodinya, dan berkas miliknya
DSN	Berkas mahasiswa di prodinya dan berkas pribadinya
MHS	Berkas pribadinya (pengunggah atau pemilik berkas)

Berkas Public berstatus Approved tersedia bagi semua pengguna terautentikasi melalui GET /documents/public. Filter hanya mempersempit hasil dari himpunan yang boleh dibaca dan tidak dapat memperluas cakupan.

4.2.3 Aturan Validasi Unggah

Aturan	Nilai	Status bila dilanggar
Tipe konten permintaan	multipart/form-data	415
Batas keras request body	11 MB (10 MB + ruang multipart); dipotong saat dibaca, tidak ditampung penuh di memori	413
Ukuran berkas	≤ 10 MB (10.485.760 byte)	422
Ekstensi (whitelist)	pdf, docx, xlsx, pptx, zip, png, jpg (.jpeg dinormalisasi menjadi jpg)	422
Isi berkas	Tipe dideteksi dari byte awal (content sniffing) dan harus cocok dengan ekstensi; berkas docx/xlsx/pptx harus berstruktur ZIP Office yang sah	422
Ekstensi ganda berbahaya	Nama seperti laporan.pdf.exe ditolak	422

Aturan	Nilai	Status bila dilanggar
title	Wajib, maksimal 255 karakter	422
description	Opsional, maksimal 2000 karakter	422
category_id	Wajib, ada, dan allowed_roles memuat peran pengunggah	422 (tidak ada) / 403 (bukan hak peran)
subject_user_id, archived_at	Hanya BAK/SA; archived_at tidak boleh di masa depan	422
prodi_id	Wajib bagi BAK/SA tanpa subject_user_id; harus sama dengan prodi subjek bila subject_user_id diisi	422

4.2.4 Contoh Request Unggah

```
POST {{base_url}}/documents
Authorization: Bearer {{token_mahasiswa}}
Accept: application/json
Content-Type: multipart/form-data; boundary=----fite

file=@proposal-ta.pdf
title=Proposal Tugas Akhir
category_id=14
visibility=public
description=Proposal TA semester ganjil
```

4.2.5 Contoh Respons Unggah (201)

```
{
  "success": true,
  "message": "Berkas berhasil diunggah dan menunggu verifikasi",
  "data": {
    "id": 301,
    "title": "Proposal Tugas Akhir",
    "description": "Proposal TA semester ganjil",
    "original_name": "proposal-ta.pdf",
    "extension": "pdf",
    "mime_type": "application/pdf",
    "size_kb": 2048,
    "category": { "id": 14, "name": "Proposal TA/PA" },
    "prodi": { "id": 2, "code": "SI", "name": "SI Sistem Informasi" },
    "uploader": { "id": 45, "name": "Nama Mahasiswa", "role": "mahasiswa", "nim": "12S23001" },
    "owner": { "id": 45, "name": "Nama Mahasiswa", "role": "mahasiswa", "nim": "12S23001", "nip": null },
    "visibility": "public",
    "verification_status": "pending",
    "archived_at": "2026-10-07T08:20:00Z",
    "created_at": "2026-10-07T08:20:00Z",
    "links": {
      "self": "/api/v1/documents/301",
      "download": "/api/v1/documents/301/download"
    }
  }
}
```

verification_status bernilai null untuk berkas Private yang tidak pernah diajukan. Lokasi objek di GCS tidak diekspos ke klien.

4.2.6 Contoh Respons Galat Unggah

422, berkas melebihi batas dan format salah:

```
{
  "success": false,
  "code": "validation_error",
  "message": "Data yang dikirim tidak valid",
  "errors": {
    "file": [
      "Ukuran berkas maksimal 10 MB.",
      "Format berkas harus salah satu dari: pdf, docx, xlsx, pptx, zip, png, jpg."
    ]
  }
}
```

Untuk berkas video, pesan menyarankan menaruh tautan pada berkas teks lalu di-*zip*.

413, *body* terlalu besar:

```
{
  "success": false,
  "code": "payload_too_large",
  "message": "Ukuran permintaan melebihi batas 11 MB."
}
```

4.3 Verification & Approval Routes (/api/v1/verifications/...)

No	Method	Endpoint	Deskripsi	Request (Header/Body)	Response (Code & Payload)	Hak Akses
1	GET	/verifications	Antrean/riwayat verifikasi berkas publik	Query: status (pending approved rejected), prodi, document_id, page, per_page	200 array Verification, meta; 401 ; 403 ; 422	BAK, SA
2	GET	/verifications/{id}	Detail satu verifikasi beserta ringkasan berkas	Header: Bearer	200 data: Verification + ringkasan Document; 403 ; 404	BAK, SA
3	PATCH	/verifications/{id}	Memutuskan Approved atau Rejected	Body: {"status":"approved"} atau {"status":"rejected", "reason":"string"} (reason wajib bila <i>rejected</i> , 5 sampai 500 karakter)	200 Verification terbaru (verified_by, verified_at terisi); 401 ; 403 ; 404 ; 409 sudah diputuskan atau kalah dalam keputusan bersamaan; 422 status tidak valid atau reason tidak memenuhi syarat	BAK, SA

Urutan daftar: untuk status=pending, yang paling lama menunggu tampil lebih dulu; selain itu yang terbaru lebih dulu.

Aturan transisi status:

Dari	Ke	Pemicu	Efek
(tidak ada)	pending	Unggah Public atau ubah visibilitas ke Public	Notifikasi ke seluruh BAK dan SA
pending	approved	Keputusan BAK/SA	Berkas tampil di GET /documents/public; notifikasi ke pengunggah
pending	rejected	Keputusan BAK/SA + alasan	Berkas tetap Private; notifikasi berisi alasan ke pengunggah

Dari	Ke	Pemicu	Efek
pending	(dibatalkan)	Pengunggah mengembalikan ke Private	Baris verifikasi dan notifikasi antrean dihapus
approved	(dicabut)	Pengunggah mengubah ke Private	Hilang dari explorer publik; baris verifikasi dihapus
rejected	pending	Pengunggah mengajukan Public kembali	Verifikasi baru berstatus <i>pending</i> (alasan lama dikosongkan)

Keputusan diproses dalam satu transaksi dengan penguncian baris (SELECT ... FOR UPDATE). Dua verifikator yang bersamaan menghasilkan satu keputusan, dan yang lain menerima 409 (TC-VER-15).

Contoh respons PATCH /verifications/17 (200):

```
{
  "success": true,
  "message": "Berkas disetujui",
  "data": {
    "id": 17,
    "document": { "id": 301, "title": "Proposal Tugas Akhir", "prodi": "SI" },
    "status": "approved",
    "reason": null,
    "requested_by": { "id": 45, "name": "Nama Mahasiswa" },
    "verified_by": { "id": 12, "name": "Staf BAAK" },
    "requested_at": "2026-10-07T08:20:00Z",
    "verified_at": "2026-10-07T09:05:00Z"
  }
}
```

4.4 Notification Routes (/api/v1/notifications/...)

No	Method	Endpoint	Deskripsi	Request (Header/Body)	Response (Code & Payload)	Hak Akses
1	GET	/notifications	Daftar notifikasi milik pengguna (terbaru dulu)	Query: is_read (true false), type, page, per_page	200 array Notification, meta ; 401 ; 422	Semua
2	GET	/notifications /unread-count	Jumlah notifikasi belum dibaca (untuk badge)	Header: Bearer	200 { "success": true, "data": { "unread": 3 } }; 401	Semua
3	PATCH	/notifications /read-all	Menandai seluruh notifikasi sudah dibaca	Header: Bearer	200 { "data": { "updated": 5 } }	Semua
4	PATCH	/notifications /{id}/read	Menandai satu notifikasi sudah dibaca	Header: Bearer	200 Notification terbaru; 403 milik pengguna lain; 404	Semua
5	DELETE	/notifications /{id}	Menghapus notifikasi milik sendiri	Header: Bearer	204 ; 403 ; 404	Semua

read-all didefinisikan sebelum {id}/read agar tidak tertangkap sebagai ID.

Tipe notifikasi:

type	Penerima	Pemicu
verification_pending	Seluruh BAK dan SA aktif	Berkas Public baru masuk antrean
verification_approved	Pengunggah	Berkas disetujui
verification_rejected	Pengunggah	Berkas ditolak (memuat alasan)

Contoh Notification:

```
{
  "id": "9f1c2a3e-7b44-4d0a-9a6e-5d1f0c8b2e11",
  "type": "verification_pending",
  "title": "Berkas publik baru menunggu verifikasi",
  "message": "Proposal Tugas Akhir oleh Nama Mahasiswa (SI)",
  "data": { "verification_id": 17, "document_id": 301 },
  "is_read": false,
  "created_at": "2026-10-07T08:20:00Z"
}
```

4.5 User & Role Management Routes (/api/v1/users/...)

Seluruh *endpoint* pada bagian ini hanya untuk **Super Admin**, kecuali GET /users/prodi. Enam peran bersifat tetap (di-seed); pengaturan hak akses dilakukan melalui *policy* di kode, bukan melalui antarmuka.

No	Method	Endpoint	Deskripsi	Request (Header/Body)	Response (Code & Payload)	Hak Akses
1	GET	/users	Daftar pengguna	Query: q, role, prodi, is_active, page, per_page	200 array User, meta; 401 ; 403	SA
2	POST	/users	Membuat pengguna	Body: {"name", "email", "password", "role", "prodi_id", "nim", "nip", "phone"} (NIM wajib untuk mahasiswa; NIP wajib untuk dosen, kaprodi, baak; prodi_id wajib untuk mahasiswa, dosen, kaprodi)	201 User baru (must_change_password=true); 403 ; 422 email/NIM/NIP duplikat atau aturan peran dilanggar	SA
3	GET	/users/roles	Daftar peran dan jumlah pengguna per peran	Header: Bearer	200 [{"key": "baak", "label": "BAAK / Staf", "users_count": 3}]	SA
4	GET	/users/prodi	Daftar program studi	Header: Bearer	200 [{"id": 1, "code": "IF", "name": "SI Informatika", "is_active": true}]	Semua

No	Method	Endpoint	Deskripsi	Request (Header/Body)	Response (Code & Payload)	Hak Akses
5	PUT	/users/prodi/{id}	Mengubah nama/status aktif prodi	Body: {"name": "...", "is_active":true}	200; 403; 404; 422	SA
6	GET	/users/{id}	Detail pengguna	Header: Bearer	200 User; 403; 404	SA
7	PUT	/users/{id}	Mengubah data pengguna	Body seperti no. 2 (tanpa password dan role)	200 User terbaru; 403; 404; 422	SA
8	PATCH	/users/{id}/role	Mengubah peran (dan prodi bila perlu)	Body: {"role": "kaprodi", "prodi_id":1}	200 User terbaru; seluruh sesi pengguna dicabut dan owner_role pada berkas miliknya disinkronkan dalam satu transaksi; 403; 404; 409 mengubah peran diri sendiri atau menurunkan Super Admin terakhir; 422	SA
9	PATCH	/users/{id}/status	Mengaktifkan/menonaktifkan akun	Body: {"is_active":false}	200 ; seluruh sesi pengguna dicabut saat dinonaktifkan; 409 menonaktifkan diri sendiri atau Super Admin aktif terakhir	SA
10	PATCH	/users/{id}/password	Mengatur ulang kata sandi	Body: {"password":"...", "password_confirmation": "..."}	200 ; must_change_password=true; seluruh sesi pengguna dicabut; 422	SA
11	DELETE	/users/{id}	Menghapus pengguna (soft delete)	Header: Bearer	204 ; seluruh sesi dicabut; 409 menghapus diri sendiri atau Super Admin terakhir; 404	SA

Rute statis (roles , prodi) didefinisikan sebelum rute {id}.

Contoh Request POST /users :

```
{
  "name": "Nama Dosen",
  "email": "dosen@contoh.ac.id",
  "password": "KataSandiSementara#1",
  "role": "dosen",
  "prodi_id": 1,
```

```
"nip": "199001012015011001"
```

```
}
```

Contoh respons 201:

```
{
  "success": true,
  "message": "Pengguna dibuat",
  "data": {
    "id": 77,
    "name": "Nama Dosen",
    "email": "dosen@contoh.ac.id",
    "role": "dosen",
    "role_label": "Dosen / TA",
    "nim": null,
    "nip": "199001012015011001",
    "phone": null,
    "prodi": { "id": 1, "code": "IF", "name": "SI Informatika" },
    "is_active": true,
    "must_change_password": true
  }
}
```

4.6 Dashboard Statistics Routes (/api/v1/dashboard/...)

Semua statistik dihitung **hanya dari berkas yang boleh dibaca peran pemanggil** (aturan 4.2.2), sehingga angka tidak membocorkan data di luar hak akses.

No	Method	Endpoint	Deskripsi	Request	Response (Code & Payload)	Hak Akses
1	GET	/dashboard/summary	Kartu ringkasan sesuai peran	Header: Bearer	200 lihat contoh; 401	Semua
2	GET	/dashboard/documents-by-prodi	Jumlah berkas per prodi	Query: year (opsional)	200 [{"prodi": "IF", "total": 120}, ...]; 403	SA, DKN, BAK
3	GET	/dashboard/documents-by-category	Jumlah berkas per kategori	Query: prodi, year	200 [{"category": "RPS", "total": 48}, ...]	Semua (dalam cakupan baca)
4	GET	/dashboard/uploads-trend	Tren unggahan per bulan	Query: from, to (format YYYY-MM; default 12 bulan terakhir)	200 [{"month": "2026-08", "total": 31}, ...]; 422 rentang tidak valid atau > 36 bulan	Semua (dalam cakupan baca)
5	GET	/dashboard/verification-stats	Statistik verifikasi (jumlah per status dan rata-rata waktu proses dalam jam)	Query: prodi, from, to	200 lihat contoh; 403	BAK, SA
6	GET	/dashboard/storage	Pemakaian penyimpanan dan posisi retensi	Header: Bearer	200 lihat contoh; 403	SA

No	Method	Endpoint	Deskripsi	Request	Response (Code & Payload)	Hak Akses
7	GET	/dashboard/storage-logs	Log penyimpanan (audit), terbaru dulu	Query: action, document_id, user_id, from, to (tanggal ISO), page, per_page	200 array StorageLog, meta; 403 ; 422	SA

Konten GET /dashboard/summary per peran:

Peran	Kartu Ringkasan
SA	Total berkas, total pengguna, antrean verifikasi, pemakaian penyimpanan
DKN	Total berkas fakultas, berkas per prodi, berkas akreditasi, berkas milik sendiri
BAK	Antrean verifikasi, berkas per prodi, unggahan hari ini, berkas milik sendiri
KPR	Berkas prodi, berkas dosen, berkas mahasiswa prodi, berkas milik sendiri
DSN	Berkas pribadi, berkas mahasiswa prodi, berkas berstatus pending/rejected
MHS	Berkas pribadi, berkas pending/approved/rejected

Contoh respons GET /dashboard/summary (BAK):

```
{
  "success": true,
  "message": "OK",
  "data": {
    "role": "baak",
    "cards": {
      "pending_verifications": 6,
      "total_documents": 842,
      "uploads_today": 9,
      "my_documents": 57
    },
    "documents_by_prodi": [
      { "prodi": "IF", "total": 310 },
      { "prodi": "SI", "total": 288 },
      { "prodi": "TE", "total": 244 }
    ]
  }
}
```

Contoh respons GET /dashboard/verification-stats (200):

```
{
  "success": true,
  "message": "OK",
  "data": {
    "pending": 6,
    "approved": 120,
    "rejected": 14,
    "avg_processing_hours": 5.4
  }
}
```

Contoh respons GET /dashboard/storage (SA):

```
{
  "success": true,
  "message": "OK",
  "data": {
    "total_documents": 842,
    "total_size_mb": 4210.5,
```

```
"expiring_within_90_days": 14,  
"soft_deleted_in_grace_period": 3  
}  
}
```

SRS Sistem Informasi Arsip Berkas Digital FITE

Contoh StorageLog pada GET /dashboard/storage-logs :

```
{  
  "id": 9012,  
  "action": "expired_soft_delete",  
  "document_id": 118,  
  "user": null,  
  "meta": { "archived_at": "2018-03-01T00:00:00Z" },  
  "created_at": "2026-10-07T02:00:03Z"  
}
```

user bernilai `null` untuk tindakan sistem (penjadwal). Nilai `action` yang sah ada pada katalog di Bab 6.

4.7 Ringkasan

Kelompok	Jumlah Endpoint
Auth & Profile	6
Document Management (termasuk kategori)	12
Verification & Approval	3
Notifications	5
User & Role Management (termasuk prodi)	11
Dashboard Statistics	7
Total	44

Keputusan yang dipakai di Bab 5 (usulan, dapat diubah):

1. **Ganti sandi pertama.** Akun baru dan akun yang sandinya direset punya `must_change_password=true`. API menolak semua *endpoint* lain dengan 403 `password_change_required` sampai sandi diganti, kecuali `/auth/me`, `/auth/password`, `/auth/refresh`, dan `/auth/logout`.
2. **Umur mutlak sesi.** *Refresh token* berlaku 7 hari per rotasi, dengan batas mutlak sesi 30 hari sejak login.
3. **Deployment satu instans.** Bila API dijalankan lebih dari satu instans, pembatas laju perlu penghitung bersama. Frontend dan API harus berada di domain induk yang sama agar *cookie* `SameSite=Strict` berfungsi.

Bab ini memakai tiga keputusan usulan dari akhir Bab 4. Ubah bila tidak sesuai.

Keputusan	Yang dipakai di kode
Ganti sandi pertama	<code>must_change_password=true</code> membuat semua <i>endpoint</i> ditolak 403 <code>password_change_required</code> , kecuali <code>/auth/me</code> , <code>/auth/password</code> , <code>/auth/refresh</code> , dan <code>/auth/logout</code>
Umur mutlak sesi	<i>Refresh token</i> 7 hari (sliding) dengan batas mutlak 30 hari sejak login (<code>SESSION_MAX_AGE</code>)
Deployment	Satu instans API. Pembatas laju login dan API ada di memori. Batasannya dijelaskan di 5.10

Catatan jujur tentang kode. Kode ditulis untuk Go ≥ 1.22 dan Gin ≥ 1.9, tetapi **belum dikompilasi di lingkungan penyusunan dokumen ini**. Jalankan `go build ./... && go vet ./...` lebih dulu. Penyesuaian kecil akibat versi pustaka mungkin diperlukan, terutama pada `cloud.google.com/go/storage` dan `golang-jwt/jwt/v5`. Penyesuaian dari Bab 6 (butir 6.7.1) sudah diterapkan langsung pada kode di bab ini.

Penyederhanaan dari Bab 4: tabel `auth_sessions` memakai **satu baris per sesi login** (bukan rantai `family_id/replaced_by`). Setiap rotasi memindahkan *hash* lama ke `previous_refresh_hash`. Pemakaian ulang *refresh token* lama mencabut sesi. Skema final ada di Bab 6.

5.1 Struktur Proyek, Dependensi, dan Konfigurasi

```
fite-arsip-api/
├── cmd/
│   ├── api/main.go           # server HTTP
│   ├── worker/main.go       # penjadwal retensi (--once untuk uji)
│   └── seed/main.go         # seeder akun uji (tidak ditampilkan)
├── internal/
│   ├── app/app.go           # pembuka koneksi DB bersama
│   ├── config/config.go
│   ├── apperr/apperr.go     # galat domain → status HTTP + code
│   ├── httpx/httpx.go       # respons, binding, validasi, Wrap
│   ├── domain/domain.go     # model GORM, konstanta peran
│   ├── storage/{storage.go,gcs.go}
│   ├── middleware/{middleware.go,auth.go,limiter.go}
│   ├── service/{auth.go,helpers.go,filecheck.go,document.go,verification.go,retention.go}
│   ├── handler/{dto.go,auth.go,document.go,verification.go}
│   └── router/router.go
└── migrations/              # SQL golang-migrate (Bab 6)
```

```
go mod init fite-arsip-api
go get github.com/gin-gonic/gin@latest gorm.io/gorm gorm.io/driver/mysql \
github.com/golang-jwt/jwt/v5 golang.org/x/crypto golang.org/x/time \
github.com/google/uuid github.com/go-playground/validator/v10 \
cloud.google.com/go/storage google.golang.org/api github.com/robfig/cron/v3
```

Variabel lingkungan (`.env` , tidak di-*commit*):

```
APP_ENV=development           # production mewajibkan COOKIE_SECURE=true
HTTP_ADDR=:8080
DB_DSN=fite_app:rahasia@tcp(127.0.0.1:3306)/fite_arsip?charset=utf8mb4&parseTime=true&loc=UTC&time_zone=%27%2B00%3A00%27
JWT_SECRET=ganti-dengan-minimal-32-byte-acak-yang-panjang
JWT_ISSUER=fite-arsip-api
JWT_AUDIENCE=fite-arsip-web
ACCESS_TTL=15m
REFRESH_TTL=168h
SESSION_MAX_AGE=720h
ALLOWED_ORIGIN=http://localhost:3000
COOKIE_DOMAIN=
COOKIE_SECURE=false
TRUSTED_PROXIES=
GCS_BUCKET=fite-arsip-private
GCS_CREDENTIALS_FILE=/etc/secrets/gcs-service-account.json
```

```
// internal/config/config.go
package config

import (
    "errors"
    "os"
    "strconv"
    "strings"
    "time"
)

type Config struct {
    Env, HTTPAddr, DBDSN          string
    JWTSecret                    []byte
    JWTIssuer, JWTAudience       string
    AccessTTL, RefreshTTL        time.Duration
    SessionMaxAge                time.Duration
    AllowedOrigin, CookieDomain  string
    CookieSecure                 bool
    TrustedProxies               []string
    GCSBucket, GCSCredentials    string
    RetentionYears, GraceDays     int
    Timezone                    string
    MaxFileBytes                 int64 // 10 MB
    MaxUploadBody, MaxJSONBody   int64 // 11 MB, 1 MB
}

func (c Config) IsProd() bool { return c.Env == "production" }

func Load() (Config, error) {
    var err error
    c := Config{
        Env:          env("APP_ENV", "development"),
        HTTPAddr:     env("HTTP_ADDR", ":8080"),
        DBDSN:        os.Getenv("DB_DSN"),
        JWTSecret:    []byte(os.Getenv("JWT_SECRET")),
        JWTIssuer:    env("JWT_ISSUER", "fite-arsip-api"),
        JWTAudience: env("JWT_AUDIENCE", "fite-arsip-web"),
        AllowedOrigin: os.Getenv("ALLOWED_ORIGIN"),
        CookieDomain: os.Getenv("COOKIE_DOMAIN"),
        CookieSecure: env("COOKIE_SECURE", "false") == "true",
        GCSBucket:    os.Getenv("GCS_BUCKET"),
        GCSCredentials: os.Getenv("GCS_CREDENTIALS_FILE"),
        Timezone:     env("TIMEZONE", "Asia/Jakarta"),
        MaxFileBytes: 10 << 20,
        MaxUploadBody: 11 << 20,
        MaxJSONBody:  1 << 20,
    }
    for _, p := range strings.Split(os.Getenv("TRUSTED_PROXIES"), ",") {
        if p = strings.TrimSpace(p); p != "" {
            c.TrustedProxies = append(c.TrustedProxies, p)
        }
    }
    if c.AccessTTL, err = duration("ACCESS_TTL", 15*time.Minute); err != nil {
        return c, err
    }
    if c.RefreshTTL, err = duration("REFRESH_TTL", 7*24*time.Hour); err != nil {
        return c, err
    }
    if c.SessionMaxAge, err = duration("SESSION_MAX_AGE", 30*24*time.Hour); err != nil {
        return c, err
    }
    if c.RetentionYears, err = integer("RETENTION_YEARS", 8); err != nil {
        return c, err
    }
    if c.GraceDays, err = integer("RETENTION_GRACE_DAYS", 90); err != nil {
        return c, err
    }

    var problems []string
    if c.DBDSN == "" {
        problems = append(problems, "DB_DSN wajib")
    }
    if len(c.JWTSecret) < 32 {
        problems = append(problems, "JWT_SECRET minimal 32 byte")
    }
    if c.AllowedOrigin == "" {

```

```

        problems = append(problems, "ALLOWED_ORIGIN wajib")
    }
    if c.GCSBucket == "" {
        problems = append(problems, "GCS_BUCKET wajib")
    }
    if c.IsProd() && !c.CookieSecure {
        problems = append(problems, "COOKIE_SECURE wajib true di production")
    }
    if len(problems) > 0 {
        return c, errors.New("konfigurasi tidak valid: " + strings.Join(problems, "; "))
    }
    return c, nil
}

func env(k, def string) string {
    if v := os.Getenv(k); v != "" {
        return v
    }
    return def
}

func duration(k string, def time.Duration) (time.Duration, error) {
    if v := os.Getenv(k); v != "" {
        return time.ParseDuration(v)
    }
    return def, nil
}

func integer(k string, def int) (int, error) {
    if v := os.Getenv(k); v != "" {
        return strconv.Atoi(v)
    }
    return def, nil
}

```

5.2 Fondasi: Galat dan Respons

```

// internal/apperr/apperr.go
package apperr

import (
    "fmt"
    "net/http"
    "time"
)

type Error struct {
    Status    int
    Code      string
    Message   string
    Fields    map[string][]string
    RetryAfter time.Duration
    Err       error
}

func (e *Error) Error() string {
    if e.Err != nil {
        return e.Message + ": " + e.Err.Error()
    }
    return e.Message
}

func (e *Error) Unwrap() error { return e.Err }

func New(status int, code, msg string) *Error { return &Error{Status: status, Code: code, Message: msg} }

func or(msg, def string) string {
    if msg == "" {
        return def
    }
    return msg
}

func Unauthenticated(msg string) *Error {
    return New(http.StatusUnauthorized, "unauthenticated",
        or(msg, "Tidak terautentikasi. Token tidak ada, tidak valid, atau kedaluwarsa. "))
}

func Forbidden(msg string) *Error {
    return New(http.StatusForbidden, "forbidden", or(msg, "Anda tidak memiliki hak akses untuk tindakan ini. "))
}

```

n.")

```

func AccountDisabled() *Error {
    return New(http.StatusForbidden, "account_disabled", "Akun Anda dinonaktifkan. Hubungi administrator.")
}
func PasswordChangeRequired() *Error {
    return New(http.StatusForbidden, "password_change_required", "Anda wajib mengganti kata sandi sebelum melanjutkan.")
}
func NotFound(msg string) *Error {
    return New(http.StatusNotFound, "not_found", or(msg, "Sumber daya tidak ditemukan.))
}
func Conflict(msg string) *Error {
    return New(http.StatusConflict, "conflict", or(msg, "Konflik status data.))
}
func TooLarge(limit int64) *Error {
    return New(http.StatusRequestEntityTooLarge, "payload_too_large",
        fmt.Sprintf("Ukuran permintaan melebihi batas %d MB.", limit>>20))
}
func UnsupportedMedia(msg string) *Error {
    return New(http.StatusUnsupportedMediaType, "unsupported_media_type", or(msg, "Tipe konten tidak didukung.))
}
func Validation(fields map[string][]string) *Error {
    e := New(http.StatusUnprocessableEntity, "validation_error", "Data yang dikirim tidak valid")
    e.Fields = fields
    return e
}
func FieldError(field, msg string) *Error { return Validation(map[string][]string{field: {msg}}) }
func RateLimited(after time.Duration) *Error {
    e := New(http.StatusTooManyRequests, "rate_limited", "Terlalu banyak permintaan. Coba lagi beberapa saat.")
    e.RetryAfter = after
    return e
}
func Internal(err error) *Error {
    e := New(http.StatusInternalServerError, "server_error", "Terjadi kesalahan pada server.")
    e.Err = err
    return e
}

```

```

// internal/httpx/httpx.go
package httpx

import (
    "encoding/json"
    "errors"
    "fmt"
    "log/slog"
    "math"
    "net/http"
    "reflect"
    "regexp"
    "strconv"
    "strings"
    "unicode"

    "fite-arsip-api/internal/apperr"

    "github.com/gin-gonic/gin"
    "github.com/gin-gonic/gin/binding"
    "github.com/go-playground/validator/v10"
    "gorm.io/gorm"
)

const RequestIDKey = "request_id"

type Meta struct {
    CurrentPage int    `json:"current_page"`
    PerPage     int    `json:"per_page"`
    Total       int64  `json:"total"`
    LastPage    int    `json:"last_page"`
}

func PageMeta(page, per int, total int64) Meta {
    last := int(math.Ceil(float64(total) / float64(per)))
    if last < 1 {
        last = 1
    }
    return Meta{CurrentPage: page, PerPage: per, Total: total, LastPage: last}
}

func RequestID(c *gin.Context) string { return c.GetString(RequestIDKey) }

func OK(c *gin.Context, status int, message string, data any) {

```

```
c.JSON(status, gin.H{"success": true, "message": message, "data": data})
```

SRS Sistem Informasi Arsip Berkas Digital FITE

```
func Paged(c *gin.Context, message string, data any, meta Meta) {
    c.JSON(http.StatusOK, gin.H{"success": true, "message": message, "data": data, "meta": meta})
}
```

```
func NoContent(c *gin.Context) { c.Status(http.StatusNoContent) }
```

```
func Abort(c *gin.Context, err error) {
    Fail(c, err)
    c.Abort()
}
```

```
func Fail(c *gin.Context, err error) {
    var ae *apperr.Error
    if !errors.As(err, &ae) {
        ae = mapError(err)
    }
    if ae.Status >= 500 {
        logError(c, err)
    }
    if ae.RetryAfter > 0 {
        c.Header("Retry-After", strconv.Itoa(int(math.Ceil(ae.RetryAfter.Seconds()))))
    }
    body := gin.H{"success": false, "code": ae.Code, "message": ae.Message}
    if len(ae.Fields) > 0 {
        body["errors"] = ae.Fields
    }
    c.JSON(ae.Status, body)
}
```

```
func logError(c *gin.Context, err error) {
    slog.ErrorContext(c.Request.Context(), "request gagal",
        "request_id", RequestID(c), "method", c.Request.Method, "path", c.FullPath(), "error", err)
}
```

```
func mapError(err error) *apperr.Error {
    var mbe *http.MaxBytesError
    switch {
    case errors.Is(err, gorm.ErrRecordNotFound):
        return apperr.NotFound("")
    case errors.As(err, &mbe):
        return apperr.Toolarge(mbe.Limit)
    case strings.Contains(err.Error(), "request body too large"):
        return apperr.Toolarge(11 << 20)
    }
    return apperr.Internal(err)
}
```

```
type HandlerFunc func(*gin.Context) error
```

```
// Wrap mengubah handler yang mengembalikan error menjadi gin.HandlerFunc.
```

```
func Wrap(h HandlerFunc) gin.HandlerFunc {
    return func(c *gin.Context) {
        if err := h(c); err != nil {
            Fail(c, err)
        }
    }
}
```

```
// ----- binding
```

```
var phoneRe = regexp.MustCompile(`^\\+?[0-9]{8,15}$`)
```

```
func Init() {
    binding.EnableDecoderDisallowUnknownFields = true
    v, ok := binding.Validator.Engine().(*validator.Validate)
    if !ok {
        return
    }
    v.RegisterTagNameFunc(func(f reflect.StructField) string {
        for _, tag := range []string{"json", "form"} {
            if name := strings.Split(f.Tag.Get(tag), ",")[0]; name != "" && name != "-" {
                return name
            }
        }
        return f.Name
    })
}
```

```
    _ = v.RegisterValidation("phone", func(fl validator.FieldLevel) bool { return phoneRe.MatchString(fl.Field().String()) })
```

```
    _ = v.RegisterValidation("strongpw", func(fl validator.FieldLevel) bool { return StrongPassword(fl.Field().String()) })
```

```

// StrongPassword: 8-72 byte (batas bcrypt), memuat huruf besar, kecil, dan angka.
func StrongPassword(s string) bool {
    if len(s) < 8 || len(s) > 72 {
        return false
    }
    var up, lo, di bool
    for _, r := range s {
        switch {
        case unicode.IsUpper(r):
            up = true
        case unicode.IsLower(r):
            lo = true
        case unicode.IsDigit(r):
            di = true
        }
    }
    return up && lo && di
}

func BindJSON(c *gin.Context, dst any) error {
    if c.ContentType() != "application/json" {
        return apperr.UnsupportedMedia("Content-Type harus application/json.")
    }
    return bindErr(c.ShouldBindJSON(dst))
}

func BindQuery(c *gin.Context, dst any) error { return bindErr(c.ShouldBindQuery(dst)) }

func BindForm(c *gin.Context, dst any) error {
    return bindErr(c.ShouldBindWith(dst, binding.FormMultipart))
}

func bindErr(err error) error {
    if err == nil {
        return nil
    }
    var ve validator.ValidationErrors
    if errors.As(err, &ve) {
        fields := map[string][]string{}
        for _, fe := range ve {
            fields[fe.Field()] = append(fields[fe.Field()], fieldMessage(fe))
        }
        return apperr.Validation(fields)
    }
    var mbe *http.MaxBytesError
    if errors.As(err, &mbe) {
        return apperr.ToolLarge(mbe.Limit)
    }
    var syn *json.SyntaxError
    var typ *json.UnmarshalTypeError
    if errors.As(err, &syn) || errors.As(err, &typ) || strings.Contains(err.Error(), "unknown field") {
        return apperr.FieldError("body", "Isi permintaan tidak valid atau memuat field yang tidak dikenal.")
    }
    return apperr.FieldError("body", "Isi permintaan tidak dapat dibaca.")
}

func fieldMessage(fe validator.FieldError) string {
    name := fe.Field()
    isStr := fe.Kind() == reflect.String
    switch fe.Tag() {
    case "required":
        return name + " wajib diisi."
    case "required_if":
        return name + " wajib diisi pada kondisi ini."
    case "email":
        return "Format email tidak valid."
    case "max":
        if isStr {
            return fmt.Sprintf("%s maksimal %s karakter.", name, fe.Param())
        }
        return fmt.Sprintf("%s maksimal %s.", name, fe.Param())
    case "min":
        if isStr {
            return fmt.Sprintf("%s minimal %s karakter.", name, fe.Param())
        }
        return fmt.Sprintf("%s minimal %s.", name, fe.Param())
    case "gt":
        return fmt.Sprintf("%s harus lebih dari %s.", name, fe.Param())
    case "oneof":

```

```

        return name + " harus salah satu dari: " + strings.ReplaceAll(fe.Param(), " ", ", ") + "."
    case "eqfield":
        return name + " tidak cocok."
    case "phone":
        return "Nomor telepon harus 8 sampai 15 digit (boleh diawali +)."
```

```

    case "strongpw":
        return "Kata sandi minimal 8 karakter dan memuat huruf besar, huruf kecil, dan angka."
    }
    return name + " tidak valid."
}

```

5.3 Model Domain (GORM)

```

// internal/domain/domain.go
package domain

import (
    "database/sql/driver"
    "encoding/json"
    "fmt"
    "time"

    "gorm.io/gorm"
)

const (
    RoleSuperAdmin = "super_admin"
    RoleDekan      = "dekan"
    RoleBaak       = "baak"
    RoleKaprodi    = "kaprodi"
    RoleDosen      = "dosen"
    RoleMahasiswa  = "mahasiswa"
)

var AllRoles = []string{RoleSuperAdmin, RoleDekan, RoleBaak, RoleKaprodi, RoleDosen, RoleMahasiswa}

// ---- tipe kolom JSON (nilai dikirim sebagai string agar diterima kolom JSON MySQL) ----

type StringList []string

func (s StringList) Value() (driver.Value, error) {
    if s == nil {
        s = StringList{}
    }
    b, err := json.Marshal([]string(s))
    return string(b), err
}

func (s *StringList) Scan(src any) error {
    switch v := src.(type) {
    case nil:
        *s = StringList{}
        return nil
    case []byte:
        return json.Unmarshal(v, s)
    case string:
        return json.Unmarshal([]byte(v), s)
    }
    return fmt.Errorf("StringList: tipe %T tidak didukung", src)
}

func (s StringList) Contains(v string) bool {
    for _, x := range s {
        if x == v {
            return true
        }
    }
    return false
}

type JSONMap map[string]any

func (m JSONMap) Value() (driver.Value, error) {
    if m == nil {
        return nil, nil
    }
    b, err := json.Marshal(map[string]any(m))
    return string(b), err
}

```

```
func (m *JSONMap) Scan(src any) error {
    switch v := src.(type) {
    case nil:
        *m = nil
        return nil
    case []byte:
        return json.Unmarshal(v, m)
    case string:
        return json.Unmarshal([]byte(v), m)
    }
    return fmt.Errorf("JSONMap: tipe %T tidak didukung", src)
}
```

```
// ---- model ----
```

```
type Role struct {
    ID          uint    `gorm:"primaryKey"`
    Key         string  `gorm:"column:key;size:30;uniqueIndex"`
    Name        string  `gorm:"size:50"`
    CreatedAt   time.Time
    UpdatedAt   time.Time
}
```

```
type Prodi struct {
    ID          uint    `gorm:"primaryKey"`
    Code        string  `gorm:"size:5;uniqueIndex"`
    Name        string  `gorm:"size:100"`
    IsActive    bool
    CreatedAt   time.Time
    UpdatedAt   time.Time
}
```

```
func (Prodi) TableName() string { return "prodi" }
```

```
type User struct {
    ID          uint    `gorm:"primaryKey"`
    Name        string  `gorm:"size:100"`
    Email       string  `gorm:"size:255;uniqueIndex"`
    PasswordHash string  `gorm:"column:password;size:255"`
    RoleID      uint
    Role        Role
    ProdiID     *uint
    Prodi       *Prodi
    NIM         *string `gorm:"column:nim;size:30"`
    NIP         *string `gorm:"column:nip;size:30"`
    Phone       *string `gorm:"size:20"`
    IsActive    bool
    MustChangePassword bool
    CreatedAt   time.Time
    UpdatedAt   time.Time
    DeletedAt   gorm.DeletedAt `gorm:"index"`
}
```

```
type AuthSession struct {
    ID          string  `gorm:"primaryKey;type:char(36)"`
    UserID      uint    `gorm:"index"`
    RefreshTokenHash string  `gorm:"size:64;uniqueIndex"`
    PreviousRefreshHash *string `gorm:"size:64;index"`
    UserAgent   string  `gorm:"size:255"`
    IP          string  `gorm:"column:ip;size:45"`
    ExpiresAt   time.Time
    AbsoluteExpiresAt time.Time
    LastUsedAt  *time.Time
    RevokedAt   *time.Time
    CreatedAt   time.Time
}
```

```
type Category struct {
    ID          uint    `gorm:"primaryKey"`
    Name        string  `gorm:"size:100;uniqueIndex"`
    AllowedRoles StringList `gorm:"type:json"`
    IsOperational bool
    CreatedAt   time.Time
    UpdatedAt   time.Time
}
```

```
type Document struct {
    ID          uint    `gorm:"primaryKey"`
    Title       string  `gorm:"size:255"`
    Description  *string `gorm:"type:text"`
    OriginalName string  `gorm:"size:255"`
}
```

```

Extension      string `gorm:"size:10"`
MimeType       string `gorm:"size:100"`
SizeKB        uint   `gorm:"column:size_kb"`
StoragePath    string `gorm:"size:500;uniqueIndex" json:"-"`
CategoryID     uint
Category       Category
ProdiID        *uint
Prodi          *Prodi
UploaderID     uint
Uploader       User `gorm:"foreignKey:UploaderID"`
OwnerID        uint
Owner          User `gorm:"foreignKey:OwnerID"`
OwnerRole      string `gorm:"size:30"`
Visibility     string `gorm:"size:10"`
VerificationStatus *string `gorm:"size:10"`
ArchivedAt     *time.Time
RetentionStatus *string `gorm:"size:10"`
CreatedAt      time.Time
UpdatedAt      time.Time
DeletedAt      gorm.DeletedAt `gorm:"index"`
}

type Verification struct {
    ID          uint `gorm:"primaryKey"`
    DocumentID  uint `gorm:"uniqueIndex"`
    Document    Document
    Status      string `gorm:"size:10"`
    Reason      *string `gorm:"size:500"`
    RequestedBy uint
    Requester   User `gorm:"foreignKey:RequestedBy"`
    VerifiedBy  *uint
    Verifier    *User `gorm:"foreignKey:VerifiedBy"`
    RequestedAt time.Time
    VerifiedAt  *time.Time
    CreatedAt   time.Time
    UpdatedAt   time.Time
}

func (Verification) TableName() string { return "document_verifications" }

type Notification struct {
    ID          string `gorm:"primaryKey;type:char(36)"`
    UserID      uint   `gorm:"index"`
    Type        string `gorm:"size:50"`
    Title       string `gorm:"size:150"`
    Message     string `gorm:"type:text"`
    Data        JSONMap `gorm:"type:json"`
    IsRead      bool
    CreatedAt   time.Time
    UpdatedAt   time.Time
}

type StorageLog struct {
    ID          uint `gorm:"primaryKey"`
    DocumentID *uint
    UserID      *uint
    Action      string `gorm:"size:40"`
    Meta        JSONMap `gorm:"type:json"`
    CreatedAt   time.Time
}

```

5.4 Abstraksi Penyimpanan (GCS)

```

// internal/storage/storage.go
package storage

import (
    "context"
    "errors"
    "io"
)

var ErrNotFound = errors.New("storage: objek tidak ditemukan")

type Storage interface {
    Put(ctx context.Context, path string, r io.Reader, contentType string) error
    Open(ctx context.Context, path string) (io.ReadCloser, int64, error)
    Delete(ctx context.Context, path string) error // 55/130
    Empty() bool // 55/130
}

```

```

}

// internal/storage/gcs.go
package storage

import (
    "context"
    "errors"
    "io"

    gcs "cloud.google.com/go/storage"
    "google.golang.org/api/option"
)

type GCS struct {
    client *gcs.Client
    bucket string
}

func NewGCS(ctx context.Context, bucket, credentialsFile string) (*GCS, error) {
    var opts []option.ClientOption
    if credentialsFile != "" {
        opts = append(opts, option.WithCredentialsFile(credentialsFile))
    }
    cl, err := gcs.NewClient(ctx, opts...)
    if err != nil {
        return nil, err
    }
    return &GCS{client: cl, bucket: bucket}, nil
}

func (g *GCS) obj(path string) *gcs.ObjectHandle { return g.client.Bucket(g.bucket).Object(path) }

// Put mengalirkan data ke GCS dengan penyangga 2 MB (memori terbatas walau unggahan paralel).
func (g *GCS) Put(ctx context.Context, path string, r io.Reader, contentType string) error {
    ctx, cancel := context.WithCancel(ctx)
    defer cancel()
    w := g.obj(path).NewWriter(ctx)
    w.ContentType = contentType
    w.ChunkSize = 2 << 20
    if _, err := io.Copy(w, r); err != nil {
        cancel() // batalkan unggahan agar tidak ada objek parsial
        _ = w.Close()
        return err
    }
    return w.Close()
}

func (g *GCS) Open(ctx context.Context, path string) (io.ReadCloser, int64, error) {
    r, err := g.obj(path).NewReader(ctx)
    if errors.Is(err, gcs.ErrObjectNotExist) {
        return nil, 0, ErrNotFound
    }
    if err != nil {
        return nil, 0, err
    }
    return r, r.Attrs.Size, nil
}

func (g *GCS) Delete(ctx context.Context, path string) error {
    err := g.obj(path).Delete(ctx)
    if errors.Is(err, gcs.ErrObjectNotExist) {
        return nil
    }
    return err
}

func (g *GCS) Ping(ctx context.Context) error {
    _, err := g.client.Bucket(g.bucket).Attrs(ctx)
    return err
}

```

5.5 Middleware

```

// internal/middleware/middleware.go
package middleware

```

```

import (
    "fmt"
    "io"
    "log/slog"
    "net/http"
    "time"

    "fite-arsip-api/internal/apperr"
    "fite-arsip-api/internal/httpx"

    "github.com/gin-gonic/gin"
    "github.com/google/uuid"
)

func RequestID() gin.HandlerFunc {
    return func(c *gin.Context) {
        id := uuid.NewString() // sengaja tidak memercayai ID dari klien
        c.Set(httpx.RequestIDKey, id)
        c.Header("X-Request-ID", id)
        c.Next()
    }
}

func Recover() gin.HandlerFunc {
    return gin.CustomRecoveryWithWriter(io.Discard, func(c *gin.Context, rec any) {
        httpx.Abort(c, apperr.Internal(fmt.Errorf("panic: %v", rec)))
    })
}

func Logger() gin.HandlerFunc {
    return func(c *gin.Context) {
        start := time.Now()
        c.Next()
        attrs := []any{
            "request_id", httpx.RequestID(c), "method", c.Request.Method, "route", c.FullPath(),
            "status", c.Writer.Status(), "ms", time.Since(start).Milliseconds(), "ip", c.ClientIP(),
        }
        if u := CurrentUser(c); u != nil {
            attrs = append(attrs, "user_id", u.ID)
        }
        slog.Info("http", attrs...)
    }
}

func CORS(origin string) gin.HandlerFunc {
    return func(c *gin.Context) {
        if o := c.GetHeader("Origin"); o != "" && o == origin {
            h := c.Writer.Header()
            h.Set("Access-Control-Allow-Origin", origin)
            h.Set("Access-Control-Allow-Credentials", "true")
            h.Add("Vary", "Origin")
            h.Set("Access-Control-Expose-Headers", "X-Request-ID, Retry-After, Content-Disposition")
            if c.Request.Method == http.MethodOptions {
                h.Set("Access-Control-Allow-Methods", "GET, POST, PUT, PATCH, DELETE, OPTIONS")
                h.Set("Access-Control-Allow-Headers", "Authorization, Content-Type, Accept")
                h.Set("Access-Control-Max-Age", "600")
                c.AbortWithStatus(http.StatusNoContent)
                return
            }
        }
        c.Next()
    }
}

func SecurityHeaders(prod bool) gin.HandlerFunc {
    return func(c *gin.Context) {
        h := c.Writer.Header()
        h.Set("X-Content-Type-Options", "nosniff")
        h.Set("Referrer-Policy", "no-referrer")
        h.Set("Content-Security-Policy", "default-src 'none'; frame-ancestors 'none'")
        if prod {
            h.Set("Strict-Transport-Security", "max-age=31536000; includeSubDomains")
        }
        c.Next()
    }
}

// LimitBody: batas keras isi permintaan. Rute unggah memakai batas lebih besar.
func LimitBody(jsonMax, uploadMax int64, uploadMethod, uploadRoute string) gin.HandlerFunc {
    return func(c *gin.Context) {
        limit := jsonMax
        if c.Request.Method == uploadMethod && c.FullPath() == uploadRoute {

```

```

    }
    if c.Request.ContentLength > limit {
        httpx.Abort(c, apperr.TooLarge(limit))
        return
    }
    c.Request.Body = http.MaxBytesReader(c.Writer, c.Request.Body, limit)
    c.Next()
}
}

```

```

// internal/middleware/auth.go
package middleware

import (
    "strings"

    "fite-arsip-api/internal/apperr"
    "fite-arsip-api/internal/domain"
    "fite-arsip-api/internal/httpx"
    "fite-arsip-api/internal/service"

    "github.com/gin-gonic/gin"
)

const (
    ctxUser      = "auth.user"
    ctxSession   = "auth.session"
)

func CurrentUser(c *gin.Context) *domain.User {
    v, _ := c.Get(ctxUser)
    u, _ := v.(*domain.User)
    return u
}

func CurrentSession(c *gin.Context) *domain.AuthSession {
    v, _ := c.Get(ctxSession)
    s, _ := v.(*domain.AuthSession)
    return s
}

// Auth memverifikasi JWT, lalu memuat sesi, pengguna, dan peran TERBARU dari basis data
// (peran dan status tidak dipercaya dari klaim token).
func Auth(auth *service.AuthService) gin.HandlerFunc {
    const prefix = "Bearer "
    return func(c *gin.Context) {
        h := c.GetHeader("Authorization")
        if len(h) <= len(prefix) || !strings.EqualFold(h[len(prefix):], prefix) {
            httpx.Abort(c, apperr.Unauthenticated(""))
            return
        }
        user, sess, err := auth.Authenticate(c.Request.Context(), strings.TrimSpace(h[len(prefix):]))
        if err != nil {
            httpx.Abort(c, err)
            return
        }
        c.Set(ctxUser, user)
        c.Set(ctxSession, sess)
        c.Next()
    }
}

// EnforcePasswordChange dipasang pada semua rute kecuali /auth/me dan /auth/password.
func EnforcePasswordChange() gin.HandlerFunc {
    return func(c *gin.Context) {
        if u := CurrentUser(c); u != nil && u.MustChangePassword {
            httpx.Abort(c, apperr.PasswordChangeRequired())
            return
        }
        c.Next()
    }
}

func RequireRoles(roles ...string) gin.HandlerFunc {
    allowed := make(map[string]struct{}, len(roles))
    for _, r := range roles {
        allowed[r] = struct{}{}
    }
    return func(c *gin.Context) {
        u := CurrentUser(c)

```

```

        if u == nil {
            httpx.Abort(c, apperr.Unauthenticated(""))
            return
        }
        if _, ok := allowed[u.Role.Key]; !ok {
            httpx.Abort(c, apperr.Forbidden(""))
            return
        }
        c.Next()
    }
}

```

```

// internal/middleware/limiter.go
package middleware

import (
    "sync"
    "time"

    "fite-arsip-api/internal/apperr"
    "fite-arsip-api/internal/httpx"

    "github.com/gin-gonic/gin"
    "golang.org/x/time/rate"
)

// FailureLimiter menghitung percobaan login GAGAL per kunci dalam jendela tetap.
type FailureLimiter struct {
    mu      sync.Mutex
    m       map[string]*failure
    max     int
    window time.Duration
}

type failure struct {
    n      int
    reset time.Time
}

func NewFailureLimiter(max int, window time.Duration) *FailureLimiter {
    l := &FailureLimiter{m: map[string]*failure{}, max: max, window: window}
    go func() {
        for range time.Tick(5 * time.Minute) {
            l.mu.Lock()
            for k, f := range l.m {
                if time.Now().After(f.reset) {
                    delete(l.m, k)
                }
            }
            l.mu.Unlock()
        }
    }()
    return l
}

// Check: diblokir bila kegagalan dalam jendela sudah mencapai batas.
func (l *FailureLimiter) Check(key string) (bool, time.Duration) {
    l.mu.Lock()
    defer l.mu.Unlock()
    f := l.m[key]
    if f == nil || time.Now().After(f.reset) || f.n < l.max {
        return true, 0
    }
    return false, time.Until(f.reset)
}

func (l *FailureLimiter) Hit(key string) {
    l.mu.Lock()
    defer l.mu.Unlock()
    now := time.Now()
    if f := l.m[key]; f != nil && now.Before(f.reset) {
        f.n++
        return
    }
    l.m[key] = &failure{n: 1, reset: now.Add(l.window)}
}

func (l *FailureLimiter) Reset(key string) {
    l.mu.Lock()
    delete(l.m, key)
    l.mu.Unlock()
}

```

```

}

// UserLimiter: 60 permintaan/menit per pengguna (ember token, di memori per instans).
type UserLimiter struct {
    mu      sync.Mutex
    m       map[uint]*userEntry
    limit   rate.Limit
    burst   int
}

type userEntry struct {
    lim *rate.Limiter
    seen time.Time
}

func NewUserLimiter(perMinute int) *UserLimiter {
    u := &UserLimiter{m: map[uint]*userEntry{}, limit: rate.Every(time.Minute / time.Duration(perMinute)), burst: perMinute}

    go func() {
        for range time.Tick(5 * time.Minute) {
            u.mu.Lock()
            for k, e := range u.m {
                if time.Since(e.seen) > 10*time.Minute {
                    delete(u.m, k)
                }
            }
            u.mu.Unlock()
        }
    }()

    return u
}

func (u *UserLimiter) Middleware() gin.HandlerFunc {
    return func(c *gin.Context) {
        user := CurrentUser(c)
        if user == nil {
            c.Next()
            return
        }
        u.mu.Lock()
        e := u.m[user.ID]
        if e == nil {
            e = &userEntry{lim: rate.NewLimiter(u.limit, u.burst)}
            u.m[user.ID] = e
        }
        e.seen = time.Now()
        ok := e.lim.Allow()
        u.mu.Unlock()
        if !ok {
            httpx.Abort(c, apperr.RateLimited(time.Second))
            return
        }
        c.Next()
    }
}

```

5.6 Service

5.6.1 Pembantu bersama (log, notifikasi, kebijakan akses)

```

// internal/service/helpers.go
package service

import (
    "strconv"
    "strings"
    "time"

    "fite-arsip-api/internal/domain"
    "fite-arsip-api/internal/storage"

    "github.com/google/uuid"
    "gorm.io/gorm"
    "gorm.io/gorm/clause"
)

var _ storage.Storage // pastikan paket terpakai oleh seluruh service

```

```

func writeLog(tx *gorm.DB, action string, docID, userID *uint, meta domain.JSONMap) error {
    return tx.Create(&domain.StorageLog{DocumentID: docID, UserID: userID, Action: action, Meta: meta}).Error
}

func notifyRoles(tx *gorm.DB, roleKeys []string, typ, title, msg string, data domain.JSONMap) error {
    var ids []uint
    err := tx.Model(&domain.User{}).
        Joins("JOIN roles ON roles.id = users.role_id").
        Where("users.is_active = ? AND roles.`key` IN ?", true, roleKeys).
        Pluck("users.id", &ids).Error
    if err != nil || len(ids) == 0 {
        return err
    }
    rows := make([]domain.Notification, 0, len(ids))
    for _, id := range ids {
        rows = append(rows, domain.Notification{ID: uuid.NewString(), UserID: id, Type: typ, Title: title, Message: msg, Data: data})
    }
    return tx.CreateInBatches(&rows, 100).Error
}

func notifyUser(tx *gorm.DB, userID uint, typ, title, msg string, data domain.JSONMap) error {
    return tx.Create(&domain.Notification{ID: uuid.NewString(), UserID: userID, Type: typ, Title: title, Message: msg, Data: data}).Error
}

// cancelVerification menghapus baris verifikasi dan notifikasi antrean terkait (tanpa rujukan menggantung).
func cancelVerification(tx *gorm.DB, docID uint) error {
    if err := tx.Where("document_id = ?", docID).Delete(&domain.Verification{}).Error; err != nil {
        return err
    }
    return tx.Where("type = ? AND JSON_UNQUOTE(JSON_EXTRACT(data, '$.document_id')) = ?",
        "verification_pending", strconv.FormatUint(uint64(docID), 10)).Delete(&domain.Notification{}).Error
}

// openVerification membuka (atau membuka ulang) antrean dan memberi tahu BAAK dan Super Admin.
func openVerification(tx *gorm.DB, doc *domain.Document, uploaderName string) error {
    now := time.Now().UTC()
    v := domain.Verification{DocumentID: doc.ID, Status: "pending", RequestedBy: doc.UploaderID, RequestedAt: now}
    err := tx.Omit(clause.Associations).Clauses(clause.OnConflict{
        Columns: []clause.Column{{Name: "document_id"}},
        DoUpdates: clause.Assignments(map[string]any{
            "status": "pending", "reason": nil, "requested_by": doc.UploaderID,
            "verified_by": nil, "requested_at": now, "verified_at": nil, "updated_at": now,
        })),
    }).Create(&v).Error
    if err != nil {
        return err
    }
    if err := tx.Where("document_id = ?", doc.ID).First(&v).Error; err != nil {
        return err
    }
    return notifyRoles(tx, []string{domain.RoleBaak, domain.RoleSuperAdmin},
        "verification_pending", "Berkas publik baru menunggu verifikasi",
        doc.Title+" oleh "+uploaderName,
        domain.JSONMap{"document_id": doc.ID, "verification_id": v.ID})
}

// ---- kebijakan akses berkas ----

// VisibleTo: cakupan baca per peran (sumber kebenaran tunggal untuk daftar dan otorisasi).
func VisibleTo(u *domain.User) func(*gorm.DB) *gorm.DB {
    return func(db *gorm.DB) *gorm.DB {
        switch u.Role.Key {
        case domain.RoleSuperAdmin, domain.RoleDekan, domain.RoleBaak:
            return db
        }
        conds := []string{"documents.uploader_id = ?", "documents.owner_id = ?"}
        args := []any{u.ID, u.ID}
        switch u.Role.Key {
        case domain.RoleKaprodi:
            conds = append(conds, "documents.owner_role = ?")
            args = append(args, domain.RoleDosen) // berkas dosen se-fakultas
            if u.ProdiID != nil {
                conds = append(conds, "(documents.owner_role = ? AND documents.prodi_id = ?)")
                args = append(args, domain.RoleMahasiswa, *u.ProdiID)
            }
        case domain.RoleDosen:
            if u.ProdiID != nil {
                conds = append(conds, "(documents.owner_role = ? AND documents.prodi_id = ?)")
                args = append(args, domain.RoleMahasiswa, *u.ProdiID)
            }
        }
    }
}

```

```

    }
    return db.Where("(" + strings.Join(conds, " OR ") + ")", args...)
}

func CanUpdateDocument(u *domain.User, d *domain.Document) bool {
    return u.Role.Key == domain.RoleSuperAdmin || d.UploaderID == u.ID ||
        (u.Role.Key == domain.RoleBaak && d.ProdiID != nil)
}

func CanChangeVisibility(u *domain.User, d *domain.Document) bool {
    return u.Role.Key == domain.RoleSuperAdmin || d.UploaderID == u.ID
}

// CanDeleteDocument: SA semua; pengunggah miliknya; BAAK berkas kategori operasional.
func CanDeleteDocument(u *domain.User, d *domain.Document) bool {
    if u.Role.Key == domain.RoleSuperAdmin || d.UploaderID == u.ID {
        return true
    }
    return u.Role.Key == domain.RoleBaak && d.Category.IsOperational
}

```

5.6.2 AuthService

```

// internal/service/auth.go
package service

import (
    "context"
    "crypto/rand"
    "crypto/sha256"
    "encoding/base64"
    "encoding/hex"
    "errors"
    "log/slog"
    "strconv"
    "strings"
    "time"

    "fite-arsip-api/internal/apperr"
    "fite-arsip-api/internal/config"
    "fite-arsip-api/internal/domain"

    "github.com/golang-jwt/jwt/v5"
    "github.com/google/uuid"
    "golang.org/x/crypto/bcrypt"
    "gorm.io/gorm"
    "gorm.io/gorm/clause"
)

const bcryptCost = 12

var ErrInvalidCredentials = errors.New("kredensial tidak valid")

// dummyHash dipakai agar waktu respons sama untuk email tidak terdaftar (anti-enumerasi).
var dummyHash, _ = bcrypt.GenerateFromPassword([]byte("dummy-password-timing"), bcryptCost)

type Claims struct {
    SID string `json:"sid"`
    jwt.RegisteredClaims
}

type TokenPair struct {
    AccessToken    string
    ExpiresIn      int
    RefreshToken   string
    RefreshExpiresAt time.Time
    User           *domain.User
}

type AuthService struct {
    db *gorm.DB
    cfg config.Config
    now func() time.Time
}

func NewAuthService(db *gorm.DB, cfg config.Config) *AuthService {
    return &AuthService{db: db, cfg: cfg, now: time.Now}
}

```

```
func (s *AuthService) Login(ctx context.Context, email, password, ua, ip string) (*TokenPair, error) {
    var user domain.User
    err := s.db.WithContext(ctx).Preload("Role").Preload("Prodi").
        Where("email = ?", strings.ToLower(strings.TrimSpace(email))).First(&user).Error
    found := err == nil
    if err != nil && !errors.Is(err, gorm.ErrRecordNotFound) {
        return nil, err
    }
    hash := dummyHash
    if found {
        hash = []byte(user.PasswordHash)
    }
    pwOK := bcrypt.CompareHashAndPassword(hash, []byte(password)) == nil
    if !found || !pwOK {
        return nil, ErrInvalidCredentials
    }
    if !user.IsActive {
        return nil, apperr.AccountDisabled()
    }
    return s.startSession(ctx, &user, ua, ip)
}

func (s *AuthService) startSession(ctx context.Context, user *domain.User, ua, ip string) (*TokenPair, error) {
    now := s.now().UTC()
    raw, hash, err := newRefreshToken()
    if err != nil {
        return nil, err
    }
    absolute := now.Add(s.cfg.SessionMaxAge)
    exp := minTime(now.Add(s.cfg.RefreshTTL), absolute)
    sess := domain.AuthSession{
        ID: uuid.NewString(), UserID: user.ID, RefreshTokenHash: hash,
        UserAgent: truncate(ua, 255), IP: ip, ExpiresAt: exp, AbsoluteExpiresAt: absolute, LastUsedAt: &now,
    }
    if err := s.db.WithContext(ctx).Create(&sess).Error; err != nil {
        return nil, err
    }
    access, err := s.signAccess(user.ID, sess.ID, now)
    if err != nil {
        return nil, err
    }
    return &TokenPair{AccessToken: access, ExpiresIn: int(s.cfg.AccessTTL.Seconds()),
        RefreshToken: raw, RefreshExpiresAt: exp, User: user}, nil
}

func (s *AuthService) signAccess(uid uint, sid string, now time.Time) (string, error) {
    claims := Claims{SID: sid, RegisteredClaims: jwt.RegisteredClaims{
        Subject:   strconv.FormatUint(uint64(uid), 10),
        Issuer:    s.cfg.JWTIssuer,
        Audience:  jwt.ClaimStrings{s.cfg.JWTAudience},
        IssuedAt:  jwt.NewNumericDate(now),
        ExpiresAt: jwt.NewNumericDate(now.Add(s.cfg.AccessTTL)),
        ID:        uuid.NewString(),
    }}
    return jwt.NewWithClaims(jwt.SigningMethodHS256, claims).SignedString(s.cfg.JWTSecret)
}

// ParseAccess memverifikasi tanda tangan (algoritma dikunci HS256), exp, iss, dan aud.
func (s *AuthService) ParseAccess(raw string) (*Claims, error) {
    claims := &Claims{}
    _, err := jwt.ParseWithClaims(raw, claims,
        func(*jwt.Token) (any, error) { return s.cfg.JWTSecret, nil },
        jwt.WithValidMethods([]string{"HS256"}),
        jwt.WithIssuer(s.cfg.JWTIssuer),
        jwt.WithAudience(s.cfg.JWTAudience),
        jwt.WithExpirationRequired(),
        jwt.WithLeeway(5*time.Second),
    )
    if err != nil || claims.SID == "" {
        return nil, apperr.Unauthenticated("")
    }
    return claims, nil
}

// Authenticate: token sah + sesi aktif + akun aktif; peran dibaca dari DB pada setiap permintaan.
func (s *AuthService) Authenticate(ctx context.Context, raw string) (*domain.User, *domain.AuthSession, error) {
    claims, err := s.ParseAccess(raw)
    if err != nil {
        return nil, nil, err
    }
    uid, err := strconv.ParseUint(claims.Subject, 10, 63/4)
    if err != nil {

```

```

    return nil, nil, apperr.Unauthenticated("")
}
now := s.now().UTC()
var sess domain.AuthSession
err = s.db.WithContext(ctx).
    Where("id = ? AND user_id = ? AND revoked_at IS NULL AND expires_at > ? AND absolute_expires_at > ?",
        claims.SID, uid, now, now).First(&sess).Error
if errors.Is(err, gorm.ErrRecordNotFound) {
    return nil, nil, apperr.Unauthenticated("")
}
if err != nil {
    return nil, nil, err
}
var user domain.User
err = s.db.WithContext(ctx).Preload("Role").Preload("Prodi").First(&user, uid).Error
if errors.Is(err, gorm.ErrRecordNotFound) || (err == nil && !user.IsActive) {
    return nil, nil, apperr.Unauthenticated("")
}
if err != nil {
    return nil, nil, err
}
return &user, &sess, nil
}

// Refresh memutar refresh token. Pemakaian ulang token lama mencabut sesi (deteksi pencurian).
func (s *AuthService) Refresh(ctx context.Context, rawToken, ua, ip string) (*TokenPair, error) {
    hash := hashToken(rawToken)
    now := s.now().UTC()
    var pair *TokenPair

    err := s.db.WithContext(ctx).Transaction(func(tx *gorm.DB) error {
        var sess domain.AuthSession
        err := tx.Clauses(clause.Locking{Strength: "UPDATE"}).Where("refresh_token_hash = ?", hash).First(&sess).Error

        if errors.Is(err, gorm.ErrRecordNotFound) {
            res := tx.Model(&domain.AuthSession{}).
                Where("previous_refresh_hash = ? AND revoked_at IS NULL", hash).Update("revoked_at", now)
            if res.Error != nil {
                return res.Error
            }
            if res.RowsAffected > 0 {
                slog.Warn("pemakaian ulang refresh token terdeteksi; sesi dicabut")
            }
            return nil // pair tetap nil → 401 (pencabutan di-commit)
        }
        if err != nil {
            return err
        }
        if sess.RevokedAt != nil || !now.Before(sess.ExpiresAt) || !now.Before(sess.AbsoluteExpiresAt) {
            return nil
        }
        var user domain.User
        err = tx.Preload("Role").Preload("Prodi").First(&user, sess.UserID).Error
        if errors.Is(err, gorm.ErrRecordNotFound) || (err == nil && !user.IsActive) {
            return tx.Model(&sess).Update("revoked_at", now).Error
        }
        if err != nil {
            return err
        }
        raw, newHash, err := newRefreshToken()
        if err != nil {
            return err
        }
        exp := minTime(now.Add(s.cfg.RefreshTTL), sess.AbsoluteExpiresAt)
        if err := tx.Model(&sess).Updates(map[string]any{
            "previous_refresh_hash": sess.RefreshTokenHash,
            "refresh_token_hash":    newHash,
            "expires_at":             exp,
            "last_used_at":          now,
            "user_agent":             truncate(ua, 255),
            "ip":                    ip,
        }).Error; err != nil {
            return err
        }
        access, err := s.signAccess(user.ID, sess.ID, now)
        if err != nil {
            return err
        }
        pair = &TokenPair{AccessToken: access, ExpiresIn: int(s.cfg.AccessTTL.Seconds()),
            RefreshToken: raw, RefreshExpiresAt: exp, User: &user}
        return nil
    })
}

```

```

    if err != nil {
        return nil, err
    }
    if pair == nil {
        return nil, apperr.Unauthenticated("")
    }
    return pair, nil
}

func (s *AuthService) RevokeSession(ctx context.Context, sid string) error {
    return s.db.WithContext(ctx).Model(&domain.AuthSession{}).
        Where("id = ? AND revoked_at IS NULL", sid).Update("revoked_at", s.now().UTC()).Error
}

func (s *AuthService) RevokeByRefreshToken(ctx context.Context, raw string) (bool, error) {
    h := hashToken(raw)
    res := s.db.WithContext(ctx).Model(&domain.AuthSession{}).
        Where("(refresh_token_hash = ? OR previous_refresh_hash = ?) AND revoked_at IS NULL", h, h).
        Update("revoked_at", s.now().UTC())
    return res.RowsAffected > 0, res.Error
}

func (s *AuthService) UpdateProfile(ctx context.Context, userID uint, name, phone string) error {
    var p any
    if phone != "" {
        p = phone
    }
    return s.db.WithContext(ctx).Model(&domain.User{}).Where("id = ?", userID).
        Updates(map[string]any{"name": name, "phone": p}).Error
}

func (s *AuthService) ChangePassword(ctx context.Context, user *domain.User, currentSID, current, next string) error {
    if bcrypt.CompareHashAndPassword([]byte(user.PasswordHash), []byte(current)) != nil {
        return apperr.FieldError("current_password", "Kata sandi lama tidak sesuai.")
    }
    if current == next {
        return apperr.FieldError("password", "Kata sandi baru harus berbeda dari kata sandi lama.")
    }
    h, err := bcrypt.GenerateFromPassword([]byte(next), bcryptCost)
    if err != nil {
        return err
    }
    now := s.now().UTC()
    return s.db.WithContext(ctx).Transaction(func(tx *gorm.DB) error {
        if err := tx.Model(&domain.User{}).Where("id = ?", user.ID).
            Updates(map[string]any{"password": string(h), "must_change_password": false}).Error; err != nil {
            return err
        }
        // sesi lain dicabut; sesi yang sedang dipakai tetap berlaku
        return tx.Model(&domain.AuthSession{}).
            Where("user_id = ? AND id <> ? AND revoked_at IS NULL", user.ID, currentSID).
            Update("revoked_at", now).Error
    })
}

// ---- util ----

func newRefreshToken() (raw, hash string, err error) {
    b := make([]byte, 32)
    if _, err = rand.Read(b); err != nil {
        return "", "", err
    }
    raw = base64.RawURLEncoding.EncodeToString(b)
    return raw, hashToken(raw), nil
}

func hashToken(raw string) string {
    sum := sha256.Sum256([]byte(raw))
    return hex.EncodeToString(sum[:])
}

func minTime(a, b time.Time) time.Time {
    if a.Before(b) {
        return a
    }
    return b
}

func truncate(s string, n int) string {
    if len(s) <= n {
        return s
    }

```

```

}
return s[:n]
}

```

5.6.3 Validasi Berkas (ekstensi, ukuran, isi)

```

// internal/service/filecheck.go
package service

import (
    "archive/zip"
    "bytes"
    "io"
    "mime/multipart"
    "net/http"
    "path/filepath"
    "strings"
    "unicode"

    "fite-arsip-api/internal/apperr"
)

var mimeByExt = map[string]string{
    "pdf": "application/pdf",
    "docx": "application/vnd.openxmlformats-officedocument.wordprocessingml.document",
    "xlsx": "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
    "pptx": "application/vnd.openxmlformats-officedocument.presentationml.presentation",
    "zip": "application/zip",
    "png": "image/png",
    "jpg": "image/jpeg",
}

var officeDir = map[string]string{"docx": "word/", "xlsx": "xl/", "pptx": "ppt/"

var blockedInner = map[string]bool{
    "exe": true, "php": true, "phtml": true, "js": true, "sh": true, "bat": true, "cmd": true,
    "com": true, "scr": true, "msi": true, "jar": true, "vbs": true, "ps1": true, "html": true, "htm": true, "svg":
true,
}

const formatMsg = "Format berkas harus salah satu dari: pdf, docx, xlsx, pptx, zip, png, jpg. " +
    "Video atau berkas besar tidak diunggah; tulis tautannya pada berkas teks lalu kompres menjadi .zip."

type UploadInfo struct {
    OriginalName string
    Ext          string
    Mime         string
    SizeBytes    int64
    File         multipart.File
}

// InspectUpload memeriksa ukuran, ekstensi, ekstensi ganda, dan isi berkas. Pemanggil menutup File.
func InspectUpload(fh *multipart.FileHeader, maxBytes int64) (*UploadInfo, error) {
    if fh.Size > maxBytes {
        return nil, apperr.FieldError("file", "Ukuran berkas maksimal 10 MB.")
    }
    if fh.Size == 0 {
        return nil, apperr.FieldError("file", "Berkas kosong.")
    }
    name := SanitizeName(fh.Filename)
    ext := strings.TrimPrefix(strings.ToLower(filepath.Ext(name)), ".")
    if ext == "jpeg" {
        ext = "jpg"
    }
    if _, ok := mimeByExt[ext]; !ok {
        return nil, apperr.FieldError("file", formatMsg)
    }
    parts := strings.Split(strings.ToLower(name), ".")
    if len(parts) > 2 {
        for _, p := range parts[1 : len(parts)-1] {
            if blockedInner[p] {
                return nil, apperr.FieldError("file", "Nama berkas mengandung ekstensi ganda yang tidak diizinkan.")
            }
        }
    }
    f, err := fh.Open()
    if err != nil {
        return nil, err
    }
    head := make([]byte, 512)

```

```

n, err := io.ReadFull(f, head)
if err != nil && err != io.ErrUnexpectedEOF && err != io.EOF {
    f.Close()
    return nil, err
}
if err := verifyContent(ext, head[:n], f, fh.Size); err != nil {
    f.Close()
    return nil, err
}
if _, err := f.Seek(0, io.SeekStart); err != nil {
    f.Close()
    return nil, err
}
return &UploadInfo{OriginalName: name, Ext: ext, Mime: mimeByExt[ext], SizeBytes: fh.Size, File: f}, nil
}

func verifyContent(ext string, head []byte, f io.ReaderAt, size int64) error {
    bad := apperr.FieldError("file", "Isi berkas tidak sesuai dengan ekstensinya (."+ext+").")
    sniff := http.DetectContentType(head)
    switch ext {
    case "pdf":
        if !bytes.HasPrefix(head, []byte("%PDF-")) {
            return bad
        }
    case "png":
        if sniff != "image/png" {
            return bad
        }
    case "jpg":
        if sniff != "image/jpeg" {
            return bad
        }
    case "zip", "docx", "xlsx", "pptx":
        if !bytes.HasPrefix(head, []byte("PK\x03\x04")) {
            return bad
        }
        zr, err := zip.NewReader(f, size)
        if err != nil {
            return bad
        }
        if dir, office := officeDir[ext]; office {
            var hasTypes, hasDir bool
            for i, zf := range zr.File {
                if i > 10000 {
                    break
                }
                hasTypes = hasTypes || zf.Name == "[Content_Types].xml"
                hasDir = hasDir || strings.HasPrefix(zf.Name, dir)
            }
            if !hasTypes || !hasDir {
                return bad
            }
        }
    }
    return nil
}

// SanitizeName menghilangkan jalur direktori dan karakter berbahaya; batas 200 karakter.
func SanitizeName(name string) string {
    name = filepath.Base(strings.ReplaceAll(name, `\'`, "/"))
    var b strings.Builder
    for _, r := range name {
        switch {
        case unicode.IsControl(r):
            continue
        case unicode.IsLetter(r), unicode.IsDigit(r), r == ' ', r == '.', r == '_', r == '-', r == '(', r == ')':
            b.WriteRune(r)
        default:
            b.WriteRune('_')
        }
    }
    out := strings.TrimSpace(b.String())
    if rs := []rune(out); len(rs) > 200 {
        out = string(rs[len(rs)-200:]) // pertahankan ekstensi di ujung
    }
    if out == "" || out == "." || out == ".." {
        return "berkas"
    }
    return out
}

```

SRS Sistem Informasi Arsip Berkas Digital FITE

a"

p := likePattern(f.Q)

q = q.Where("(documents.title LIKE ? OR documents.original_name LIKE ?)", p, p)

}

if f.Prodi != "" {

q = q.Where("documents.prodi_id = (SELECT id FROM prodi WHERE code = ?)", f.Prodi)

}

if f.CategoryID != nil {

q = q.Where("documents.category_id = ?", *f.CategoryID)

}

if f.Extension != "" {

q = q.Where("documents.extension = ?", f.Extension)

}

if public {

return q

}

if f.UploaderRole != "" {

q = q.Where("documents.uploader_id IN (SELECT u.id FROM users u JOIN roles r ON r.id = u.role_id WHERE r.`key` = ?)", f.UploaderRole)

}

if f.NIM != "" {

q = q.Where("documents.owner_id IN (SELECT id FROM users WHERE nim = ?)", f.NIM)

}

if f.NIP != "" {

q = q.Where("documents.owner_id IN (SELECT id FROM users WHERE nip = ?)", f.NIP)

}

switch f.Visibility {

case "private":

q = q.Where("documents.visibility = ?", "private")

case "public_approved":

q = q.Where("documents.visibility = ? AND documents.verification_status = ?", "public", "approved")

}

return q

}

func withRelations(db *gorm.DB) *gorm.DB {

unscoped := func(tx *gorm.DB) *gorm.DB { return tx.Unscoped() } // pengguna yang di-soft-delete tetap tampil sebagai pengunggah

return db.Preload("Category").Preload("Prodi").

Preload("Uploader", unscoped).Preload("Uploader.Role").

Preload("Owner", unscoped).Preload("Owner.Role")

}

func (s *DocumentService) List(ctx context.Context, actor *domain.User, f DocumentFilter, publicOnly bool) ([]domain.Document, int64, error) {

q := s.db.WithContext(ctx).Model(&domain.Document{})

if publicOnly {

q = q.Where("documents.visibility = ? AND documents.verification_status = ?", "public", "approved")

} else {

q = q.Scopes(VisibleTo(actor))

}

base := applyFilter(q, f, publicOnly).Session(&gorm.Session{})

var total int64

if err := base.Count(&total).Error; err != nil {

return nil, 0, err

}

docs := make([]domain.Document, 0, f.PerPageNum())

err := withRelations(base).Order(sortClauses[f.Sort]).

Offset((f.PageNum() - 1) * f.PerPageNum()).Limit(f.PerPageNum()).Find(&docs).Error

return docs, total, err

}

func (s *DocumentService) Get(ctx context.Context, id uint) (*domain.Document, error) {

var d domain.Document

if err := withRelations(s.db.WithContext(ctx)).First(&d, id).Error; err != nil {

return nil, err

}

return &d, nil

}

// GetUnscoped termasuk berkas yang sudah di-soft-delete (untuk penghapusan oleh Super Admin).

func (s *DocumentService) GetUnscoped(ctx context.Context, id uint) (*domain.Document, error) {

var d domain.Document

if err := withRelations(s.db.WithContext(ctx).Unscoped()).First(&d, id).Error; err != nil {

return nil, err

}

return &d, nil

}

func (s *DocumentService) CanView(ctx context.Context, actor *domain.User, d *domain.Document) (bool, error) {

if d.Visibility == "public" && d.VerificationStatus == nil && *d.VerificationStatus == "approved" {

return true, nil

```

    }
    var n int64
    err := s.db.WithContext(ctx).Model(&domain.Document{}).Scopes(VisibleTo(actor)).
        Where("documents.id = ?", d.ID).Count(&n).Error
    return n > 0, err
}

// ----- unggah

type CreateInput struct {
    Actor      *domain.User
    Title      string
    Description *string
    CategoryID uint
    Public     bool
    ProdiID    *uint
    SubjectUserID *uint
    ArchivedAt *time.Time
    Upload     *UploadInfo
}

// Create: unggah ke GCS lebih dulu, lalu metadata dalam satu transaksi. Bila transaksi gagal,
// objek di GCS dihapus kembali (kompensasi) sehingga tidak ada objek yatim (SLA-DI-03).
func (s *DocumentService) Create(ctx context.Context, in CreateInput) (*domain.Document, error) {
    actor := in.Actor
    var cat domain.Category
    if err := s.db.WithContext(ctx).First(&cat, in.CategoryID).Error; err != nil {
        if errors.Is(err, gorm.ErrRecordNotFound) {
            return nil, apperr.FieldError("category_id", "Kategori tidak ditemukan.")
        }
        return nil, err
    }
    if !cat.AllowedRoles.Contains(actor.Role.Key) {
        return nil, apperr.Forbidden("Kategori ini tidak diizinkan untuk peran Anda.")
    }

    privileged := actor.Role.Key == domain.RoleSuperAdmin || actor.Role.Key == domain.RoleBaak
    if !privileged {
        fields := map[string][]string{}
        if in.SubjectUserID != nil {
            fields["subject_user_id"] = []string{"Field ini tidak diizinkan untuk peran Anda."}
        }
        if in.ArchivedAt != nil {
            fields["archived_at"] = []string{"Field ini tidak diizinkan untuk peran Anda."}
        }
        if len(fields) > 0 {
            return nil, apperr.Validation(fields)
        }
    }

    owner := actor
    if in.SubjectUserID != nil && *in.SubjectUserID != actor.ID {
        var subj domain.User
        err := s.db.WithContext(ctx).Preload("Role").Preload("Prodi").First(&subj, *in.SubjectUserID).Error
        if errors.Is(err, gorm.ErrRecordNotFound) {
            return nil, apperr.FieldError("subject_user_id", "Pengguna subjek tidak ditemukan.")
        }
        if err != nil {
            return nil, err
        }
        switch subj.Role.Key {
        case domain.RoleMahasiswa, domain.RoleDosen, domain.RoleKaprodi:
            default:
                return nil, apperr.FieldError("subject_user_id", "Subjek berkas harus mahasiswa, dosen, atau kaprodi.")
        }
        owner = &subj
    }

    var prodiID *uint
    switch {
    case owner.ProdiID != nil:
        if in.ProdiID != nil && *in.ProdiID != *owner.ProdiID {
            return nil, apperr.FieldError("prodi_id", "Prodi tidak sesuai dengan prodi pemilik berkas.")
        }
        prodiID = owner.ProdiID
    case privileged:
        if in.ProdiID == nil {
            return nil, apperr.FieldError("prodi_id", "Program studi wajib dipilih.")
        }
        prodiID = in.ProdiID
    }
    prodiCode := "fakultas"

```

```

    if prodiID != nil {
        var p domain.Prodi
        if err := s.db.WithContext(ctx).First(&p, *prodiID).Error; err != nil {
            if errors.Is(err, gorm.ErrRecordNotFound) {
                return nil, apperr.FieldError("prodi_id", "Program studi tidak ditemukan.")
            }
            return nil, err
        }
        if !p.IsActive {
            return nil, apperr.FieldError("prodi_id", "Program studi tidak aktif.")
        }
        prodiCode = p.Code
    }

    now := s.now().UTC()
    archived := now
    if in.ArchivedAt != nil {
        archived = in.ArchivedAt.UTC()
    }
    path := fmt.Sprintf("documents/%s/%04d/%02d/%s.%s", prodiCode, now.Year(), int(now.Month()), uuid.NewString(), in.Upload.Ext)

    if _, err := in.Upload.File.Seek(0, io.SeekStart); err != nil {
        return nil, err
    }
    if err := s.store.Put(ctx, path, in.Upload.File, in.Upload.Mime); err != nil {
        return nil, apperr.Internal(fmt.Errorf("gagal menyimpan ke GCS: %w", err))
    }

    visibility := "private"
    var vstatus *string
    if in.Public {
        visibility = "public"
        p := "pending"
        vstatus = &p
    }
    doc := domain.Document{
        Title: in.Title, Description: in.Description,
        OriginalName: in.Upload.OriginalName, Extension: in.Upload.Ext, MimeType: in.Upload.Mime,
        SizeKB: uint(max64(1, (in.Upload.SizeBytes+1023)/1024)), StoragePath: path,
        CategoryID: cat.ID, ProdiID: prodiID, UploaderID: actor.ID, OwnerID: owner.ID, OwnerRole: owner.Role.Key,
        Visibility: visibility, VerificationStatus: vstatus, ArchivedAt: &archived,
    }
    err := s.db.WithContext(ctx).Transaction(func(tx *gorm.DB) error {
        if err := tx.Omit(clause.Associations).Create(&doc).Error; err != nil {
            return err
        }
        if in.Public {
            if err := openVerification(tx, &doc, actor.Name); err != nil {
                return err
            }
        }
        return writeLog(tx, "upload", &doc.ID, &actor.ID, domain.JSONMap{"size_kb": doc.SizeKB, "visibility": doc.Visi
sibility})
    })
    if err != nil {
        cctx, cancel := context.WithTimeout(context.Background(), 15*time.Second)
        defer cancel()
        if derr := s.store.Delete(cctx, path); derr != nil {
            slog.Error("kompensasi hapus objek GCS gagal", "path", path, "error", derr)
        }
        return nil, err
    }
    return s.Get(ctx, doc.ID)
}

func max64(a, b int64) int64 {
    if a > b {
        return a
    }
    return b
}

// ----- ubah, visibilitas, unduh, hapus

type UpdateMetaInput struct {
    Title      *string
    Description *string
    CategoryID *uint
}

func (s *DocumentService) UpdateMeta(ctx context.Context, actor *domain.User, d *domain.Document, in UpdateMetaInput

```

```

t) (*domain.Document, error) {
    updates := map[string]any{}
    if in.Title != nil {
        updates["title"] = *in.Title
    }
    if in.Description != nil {
        if *in.Description == "" {
            updates["description"] = nil
        } else {
            updates["description"] = *in.Description
        }
    }
    if in.CategoryID != nil {
        var cat domain.Category
        if err := s.db.WithContext(ctx).First(&cat, *in.CategoryID).Error; err != nil {
            if errors.Is(err, gorm.ErrRecordNotFound) {
                return nil, apperr.FieldError("category_id", "Kategori tidak ditemukan.")
            }
            return nil, err
        }
        if !cat.AllowedRoles.Contains(d.Uploader.Role.Key) {
            return nil, apperr.FieldError("category_id", "Kategori tidak sesuai dengan peran pengunggah berkas.")
        }
        updates["category_id"] = cat.ID
    }
    if len(updates) == 0 {
        return nil, apperr.FieldError("body", "Tidak ada perubahan yang dikirim.")
    }
    err := s.db.WithContext(ctx).Transaction(func(tx *gorm.DB) error {
        if err := tx.Model(&domain.Document{}).Where("id = ?", d.ID).Updates(updates).Error; err != nil {
            return err
        }
        keys := make([]string, 0, len(updates))
        for k := range updates {
            keys = append(keys, k)
        }
        return writeLog(tx, "update", &d.ID, &actor.ID, domain.JSONMap{"fields": keys})
    })
    if err != nil {
        return nil, err
    }
    return s.Get(ctx, d.ID)
}

// ChangeVisibility: baris dokumen dikunci lebih dulu, lalu verifikasi (urutan kunci sama dengan Decide).
func (s *DocumentService) ChangeVisibility(ctx context.Context, actor *domain.User, id uint, target string) (*domain.Document, error) {
    err := s.db.WithContext(ctx).Transaction(func(tx *gorm.DB) error {
        var doc domain.Document
        if err := tx.Clauses(clause.Locking{Strength: "UPDATE"}).First(&doc, id).Error; err != nil {
            return err
        }
        if target == "public" {
            if doc.Visibility == "public" {
                return apperr.Conflict("Berkas sudah berstatus publik.")
            }
            pending := "pending"
            if err := tx.Model(&doc).Updates(map[string]any{"visibility": "public", "verification_status": pending}).Error; err != nil {
                return err
            }
            var up domain.User
            if err := tx.Unscoped().First(&up, doc.UploaderID).Error; err != nil {
                return err
            }
            if err := openVerification(tx, &doc, up.Name); err != nil {
                return err
            }
        } else {
            if doc.Visibility == "private" {
                return nil // idempoten (termasuk berkas rejected)
            }
            if err := cancelVerification(tx, doc.ID); err != nil {
                return err
            }
            if err := tx.Model(&doc).Updates(map[string]any{"visibility": "private", "verification_status": nil}).Error; err != nil {
                return err
            }
        }
        return writeLog(tx, "visibility_change", &doc.ID, &actor.ID, domain.JSONMap{"to": target})
    })
}

```

```

    if err != nil {
        return nil, err
    }
    return s.Get(ctx, id)
}

func (s *DocumentService) Download(ctx context.Context, actor *domain.User, d *domain.Document) (io.ReadCloser, int64, error) {
    rc, size, err := s.store.Open(ctx, d.StoragePath)
    if errors.Is(err, storage.ErrNotFound) {
        return nil, 0, apperr.NotFound("Objek berkas tidak ditemukan di penyimpanan.")
    }
    if err != nil {
        return nil, 0, err
    }
    if lerr := writeLog(s.db.WithContext(ctx), "download", &d.ID, &actor.ID, nil); lerr != nil {
        slog.Error("log unduhan gagal", "document_id", d.ID, "error", lerr)
    }
    return rc, size, nil
}

// Delete: soft delete, atau hard delete (hanya Super Admin; dijaga handler). Pada hard delete, objek GCS
// dihapus terakhir di dalam transaksi: GCS gagal → rollback, metadata utuh, operasi dapat diulang.
func (s *DocumentService) Delete(ctx context.Context, actor *domain.User, d *domain.Document, force bool) error {
    return s.db.WithContext(ctx).Transaction(func(tx *gorm.DB) error {
        if err := cancelVerification(tx, d.ID); err != nil {
            return err
        }
        if force {
            if err := writeLog(tx, "hard_delete", &d.ID, &actor.ID, domain.JSONMap{"title": d.Title}); err != nil {
                return err
            }
            if err := tx.Unscoped().Delete(&domain.Document{}, d.ID).Error; err != nil {
                return err
            }
            return s.store.Delete(ctx, d.StoragePath)
        }
        if err := tx.Model(&domain.Document{}).Where("id = ?", d.ID).
            Updates(map[string]any{"verification_status": nil, "visibility": "private"}).Error; err != nil {
            return err
        }
        if err := tx.Delete(&domain.Document{}, d.ID).Error; err != nil {
            return err
        }
        return writeLog(tx, "soft_delete", &d.ID, &actor.ID, nil)
    })
}

### 5.6.5 VerificationService

```go
// internal/service/verification.go
package service

import (
 "context"
 "time"

 "fite-arsip-api/internal/apperr"
 "fite-arsip-api/internal/domain"

 "gorm.io/gorm"
 "gorm.io/gorm/clause"
)

type VerificationService struct {
 db *gorm.DB
 now func() time.Time
}

func NewVerificationService(db *gorm.DB) *VerificationService {
 return &VerificationService{db: db, now: time.Now}
}

type VerificationFilter struct {
 Status string
 Prodi string
 DocumentID *uint
 Page int
 PerPage int
}

```

```

func verificationRelations(db *gorm.DB) *gorm.DB {
 unscoped := func(tx *gorm.DB) *gorm.DB { return tx.Unscoped() }
 return db.Preload("Document", unscoped).Preload("Document.Prodi").Preload("Requester").Preload("Verifier")
}

func (s *VerificationService) List(ctx context.Context, f VerificationFilter) ([]domain.Verification, int64, error) {
 q := s.db.WithContext(ctx).Model(&domain.Verification{})
 if f.Status != "" {
 q = q.Where("document_verifications.status = ?", f.Status)
 }
 if f.DocumentID != nil {
 q = q.Where("document_verifications.document_id = ?", *f.DocumentID)
 }
 if f.Prodi != "" {
 q = q.Where("document_verifications.document_id IN (SELECT d.id FROM documents d JOIN prodi p ON p.id = d.prodi_id WHERE p.code = ?)", f.Prodi)
 }
 base := q.Session(&gorm.Session{})

 var total int64
 if err := base.Count(&total).Error; err != nil {
 return nil, 0, err
 }
 order := "document_verifications.requested_at DESC, document_verifications.id DESC"
 if f.Status == "pending" {
 order = "document_verifications.requested_at ASC, document_verifications.id ASC" // paling lama menunggu dulu
 }
 items := make([]domain.Verification, 0, f.PerPage)
 err := verificationRelations(base).Order(order).Offset((f.Page - 1) * f.PerPage).Limit(f.PerPage).Find(&items).Error
 return items, total, err
}

func (s *VerificationService) Get(ctx context.Context, id uint) (*domain.Verification, error) {
 var v domain.Verification
 if err := verificationRelations(s.db.WithContext(ctx)).First(&v, id).Error; err != nil {
 return nil, err
 }
 return &v, nil
}

// Decide: Pending → Approved|Rejected. Dokumen dikunci lebih dulu, lalu verifikasi (urutan sama dengan
// DocumentService.ChangeVisibility) sehingga keputusan bersamaan hanya berhasil satu (409 untuk lainnya).
func (s *VerificationService) Decide(ctx context.Context, actor *domain.User, id uint, status, reason string) (*domain.Verification, error) {
 err := s.db.WithContext(ctx).Transaction(func(tx *gorm.DB) error {
 var probe domain.Verification
 if err := tx.Select("id", "document_id").First(&probe, id).Error; err != nil {
 return err
 }
 var doc domain.Document
 if err := tx.Clauses(clause.Locking{Strength: "UPDATE"}).First(&doc, probe.DocumentID).Error; err != nil {
 return err
 }
 var v domain.Verification
 if err := tx.Clauses(clause.Locking{Strength: "UPDATE"}).First(&v, id).Error; err != nil {
 return err
 }
 if v.Status != "pending" {
 return apperr.Conflict("Berkas ini sudah diverifikasi.")
 }

 var reasonVal any
 if status == "rejected" {
 reasonVal = reason
 }
 if err := tx.Model(&v).Updates(map[string]any{
 "status": status, "reason": reasonVal, "verified_by": actor.ID, "verified_at": s.now().UTC(),
 }).Error; err != nil {
 return err
 }

 docUpdate := map[string]any{"visibility": "private", "verification_status": "rejected"}
 title, body := "Berkas Anda ditolak", ``+doc.Title+` ditolak. Alasan: `+reason
 if status == "approved" {
 docUpdate = map[string]any{"visibility": "public", "verification_status": "approved"}
 title, body = "Berkas Anda disetujui", ``+doc.Title+` kini tampil di explorer publik.`
 }
 if err := tx.Model(&doc).Updates(docUpdate).Error; err != nil {
 return err
 }
 })
}

```

```

 }
 meta := domain.JSONMap{}
 if status == "rejected" {
 meta["reason"] = reason
 }
 if err := writeLog(tx, "verify_"+status, &doc.ID, &actor.ID, meta); err != nil {
 return err
 }
 return notifyUser(tx, doc.UploaderID, "verification_"+status, title, body,
 domain.JSONMap{"document_id": doc.ID, "verification_id": v.ID})
})
if err != nil {
 return nil, err
}
return s.Get(ctx, id)
}

```

## 5.6.6 RetentionService (dipakai worker)

```

// internal/service/retention.go
package service

import (
 "context"
 "database/sql"
 "fmt"
 "log/slog"
 "time"

 "fite-arsip-api/internal/config"
 "fite-arsip-api/internal/domain"
 "fite-arsip-api/internal/storage"

 "gorm.io/gorm"
)

const (
 retentionLock = "fite_retention_cleanup"
 retentionChunk = 200
)

type RetentionReport struct {
 Skipped bool // instans lain sedang berjalan
 SoftDeleted int
 HardDeleted int
 Failed int
 Anomalies int // berkas tanpa archived_at
 Sessions int64
}

type RetentionService struct {
 db *gorm.DB
 store storage.Storage
 cfg config.Config
 Now func() time.Time // dapat diganti pada pengujian
}

func NewRetentionService(db *gorm.DB, store storage.Storage, cfg config.Config) *RetentionService {
 return &RetentionService{db: db, store: store, cfg: cfg, Now: time.Now}
}

func (s *RetentionService) Run(ctx context.Context) (RetentionReport, error) {
 var rep RetentionReport
 sqlDB, err := s.db.DB()
 if err != nil {
 return rep, err
 }
 // Kunci GET_LOCK terikat pada satu koneksi; koneksi dipegang selama proses berjalan.
 conn, err := sqlDB.Conn(ctx)
 if err != nil {
 return rep, err
 }
 defer conn.Close()
 var got sql.NullInt64
 if err := conn.QueryRowContext(ctx, "SELECT GET_LOCK(?, 0)", retentionLock).Scan(&got); err != nil {
 return rep, err
 }
 if !got.Valid || got.Int64 != 1 {
 rep.Skipped = true
 return rep, nil
 }

```

```

}
defer func() { _, _ = conn.ExecContext(context.Background(), "SELECT RELEASE_LOCK(?)", retentionLock) }()

now := s.Now().UTC()
cutoff := now.AddDate(-s.cfg.RetentionYears, 0, 0) // lebih dari 8 tahun menurut kalender
graceCutoff := now.AddDate(0, 0, -s.cfg.GraceDays) // masa tenggang 90 hari

if err := s.softStage(ctx, cutoff, &rep); err != nil {
 return rep, err
}
if err := s.hardStage(ctx, graceCutoff, &rep); err != nil {
 return rep, err
}

var n int64
if err := s.db.WithContext(ctx).Model(&domain.Document{}).Where("archived_at IS NULL").Count(&n).Error; err == nil {
 rep.Anomalies = int(n)
 if n > 0 {
 slog.Warn("retensi: berkas tanpa archived_at dilewati", "jumlah", n)
 }
}
old := now.AddDate(0, 0, -30)
res := s.db.WithContext(ctx).Where("expires_at < ? OR absolute_expires_at < ? OR revoked_at < ?", old, old, old).Delete(&domain.AuthSession{})
rep.Sessions = res.RowsAffected
return rep, res.Error
}

func (s *RetentionService) softStage(ctx context.Context, cutoff time.Time, rep *RetentionReport) error {
 var last uint
 for {
 var docs []domain.Document
 err := s.db.WithContext(ctx).Where("id > ? AND archived_at IS NOT NULL AND archived_at < ?", last, cutoff).Order("id").Limit(retentionChunk).Find(&docs).Error
 if err != nil {
 return err
 }
 if len(docs) == 0 {
 return nil
 }
 for i := range docs {
 last = docs[i].ID
 if err := s.expireSoft(ctx, &docs[i]); err != nil {
 rep.Failed++
 slog.Error("retensi soft delete gagal", "document_id", docs[i].ID, "error", err)
 continue
 }
 rep.SoftDeleted++
 }
 }
}

func (s *RetentionService) hardStage(ctx context.Context, graceCutoff time.Time, rep *RetentionReport) error {
 var last uint
 for {
 var docs []domain.Document
 err := s.db.WithContext(ctx).Unscoped().Where("id > ? AND deleted_at IS NOT NULL AND deleted_at <= ?", last, graceCutoff).Order("id").Limit(retentionChunk).Find(&docs).Error
 if err != nil {
 return err
 }
 if len(docs) == 0 {
 return nil
 }
 for i := range docs {
 last = docs[i].ID
 if err := s.expireHard(ctx, &docs[i]); err != nil {
 rep.Failed++ // GCS gagal → rollback, dicoba lagi pada putaran berikutnya
 slog.Error("retensi hard delete gagal", "document_id", docs[i].ID, "error", err)
 continue
 }
 rep.HardDeleted++
 }
 }
}

func (s *RetentionService) expireSoft(ctx context.Context, d *domain.Document) error {
 return s.db.WithContext(ctx).Transaction(func(tx *gorm.DB) error {

```

```

 if err := cancelVerification(tx, d.ID); err != nil {
 return err
 }
 if err := tx.Model(&domain.Document{}).Where("id = ?", d.ID).
 Updates(map[string]any{"retention_status": "expired", "verification_status": nil, "visibility": "private"}).Error; err != nil {
 return err
 }
 if err := tx.Delete(&domain.Document{}, d.ID).Error; err != nil {
 return err
 }
 meta := domain.JSONMap{}
 if d.ArchivedAt != nil {
 meta["archived_at"] = d.ArchivedAt.UTC().Format(time.RFC3339)
 }
 return writeLog(tx, "expired_soft_delete", &d.ID, nil, meta)
 })
}

func (s *RetentionService) expireHard(ctx context.Context, d *domain.Document) error {
 action := "deleted_hard_delete" // dihapus pengguna, bukan karena kedaluwarsa
 if d.RetentionStatus != nil && *d.RetentionStatus == "expired" {
 action = "expired_hard_delete"
 }
 return s.db.WithContext(ctx).Transaction(func(tx *gorm.DB) error {
 if err := writeLog(tx, action, &d.ID, nil, domain.JSONMap{"title": d.Title}); err != nil {
 return err
 }
 if err := tx.Unscoped().Delete(&domain.Document{}, d.ID).Error; err != nil {
 return err
 }
 if err := s.store.Delete(ctx, d.StoragePath); err != nil {
 return fmt.Errorf("hapus objek GCS: %w", err)
 }
 return nil
 })
}

```

## 5.7 Handler

### 5.7.1 DTO Respons dan Pemeta

```

// internal/handler/dto.go
package handler

import (
 "strconv"
 "time"

 "fite-arsip-api/internal/apperr"
 "fite-arsip-api/internal/domain"

 "github.com/gin-gonic/gin"
)

func ts(t time.Time) string { return t.UTC().Format("2006-01-02T15:04:05Z") }

func tsp(t *time.Time) *string {
 if t == nil {
 return nil
 }
 s := ts(*t)
 return &s
}

func idParam(c *gin.Context) (uint, error) {
 n, err := strconv.ParseUint(c.Param("id"), 10, 64)
 if err != nil || n == 0 {
 return 0, apperr.NotFound("")
 }
 return uint(n), nil
}

type ProdiRef struct {
 ID uint `json:"id"`
 Code string `json:"code"`
 Name string `json:"name"`
}

```

```

type UserResponse struct {
 ID uint `json:"id"`
 Name string `json:"name"`
 Email string `json:"email"`
 Role string `json:"role"`
 RoleLabel string `json:"role_label"`
 NIM *string `json:"nim"`
 NIP *string `json:"nip"`
 Phone *string `json:"phone"`
 Prodi *ProdiRef `json:"prodi"`
 IsActive bool `json:"is_active"`
 MustChangePassword bool `json:"must_change_password"`
}

func toProdiRef(p *domain.Prodi) *ProdiRef {
 if p == nil || p.ID == 0 {
 return nil
 }
 return &ProdiRef{ID: p.ID, Code: p.Code, Name: p.Name}
}

func toUserResponse(u *domain.User) UserResponse {
 return UserResponse{
 ID: u.ID, Name: u.Name, Email: u.Email, Role: u.Role.Key, RoleLabel: u.Role.Name,
 NIM: u.NIM, NIP: u.NIP, Phone: u.Phone, Prodi: toProdiRef(u.Prodi),
 IsActive: u.IsActive, MustChangePassword: u.MustChangePassword,
 }
}

type UserRef struct {
 ID uint `json:"id"`
 Name string `json:"name"`
 Role string `json:"role"`
 NIM *string `json:"nim"`
 NIP *string `json:"nip"`
}

func toUserRef(u domain.User) UserRef {
 return UserRef{ID: u.ID, Name: u.Name, Role: u.Role.Key, NIM: u.NIM, NIP: u.NIP}
}

type DocumentResponse struct {
 ID uint `json:"id"`
 Title string `json:"title"`
 Description *string `json:"description"`
 OriginalName string `json:"original_name"`
 Extension string `json:"extension"`
 MimeType string `json:"mime_type"`
 SizeKB uint `json:"size_kb"`
 Category struct {
 ID uint `json:"id"`
 Name string `json:"name"`
 } `json:"category"`
 Prodi *ProdiRef `json:"prodi"`
 Uploader UserRef `json:"uploader"`
 Owner UserRef `json:"owner"`
 Visibility string `json:"visibility"`
 VerificationStatus *string `json:"verification_status"`
 ArchivedAt *string `json:"archived_at"`
 CreatedAt string `json:"created_at"`
 Links struct {
 Self string `json:"self"`
 Download string `json:"download"`
 } `json:"links"`
}

func toDocumentResponse(d *domain.Document) DocumentResponse {
 r := DocumentResponse{
 ID: d.ID, Title: d.Title, Description: d.Description, OriginalName: d.OriginalName,
 Extension: d.Extension, MimeType: d.MimeType, SizeKB: d.SizeKB, Prodi: toProdiRef(d.Prodi),
 Uploader: toUserRef(d.Uploader), Owner: toUserRef(d.Owner),
 Visibility: d.Visibility, VerificationStatus: d.VerificationStatus,
 ArchivedAt: tsp(d.ArchivedAt), CreatedAt: ts(d.CreatedAt),
 }
 r.Category.ID, r.Category.Name = d.Category.ID, d.Category.Name
 base := "/api/v1/documents/" + strconv.FormatUint(uint64(d.ID), 10)
 r.Links.Self, r.Links.Download = base, base+"/download"
 return r
}

func toDocumentResponses(ds []domain.Document) []DocumentResponse {

```

```

 out := make([]DocumentResponse, 0, len(ds))
 for i := range ds {
 out = append(out, toDocumentResponse(&ds[i]))
 }
 return out
}

type VerificationResponse struct {
 ID uint `json:"id"`
 Document struct {
 ID uint `json:"id"`
 Title string `json:"title"`
 Prodi *string `json:"prodi"`
 } `json:"document"`
 Status string `json:"status"`
 Reason *string `json:"reason"`
 RequestedBy struct {
 ID uint `json:"id"`
 Name string `json:"name"`
 } `json:"requested_by"`
 VerifiedBy *struct {
 ID uint `json:"id"`
 Name string `json:"name"`
 } `json:"verified_by"`
 RequestedAt string `json:"requested_at"`
 VerifiedAt *string `json:"verified_at"`
}

func toVerificationResponse(v *domain.Verification) VerificationResponse {
 r := VerificationResponse{ID: v.ID, Status: v.Status, Reason: v.Reason,
 RequestedAt: ts(v.RequestedAt), VerifiedAt: tsp(v.VerifiedAt)}
 r.Document.ID, r.Document.Title = v.Document.ID, v.Document.Title
 if v.Document.Prodi != nil {
 code := v.Document.Prodi.Code
 r.Document.Prodi = &code
 }
 r.RequestedBy.ID, r.RequestedBy.Name = v.Requester.ID, v.Requester.Name
 if v.Verifier != nil {
 r.VerifiedBy = &struct {
 ID uint `json:"id"`
 Name string `json:"name"`
 }{v.Verifier.ID, v.Verifier.Name}
 }
 return r
}

```

## 5.7.2 AuthHandler

```

// internal/handler/auth.go
package handler

import (
 "errors"
 "net/http"
 "strings"
 "time"

 "fite-arsip-api/internal/apperr"
 "fite-arsip-api/internal/config"
 "fite-arsip-api/internal/httpx"
 "fite-arsip-api/internal/middleware"
 "fite-arsip-api/internal/service"

 "github.com/gin-gonic/gin"
)

const (
 refreshCookie = "refresh_token"
 cookiePath = "/api/v1/auth"
)

type AuthHandler struct {
 svc *service.AuthService
 cfg config.Config
 limiter *middleware.FailureLimiter
}

func NewAuthHandler(svc *service.AuthService, cfg config.Config, l *middleware.FailureLimiter) *AuthHandler {
 return &AuthHandler{svc: svc, cfg: cfg, limiter: l}
}

```

```

type loginRequest struct {
 Email string `json:"email" binding:"required,email,max=255"`
 Password string `json:"password" binding:"required,max=255"`
}

type profileRequest struct {
 Name string `json:"name" binding:"required,max=100"`
 Phone string `json:"phone" binding:"omitempty,phone"`
}

type passwordRequest struct {
 CurrentPassword string `json:"current_password" binding:"required,max=255"`
 Password string `json:"password" binding:"required,strongpw"`
 PasswordConfirmation string `json:"password_confirmation" binding:"required,eqfield=Password"`
}

func (h *AuthHandler) tokenBody(p *service.TokenPair) gin.H {
 return gin.H{
 "access_token": p.AccessToken, "token_type": "Bearer", "expires_in": p.ExpiresIn,
 "user": toUserResponse(p.User),
 }
}

func (h *AuthHandler) setRefreshCookie(c *gin.Context, raw string, exp time.Time) {
 http.SetCookie(c.Writer, &http.Cookie{
 Name: refreshCookie, Value: raw, Path: cookiePath, Domain: h.cfg.CookieDomain,
 Expires: exp, MaxAge: int(time.Until(exp).Seconds()),
 HttpOnly: true, Secure: h.cfg.CookieSecure, SameSite: http.SameSiteStrictMode,
 })
}

func (h *AuthHandler) clearRefreshCookie(c *gin.Context) {
 http.SetCookie(c.Writer, &http.Cookie{
 Name: refreshCookie, Value: "", Path: cookiePath, Domain: h.cfg.CookieDomain,
 MaxAge: -1, HttpOnly: true, Secure: h.cfg.CookieSecure, SameSite: http.SameSiteStrictMode,
 })
}

// checkOrigin: pertahanan CSRF tambahan untuk rute yang memakai cookie. Di production header Origin wajib ada.
func (h *AuthHandler) checkOrigin(c *gin.Context) error {
 o := c.GetHeader("Origin")
 if o == "" && !h.cfg.IsProd() {
 return nil // memudahkan pengujian dengan Postman
 }
 if o != h.cfg.AllowedOrigin {
 return apperr.Forbidden("Origin permintaan tidak sah.")
 }
 return nil
}

func (h *AuthHandler) Login(c *gin.Context) error {
 var req loginRequest
 if err := httpx.BindJSON(c, &req); err != nil {
 return err
 }
 c.Header("Cache-Control", "no-store")
 key := strings.ToLower(strings.TrimSpace(req.Email)) + "|" + c.ClientIP()
 if ok, retry := h.limiter.Check(key); !ok {
 return apperr.RateLimited(retry)
 }
 pair, err := h.svc.Login(c.Request.Context(), req.Email, req.Password, c.Request.UserAgent(), c.ClientIP())
 if errors.Is(err, service.ErrInvalidCredentials) {
 h.limiter.Hit(key) // hanya percobaan gagal yang dihitung
 return apperr.Unauthenticated("Email atau kata sandi salah.")
 }
 if err != nil {
 return err
 }
 h.limiter.Reset(key)
 h.setRefreshCookie(c, pair.RefreshToken, pair.RefreshExpiresAt)
 httpx.OK(c, http.StatusOK, "Login berhasil", h.tokenBody(pair))
 return nil
}

func (h *AuthHandler) Refresh(c *gin.Context) error {
 c.Header("Cache-Control", "no-store")
 if err := h.checkOrigin(c); err != nil {
 return err
 }
 raw, err := c.Cookie(refreshCookie)
 if err != nil || raw == "" {

```

```

 }
 pair, err := h.svc.Refresh(c.Request.Context(), raw, c.Request.UserAgent(), c.ClientIP())
 if err != nil {
 h.clearRefreshCookie(c)
 return err
 }
 h.setRefreshCookie(c, pair.RefreshToken, pair.RefreshExpiresAt)
 httpx.OK(c, http.StatusOK, "Token diperbarui", h.tokenBody(pair))
 return nil
}

// Logout tidak memakai middleware Auth agar tetap bekerja saat access token sudah kedaluwarsa
// (sesi ditentukan dari cookie refresh).
func (h *AuthHandler) Logout(c *gin.Context) error {
 if err := h.checkOrigin(c); err != nil {
 return err
 }
 ctx := c.Request.Context()
 revoked := false
 if hdr := c.GetHeader("Authorization"); len(hdr) > 7 && strings.EqualFold(hdr[:7], "Bearer ") {
 if claims, err := h.svc.ParseAccess(strings.TrimSpace(hdr[7:])); err == nil {
 if err := h.svc.RevokeSession(ctx, claims.SID); err != nil {
 return err
 }
 revoked = true
 }
 }
 if raw, err := c.Cookie(refreshCookie); err == nil && raw != "" {
 ok, err := h.svc.RevokeByRefreshToken(ctx, raw)
 if err != nil {
 return err
 }
 revoked = revoked || ok
 }
 h.clearRefreshCookie(c)
 if !revoked {
 return apperr.Unauthenticated("")
 }
 httpx.OK(c, http.StatusOK, "Logout berhasil", nil)
 return nil
}

func (h *AuthHandler) Me(c *gin.Context) error {
 u := middleware.CurrentUser(c)
 httpx.OK(c, http.StatusOK, "OK", toUserResponse(u))
 return nil
}

func (h *AuthHandler) UpdateProfile(c *gin.Context) error {
 var req profileRequest
 if err := httpx.BindJSON(c, &req); err != nil {
 return err
 }
 u := middleware.CurrentUser(c)
 if err := h.svc.UpdateProfile(c.Request.Context(), u.ID, req.Name, req.Phone); err != nil {
 return err
 }
 u.Name = req.Name
 if req.Phone == "" {
 u.Phone = nil
 } else {
 u.Phone = &req.Phone
 }
 httpx.OK(c, http.StatusOK, "Profil diperbarui", toUserResponse(u))
 return nil
}

func (h *AuthHandler) ChangePassword(c *gin.Context) error {
 var req passwordRequest
 if err := httpx.BindJSON(c, &req); err != nil {
 return err
 }
 u, sess := middleware.CurrentUser(c), middleware.CurrentSession(c)
 if err := h.svc.ChangePassword(c.Request.Context(), u, sess.ID, req.CurrentPassword, req.Password); err != nil {
 return err
 }
 httpx.OK(c, http.StatusOK, "Kata sandi diperbarui", nil)
 return nil
}

```

```
// internal/handler/document.go
package handler

import (
 "errors"
 "mime"
 "net/http"
 "strconv"
 "time"

 "fite-arsip-api/internal/apperr"
 "fite-arsip-api/internal/config"
 "fite-arsip-api/internal/domain"
 "fite-arsip-api/internal/httpx"
 "fite-arsip-api/internal/middleware"
 "fite-arsip-api/internal/service"

 "github.com/gin-gonic/gin"
)

type DocumentHandler struct {
 svc *service.DocumentService
 cfg config.Config
}

func NewDocumentHandler(svc *service.DocumentService, cfg config.Config) *DocumentHandler {
 return &DocumentHandler{svc: svc, cfg: cfg}
}

type createDocumentForm struct {
 Title string `form:"title" binding:"required,max=255"`
 Description string `form:"description" binding:"omitempty,max=2000"`
 CategoryID uint `form:"category_id" binding:"required,gt=0"`
 Visibility string `form:"visibility" binding:"omitempty,oneof=private public"`
 ProdiID *uint `form:"prodi_id" binding:"omitempty,gt=0"`
 SubjectUserID *uint `form:"subject_user_id" binding:"omitempty,gt=0"`
 ArchivedAt string `form:"archived_at"`
}

type updateDocumentRequest struct {
 Title *string `json:"title" binding:"omitempty,min=1,max=255"`
 Description *string `json:"description" binding:"omitempty,max=2000"`
 CategoryID *uint `json:"category_id" binding:"omitempty,gt=0"`
}

type visibilityRequest struct {
 Visibility string `json:"visibility" binding:"required,oneof=private public"`
}

// loadViewable memuat berkas dan memastikan pengguna berhak membacanya (403 bila tidak).
func (h *DocumentHandler) loadViewable(c *gin.Context) (*domain.Document, *domain.User, error) {
 id, err := idParam(c)
 if err != nil {
 return nil, nil, err
 }
 doc, err := h.svc.Get(c.Request.Context(), id)
 if err != nil {
 return nil, nil, err
 }
 user := middleware.CurrentUser(c)
 ok, err := h.svc.CanView(c.Request.Context(), user, doc)
 if err != nil {
 return nil, nil, err
 }
 if !ok {
 return nil, nil, apperr.Forbidden("")
 }
 return doc, user, nil
}

func (h *DocumentHandler) Index(c *gin.Context) error {
 return h.list(c, false)
}

func (h *DocumentHandler) PublicIndex(c *gin.Context) error {
 return h.list(c, true)
}

```

```

func (h *DocumentHandler) list(c *gin.Context, public bool) error {
 var f service.DocumentFilter
 if err := httpx.BindQuery(c, &f); err != nil {
 return err
 }
 docs, total, err := h.svc.List(c.Request.Context(), middleware.CurrentUser(c), f, public)
 if err != nil {
 return err
 }
 httpx.Paged(c, "OK", toDocumentResponses(docs), httpx.PageMeta(f.PageNum(), f.PerPageNum(), total))
 return nil
}

func (h *DocumentHandler) Store(c *gin.Context) error {
 if c.ContentType() != "multipart/form-data" {
 return apperr.UnsupportedMedia("Gunakan multipart/form-data untuk mengunggah berkas.")
 }
 var form createDocumentForm
 if err := httpx.BindForm(c, &form); err != nil {
 return err
 }
 fh, err := c.FormFile("file")
 if errors.Is(err, http.ErrMissingFile) {
 return apperr.FieldError("file", "Berkas wajib dilampirkan.")
 }
 if err != nil {
 return err // termasuk MaxBytesError → 413
 }

 var archived *time.Time
 if form.ArchivedAt != "" {
 t, perr := time.Parse(time.RFC3339, form.ArchivedAt)
 if perr != nil {
 if t, perr = time.Parse("2006-01-02", form.ArchivedAt); perr != nil {
 return apperr.FieldError("archived_at", "Format tanggal harus RFC3339 atau YYYY-MM-DD.")
 }
 }
 if t.After(time.Now()) {
 return apperr.FieldError("archived_at", "Tanggal arsip tidak boleh di masa depan.")
 }
 archived = &t
 }

 up, err := service.InspectUpload(fh, h.cfg.MaxFileBytes)
 if err != nil {
 return err
 }
 defer up.File.Close()

 var desc *string
 if form.Description != "" {
 desc = &form.Description
 }
 doc, err := h.svc.Create(c.Request.Context(), service.CreateInput{
 Actor: middleware.CurrentUser(c), Title: form.Title, Description: desc,
 CategoryID: form.CategoryID, Public: form.Visibility == "public",
 ProdiID: form.ProdiID, SubjectUserID: form.SubjectUserID, ArchivedAt: archived, Upload: up,
 })
 if err != nil {
 return err
 }
 msg := "Berkas berhasil diunggah"
 if doc.VerificationStatus != nil && *doc.VerificationStatus == "pending" {
 msg = "Berkas berhasil diunggah dan menunggu verifikasi"
 }
 httpx.OK(c, http.StatusCreated, msg, toDocumentResponse(doc))
 return nil
}

func (h *DocumentHandler) Show(c *gin.Context) error {
 doc, _, err := h.loadViewable(c)
 if err != nil {
 return err
 }
 httpx.OK(c, http.StatusOK, "OK", toDocumentResponse(doc))
 return nil
}

func (h *DocumentHandler) Download(c *gin.Context) error {
 doc, user, err := h.loadViewable(c)
 if err != nil {
 return err
 }

```

```

 }
 rc, size, err := h.svc.Download(c.Request.Context(), user, doc)
 if err != nil {
 return err
 }
 defer rc.Close()
 c.Header("Content-Disposition", mime.FormatMediaType("attachment", map[string]string{"filename": doc.OriginalNam
e}))

 c.Header("X-Content-Type-Options", "nosniff")
 c.Header("Cache-Control", "private, no-store")
 c.DataFromReader(http.StatusOK, size, doc.MimeType, rc, nil)
 return nil
}

func (h *DocumentHandler) Update(c *gin.Context) error {
 id, err := idParam(c)
 if err != nil {
 return err
 }
 var req updateDocumentRequest
 if err := httpx.BindJSON(c, &req); err != nil {
 return err
 }
 doc, err := h.svc.Get(c.Request.Context(), id)
 if err != nil {
 return err
 }
 user := middleware.CurrentUser(c)
 if !service.CanUpdateDocument(user, doc) {
 return apperr.Forbidden("")
 }
 updated, err := h.svc.UpdateMeta(c.Request.Context(), user, doc,
 service.UpdateMetaInput{Title: req.Title, Description: req.Description, CategoryID: req.CategoryID})
 if err != nil {
 return err
 }
 httpx.OK(c, http.StatusOK, "Berkas diperbarui", toDocumentResponse(updated))
 return nil
}

func (h *DocumentHandler) UpdateVisibility(c *gin.Context) error {
 id, err := idParam(c)
 if err != nil {
 return err
 }
 var req visibilityRequest
 if err := httpx.BindJSON(c, &req); err != nil {
 return err
 }
 doc, err := h.svc.Get(c.Request.Context(), id)
 if err != nil {
 return err
 }
 user := middleware.CurrentUser(c)
 if !service.CanChangeVisibility(user, doc) {
 return apperr.Forbidden("")
 }
 updated, err := h.svc.ChangeVisibility(c.Request.Context(), user, id, req.Visibility)
 if err != nil {
 return err
 }
 msg := "Visibilitas berkas diperbarui"
 if updated.VerificationStatus != nil && *updated.VerificationStatus == "pending" {
 msg = "Visibilitas diubah ke publik dan menunggu verifikasi"
 }
 httpx.OK(c, http.StatusOK, msg, toDocumentResponse(updated))
 return nil
}

func (h *DocumentHandler) Destroy(c *gin.Context) error {
 id, err := idParam(c)
 if err != nil {
 return err
 }
 force := false
 if v := c.Query("force"); v != "" {
 if force, err = strconv.ParseBool(v); err != nil {
 return apperr.FieldError("force", "Nilai force harus true atau false.")
 }
 }
 user := middleware.CurrentUser(c)
 isSA := user.Role.Key == domain.RoleSuperAdmin

```

```

doc, err := h.svc.GetUnscoped(c.Request.Context(), id)
if err != nil {
 return err
}
// berkas yang sudah di-soft-delete hanya dapat ditangani Super Admin dengan force=true
if doc.DeletedAt.Valid && !(force && isSA) {
 return apperr.NotFound("")
}
if force {
 if !isSA {
 return apperr.Forbidden("Hapus permanen hanya untuk Super Admin.")
 }
} else if !service.CanDeleteDocument(user, doc) {
 return apperr.Forbidden("")
}
if err := h.svc.Delete(c.Request.Context(), user, doc, force); err != nil {
 return err
}
httpx.NoContent(c)
return nil
}

```

## 5.7.4 VerificationHandler

```

// internal/handler/verification.go
package handler

import (
 "net/http"

 "fite-arsip-api/internal/httpx"
 "fite-arsip-api/internal/middleware"
 "fite-arsip-api/internal/service"

 "github.com/gin-gonic/gin"
)

type VerificationHandler struct{ svc *service.VerificationService }

func NewVerificationHandler(svc *service.VerificationService) *VerificationHandler {
 return &VerificationHandler{svc: svc}
}

type verificationQuery struct {
 Status string `form:"status" binding:"omitempty,oneof=pending approved rejected"`
 Prodi string `form:"prodi" binding:"omitempty,oneof=IF SI TE"`
 DocumentID *uint `form:"document_id" binding:"omitempty,gt=0"`
 Page *int `form:"page" binding:"omitempty,min=1"`
 PerPage *int `form:"per_page" binding:"omitempty,min=1,max=100"`
}

type decideRequest struct {
 Status string `json:"status" binding:"required,oneof=approved rejected"`
 Reason string `json:"reason" binding:"required_if=Status rejected,omitempty,min=5,max=500"`
}

func (h *VerificationHandler) Index(c *gin.Context) error {
 var q verificationQuery
 if err := httpx.BindQuery(c, &q); err != nil {
 return err
 }
 f := service.VerificationFilter{Status: q.Status, Prodi: q.Prodi, DocumentID: q.DocumentID, Page: 1, PerPage: 1}

 if q.Page != nil {
 f.Page = *q.Page
 }
 if q.PerPage != nil {
 f.PerPage = *q.PerPage
 }
 items, total, err := h.svc.List(c.Request.Context(), f)
 if err != nil {
 return err
 }
 out := make([]VerificationResponse, 0, len(items))
 for i := range items {
 out = append(out, toVerificationResponse(&items[i]))
 }
 httpx.Paged(c, "OK", out, httpx.PageMeta(f.Page, f.PerPage, total))
 return nil
}

```

```

}

func (h *VerificationHandler) Show(c *gin.Context) error {
 id, err := idParam(c)
 if err != nil {
 return err
 }
 v, err := h.svc.Get(c.Request.Context(), id)
 if err != nil {
 return err
 }
 httpx.OK(c, http.StatusOK, "OK", toVerificationResponse(v))
 return nil
}

func (h *VerificationHandler) Update(c *gin.Context) error {
 id, err := idParam(c)
 if err != nil {
 return err
 }
 var req decideRequest
 if err := httpx.BindJSON(c, &req); err != nil {
 return err
 }
 v, err := h.svc.Decide(c.Request.Context(), middleware.CurrentUser(c), id, req.Status, req.Reason)
 if err != nil {
 return err
 }
 msg := "Berkas disetujui"
 if req.Status == "rejected" {
 msg = "Berkas ditolak"
 }
 httpx.OK(c, http.StatusOK, msg, toVerificationResponse(v))
 return nil
}
}

```

## 5.8 Route Definition

Rute statis didefinisikan sebelum rute berparameter. Gin  $\geq 1.9$  mendahulukan rute statis (`/documents/public`) atas `/:id`. Handlers berisi semua *handler*; tujuh *handler* tambahan di luar bab ini (kategori, notifikasi, pengguna, dasbor) memakai pola yang sama dan diringkas di 5.10.

```

// internal/router/router.go
package router

import (
 "context"
 "net/http"

 "fite-arsip-api/internal/apperr"
 "fite-arsip-api/internal/config"
 "fite-arsip-api/internal/domain"
 "fite-arsip-api/internal/handler"
 "fite-arsip-api/internal/httpx"
 "fite-arsip-api/internal/middleware"
 "fite-arsip-api/internal/service"

 "github.com/gin-gonic/gin"
)

type Handlers struct {
 Auth *handler.AuthHandler
 Document *handler.DocumentHandler
 Verification *handler.VerificationHandler
 Category *handler.CategoryHandler // pola sama dengan DocumentHandler
 Notification *handler.NotificationHandler // hanya data milik pengguna sendiri
 User *handler.UserHandler // khusus Super Admin
 Dashboard *handler.DashboardHandler // statistik dalam cakupan baca peran
}

func New(cfg config.Config, auth *service.AuthService, h Handlers, ready func(context.Context) error) *gin.Engine {
 r := gin.New()
 r.HandleMethodNotAllowed = true
 r.MaxMultipartMemory = 1 << 20 // bagian multipart > 1 MB ditulis ke berkas sementara, bukan RAM
 _ = r.SetTrustedProxies(cfg.TrustedProxies)

 r.Use(

```

```

middleware.RequestID(),
middleware.Recover(),
middleware.Logger(),
middleware.CORS(cfg.AllowedOrigin),
middleware.SecurityHeaders(cfg.IsProd()),
middleware.LimitBody(cfg.MaxJSONBody, cfg.MaxUploadBody, http.MethodPost, "/api/v1/documents"),
)
r.NoRoute(func(c *gin.Context) { httpx.Fail(c, apperr.NotFound("")) })
r.NoMethod(func(c *gin.Context) {
 httpx.Fail(c, apperr.New(http.StatusMethodNotAllowed, "method_not_allowed", "Metode HTTP tidak diizinkan.))
})

r.GET("/healthz", func(c *gin.Context) { c.JSON(http.StatusOK, gin.H{"status": "ok"}) })
r.GET("/readyz", func(c *gin.Context) {
 if err := ready(c.Request.Context()); err != nil {
 c.JSON(http.StatusServiceUnavailable, gin.H{"status": "not ready"})
 return
 }
 c.JSON(http.StatusOK, gin.H{"status": "ready"})
})

w := httpx.Wrap
api := r.Group("/api/v1")

// ---- publik (dilindungi cookie/pembatas, tanpa Bearer) ----
pub := api.Group("/auth")
pub.POST("/login", w(h.Auth.Login))
pub.POST("/refresh", w(h.Auth.Refresh))
pub.POST("/logout", w(h.Auth.Logout))

// ---- terautentikasi: Bearer + sesi aktif + 60 permintaan/menit/pengguna ----
userLimiter := middleware.NewUserLimiter(60)
authed := api.Group("", middleware.Auth(auth), userLimiter.Middleware())
authed.GET("/auth/me", w(h.Auth.Me)) // boleh walau wajib ganti sandi
authed.PUT("/auth/password", w(h.Auth.ChangePassword))

// ---- semua rute lain: wajib sudah mengganti sandi awal ----
g := authed.Group("", middleware.EnforcePasswordChange())
g.PUT("/auth/profile", w(h.Auth.UpdateProfile))

sa := middleware.RequireRoles(domain.RoleSuperAdmin)
verifier := middleware.RequireRoles(domain.RoleBaak, domain.RoleSuperAdmin)

// 2. Document Management
docs := g.Group("/documents")
docs.GET("", w(h.Document.Index))
docs.POST("", w(h.Document.Store))
docs.GET("/public", w(h.Document.PublicIndex))
docs.GET("/categories", w(h.Category.Index))
docs.POST("/categories", sa, w(h.Category.Store))
docs.PUT("/categories/:id", sa, w(h.Category.Update))
docs.DELETE("/categories/:id", sa, w(h.Category.Destroy))
docs.GET("/:id", w(h.Document.Show))
docs.GET("/:id/download", w(h.Document.Download))
docs.PATCH("/:id", w(h.Document.Update))
docs.PATCH("/:id/visibility", w(h.Document.UpdateVisibility))
docs.DELETE("/:id", w(h.Document.Destroy))

// 3. Verification & Approval
ver := g.Group("/verifications", verifier)
ver.GET("", w(h.Verification.Index))
ver.GET("/:id", w(h.Verification.Show))
ver.PATCH("/:id", w(h.Verification.Update))

// 4. Notifications
n := g.Group("/notifications")
n.GET("", w(h.Notification.Index))
n.GET("/unread-count", w(h.Notification.UnreadCount))
n.PATCH("/read-all", w(h.Notification.ReadAll))
n.PATCH("/:id/read", w(h.Notification.MarkRead))
n.DELETE("/:id", w(h.Notification.Destroy))

// 5. User & Role Management
u := g.Group("/users")
u.GET("/prodi", w(h.User.ProdiIndex)) // semua peran
u.GET("/roles", sa, w(h.User.Roles))
u.PUT("/prodi/:id", sa, w(h.User.ProdiUpdate))
u.GET("", sa, w(h.User.Index))
u.POST("", sa, w(h.User.Store))
u.GET("/:id", sa, w(h.User.Show))
u.PUT("/:id", sa, w(h.User.Update))
u.PATCH("/:id/role", sa, w(h.User.UpdateRole))

```

```
u.PATCH("/:id/status", sa, w(h.User.UpdateStatus))
u.PATCH("/:id/password", sa, w(h.User.ResetPassword))
u.DELETE("/:id", sa, w(h.User.Destroy))
```

SRS Sistem Informasi Arsip Berkas Digital FITE

```
// 6. Dashboard Statistics
```

```
d := g.Group("/dashboard")
```

```
d.GET("/summary", w(h.Dashboard.Summary))
```

```
d.GET("/documents-by-category", w(h.Dashboard.DocumentsByCategory))
```

```
d.GET("/uploads-trend", w(h.Dashboard.UploadsTrend))
```

```
d.GET("/documents-by-prodi", middleware.RequireRoles(domain.RoleSuperAdmin, domain.RoleDekan, domain.RoleBaak),
```

```
w(h.Dashboard.DocumentsByProdi))
```

```
d.GET("/verification-stats", verifier, w(h.Dashboard.VerificationStats))
```

```
d.GET("/storage", sa, w(h.Dashboard.Storage))
```

```
d.GET("/storage-logs", sa, w(h.Dashboard.StorageLogs))
```

```
return r
```

```
}
```

## 5.9 Titik Masuk Aplikasi

```
// internal/app/app.go (dipakai bersama oleh cmd/api dan cmd/worker)
```

```
package app
```

```
import (
```

```
 "time"
```

```
 "fite-arsip-api/internal/config"
```

```
 "gorm.io/driver/mysql"
```

```
 "gorm.io/gorm"
```

```
 "gorm.io/gorm/logger"
```

```
)
```

```
func OpenDB(cfg config.Config) (*gorm.DB, error) {
```

```
 db, err := gorm.Open(mysql.Open(cfg.DBDSN), &gorm.Config{
```

```
 NowFunc: func() time.Time { return time.Now().UTC() },
```

```
 Logger: logger.Default.LogMode(logger.Warn),
```

```
 })
```

```
 if err != nil {
```

```
 return nil, err
```

```
 }
```

```
 sqlDB, err := db.DB()
```

```
 if err != nil {
```

```
 return nil, err
```

```
 }
```

```
 sqlDB.SetMaxOpenConns(25)
```

```
 sqlDB.SetMaxIdleConns(10)
```

```
 sqlDB.SetConnMaxLifetime(30 * time.Minute)
```

```
 return db, nil
```

```
}
```

```
// cmd/api/main.go
```

```
package main
```

```
import (
```

```
 "context"
```

```
 "errors"
```

```
 "log/slog"
```

```
 "net/http"
```

```
 "os"
```

```
 "os/signal"
```

```
 "syscall"
```

```
 "time"
```

```
 "fite-arsip-api/internal/app"
```

```
 "fite-arsip-api/internal/config"
```

```
 "fite-arsip-api/internal/handler"
```

```
 "fite-arsip-api/internal/httpx"
```

```
 "fite-arsip-api/internal/middleware"
```

```
 "fite-arsip-api/internal/router"
```

```
 "fite-arsip-api/internal/service"
```

```
 "fite-arsip-api/internal/storage"
```

```
 "github.com/gin-gonic/gin"
```

```
)
```

```
func main() {
```

```
 if err := run(); err != nil {
```

```

}

func run() error {
 cfg, err := config.Load()
 if err != nil {
 return err
 }
 slog.SetDefault(slog.New(slog.NewJSONHandler(os.Stdout, nil)))
 if cfg.IsProd() {
 gin.SetMode(gin.ReleaseMode)
 }
 httpx.Init()

 ctx, stop := signal.NotifyContext(context.Background(), syscall.SIGINT, syscall.SIGTERM)
 defer stop()

 db, err := app.OpenDB(cfg)
 if err != nil {
 return err
 }
 sqlDB, _ := db.DB()
 store, err := storage.NewGCS(ctx, cfg.GCSBucket, cfg.GCSCredentials)
 if err != nil {
 return err
 }

 authSvc := service.NewAuthService(db, cfg)
 h := router.Handlers{
 Auth: handler.NewAuthHandler(authSvc, cfg, middleware.NewFailureLimiter(5, time.Minute)),
 Document: handler.NewDocumentHandler(service.NewDocumentService(db, store), cfg),
 Verification: handler.NewVerificationHandler(service.NewVerificationService(db)),
 // Category, Notification, User, Dashboard: lihat 5.10
 }
 ready := func(c context.Context) error {
 if err := sqlDB.PingContext(c); err != nil {
 return err
 }
 return store.Ping(c)
 }

 srv := &http.Server{
 Addr: cfg.HTTPAddr,
 Handler: router.New(cfg, authSvc, h, ready),
 ReadHeaderTimeout: 10 * time.Second,
 ReadTimeout: 2 * time.Minute, // unggahan 10 MB pada koneksi lambat
 WriteTimeout: 2 * time.Minute, // unduhan berkas
 IdleTimeout: 2 * time.Minute,
 }
 errCh := make(chan error, 1)
 go func() { errCh <- srv.ListenAndServe() }()
 slog.Info("API berjalan", "addr", cfg.HTTPAddr)

 select {
 case err := <-errCh:
 if !errors.Is(err, http.ErrServerClosed) {
 return err
 }
 case <-ctx.Done():
 shutdown, cancel := context.WithTimeout(context.Background(), 30*time.Second)
 defer cancel()
 return srv.Shutdown(shutdown)
 }
 return nil
}

```

// cmd/worker/main.go

package main

import (

"context"  
"flag"  
"log/slog"  
"os"  
"os/signal"  
"syscall"  
"time"

"fite-arsip-api/internal/app"

```
"github.com/robfig/cron/v3"
- "time/tzdata" // zona waktu tertanam agar Asia/Jakarta tersedia di kontainer minimal
)

func main() {
 once := flag.Bool("once", false, "jalankan satu putaran retensi lalu keluar (untuk pengujian)")
 flag.Parse()
 slog.SetDefault(slog.New(slog.NewJSONHandler(os.Stdout, nil)))

 cfg, err := config.Load()
 if err != nil {
 slog.Error("konfigurasi", "error", err)
 os.Exit(1)
 }
 ctx, stop := signal.NotifyContext(context.Background(), syscall.SIGINT, syscall.SIGTERM)
 defer stop()

 db, err := app.OpenDB(cfg)
 if err != nil {
 slog.Error("database", "error", err)
 os.Exit(1)
 }
 store, err := storage.NewGCS(ctx, cfg.GCSBucket, cfg.GCSCredentials)
 if err != nil {
 slog.Error("gcs", "error", err)
 os.Exit(1)
 }
 svc := service.NewRetentionService(db, store, cfg)

 run := func() bool {
 rep, err := svc.Run(ctx)
 slog.Info("retensi selesai", "skipped", rep.Skipped, "soft", rep.SoftDeleted, "hard", rep.HardDeleted,
 "failed", rep.Failed, "anomalies", rep.Anomalies, "sessions_purged", rep.Sessions)
 if err != nil {
 slog.Error("retensi galat", "error", err)
 }
 return err == nil && rep.Failed == 0
 }

 if *once {
 if !run() {
 os.Exit(1)
 }
 return
 }

 loc, err := time.LoadLocation(cfg.Timezone)
 if err != nil {
 slog.Error("zona waktu", "error", err)
 os.Exit(1)
 }
 c := cron.New(cron.WithLocation(loc),
 cron.WithChain(cron.Recover(cron.DefaultLogger), cron.SkipIfStillRunning(cron.DefaultLogger)))
 if _, err := c.AddFunc("0 2 * * *", func() { run() }); err != nil { // 02:00 WIB setiap hari
 slog.Error("jadwal", "error", err)
 os.Exit(1)
 }
 c.Start()
 slog.Info("worker retensi aktif", "jadwal", "02:00 "+cfg.Timezone)
 <-ctx.Done()
 <-c.Stop().Done()
}
```

## 5.10 Keputusan Implementasi, Pemetaan Uji, dan Hal yang Belum Ditulis

### Pemetaan keputusan ke test case

Keputusan implementasi	Test case
JWT HS256 dikunci (WithValidMethods), iss/aud/exp wajib, sid dicek ke DB pada setiap permintaan	TC-AUTH-08, 09, 10, 11, 29
Peran dan status akun dibaca dari DB (bukan klaim token)	TC-AUTH-30, 31

Keputusan implementasi	Test case
Satu baris <code>auth_sessions</code> per sesi; rotasi memindahkan hash ke <code>previous_refresh_hash</code> ; hash lama dipakai lagi → sesi dicabut dan di- <i>commit</i> meski mengembalikan 401	TC-AUTH-24, 25, 26, 27, 28
Bcrypt cost 12 dengan <code>dummyHash</code> untuk email tak dikenal; <code>max=72</code> byte pada sandi	TC-AUTH-03, 04
<code>FailureLimiter</code> menghitung kegagalan saja; percobaan ke-6 diblokir dengan <code>Retry-After</code>	TC-AUTH-07
<code>Cookie</code> <code>HttpOnly</code> ; <code>Secure</code> ; <code>SameSite=Strict</code> ; <code>Path=/api/v1/auth</code> ; cek <code>Origin</code> pada <code>refresh/logout</code>	TC-AUTH-01, 33, 38
<code>EnforcePasswordChange</code> memisahkan <code>/auth/me</code> dan <code>/auth/password</code> dari rute lain	TC-AUTH-32
<code>LimitBody</code> 11 MB (413), <code>InspectUpload</code> 10 MB (422), <code>Content-Type</code> salah (415)	TC-UPL-03, 04, 28
<i>Content sniffing</i> + struktur ZIP Office + ekstensi ganda; <code>.svg/.html</code> ditolak	TC-UPL-08-13
<code>MaxMultipartMemory</code> 1 MB dan <code>GCS ChunkSize</code> 2 MB (berkas dialirkan)	TC-UPL-29
GCS lebih dulu, lalu transaksi DB; gagal → objek dikompensasi	TC-UPL-23, 24
<code>VisibleTo</code> satu sumber kebenaran untuk daftar dan otorisasi	TC-AUTH-12-19, TC-SRC-12, 13
LIKE dengan karakter <code>% _</code> di- <i>escape</i> ; urutan memakai <i>whitelist</i>	TC-SRC-17, 21, 22
Pointer <code>*int</code> pada <code>page/per_page</code> agar <code>per_page=0</code> menghasilkan 422	TC-SRC-16
Kunci baris dokumen → verifikasi pada <code>Decide</code> dan <code>ChangeVisibility</code>	TC-VER-15
Hard delete: GCS dihapus terakhir dalam transaksi	TC-DEL-20, TC-RET-09
<code>GET_LOCK</code> MySQL pada koneksi khusus; <i>reset-pagination</i> berdasarkan <code>id</code>	TC-RET-07, 08, 15, 16
Cutoff kalender <code>AddDate(-8,0,0)</code> ; 29 Februari ikut benar	TC-RET-02, 14
Dua tahap retensi dengan aksi log berbeda ( <code>expired_*</code> dan <code>deleted_hard_delete</code> )	TC-RET-06, 18
Pembersihan sesi > 30 hari kedaluwarsa/dicabut di <i>worker</i>	TC-RET-17

## Komponen yang tidak dicantumkan kodenya dan aturan yang wajib dipenuhi

`CategoryHandler`, `NotificationHandler`, `UserHandler`, dan `DashboardHandler` terdaftar di router dan memakai pola yang sama (`httpx.Wrap`, `DTO binding`, `service`). Yang wajib mereka patuhi:

- PATCH `/users/{id}/role`:** dalam satu transaksi, perbarui peran, cabut seluruh sesi pengguna (`UPDATE auth_sessions SET revoked_at = now() WHERE user_id = ?`), dan **sinkronkan `documents.owner_role`** untuk berkas dengan `owner_id` pengguna itu. Tanpa ini, cakupan baca Kaprodi dan Dosen memakai peran lama.
- Penonaktifan, reset sandi, dan hapus pengguna:** cabut seluruh sesi pengguna. Penolakan 409 untuk operasi pada diri sendiri atau Super Admin aktif terakhir. Cek jumlah SA aktif dalam transaksi yang sama.
- POST `/users` dan reset sandi:** set `must_change_password = true`; hash bcrypt cost 12; validasi `strongpw`; aturan NIM/NIP/prodi per peran.
- CategoryHandler:** validasi tiap elemen `allowed_roles` terhadap `domain.AllRoles`; hapus kategori yang masih dipakai → 409.
- NotificationHandler:** semua *query* ber-`user_id` = pengguna saat ini; akses ke notifikasi milik orang lain → 403.
- DashboardHandler:** semua statistik berkas melalui `VisibleTo(user)` agar angka tidak bocor; `storage-logs` hanya SA.

1. **Pembatas laju di memori per instans.** Bila API dijalankan lebih dari satu instans, batas login dan 60 permintaan per menit berlaku per instans. Pembatas login berdasarkan email|IP juga tidak menghentikan serangan terdistribusi (banyak IP) terhadap satu email; mitigasinya biasanya pembatas tambahan per email atau CAPTCHA.
2. **Unggah GCS lalu transaksi DB.** Bila proses mati di antara keduanya, tersisa objek yatim di GCS. Ini ditangani oleh skrip rekonsiliasi (SLA-DI-01) yang membandingkan objek GCS dengan documents.storage\_path dan menghapus objek yang lebih tua dari 24 jam tanpa baris metadata. Skrip itu belum ditulis.
3. **Hard delete.** Bila GCS sukses menghapus objek tetapi commit transaksi gagal, baris metadata tetap ada tanpa objek. Operasi dapat diulang karena Delete GCS idempoten.
4. **Pemeriksaan isi berkas bukan pemindai antivirus.** Berkas Office dan ZIP yang sah secara struktur tetap bisa berisi makro atau muatan berbahaya. Bila risiko ini penting bagi fakultas, tambahkan pemindaian (mis. ClamAV) sebelum berkas berstatus Public.
5. **Pengujian.** Kode ini belum dikompilasi atau diuji. Susun uji integrasi go test dengan MySQL nyata (Docker) dan Storage palsu (implementasi storage.Storage di memori) untuk TC-UPL-23/24 dan TC-DEL-20.

Konsekuensi untuk Bab 6 (skema basis data)

1. auth\_sessions (versi yang disederhanakan di atas): id CHAR(36) PK, user\_id FK, refresh\_token\_hash CHAR(64) UNIQUE, previous\_refresh\_hash CHAR(64) NULL INDEX, user\_agent, ip, expires\_at, absolute\_expires\_at, last\_used\_at, revoked\_at, created\_at.
2. documents: owner\_id, owner\_role, retention\_status, deleted\_at, indeks archived\_at, (prodi\_id, owner\_role), (visibility, verification\_status), deleted\_at, dan storage\_path unik.
3. document\_verifications.document\_id UNIQUE, FK ON DELETE CASCADE.
4. notifications.id CHAR(36), data JSON; indeks (user\_id, is\_read). Kueri JSON\_EXTRACT(data, '\$.document\_id') untuk pembersihan antrean akan diganti kolom virtual terindeks bila volumenya membesar.
5. storage\_logs.document\_id tanpa FK; created\_at saja (tanpa updated\_at).
6. Migrasi dengan golang-migrate (berkas .up.sql dan .down.sql), bukan AutoMigrate GORM, agar skema dapat di-review dan di-rollback.

Ketik “lanjut” untuk Bab 6 (Database Design & Architecture): sitemap, relasi tabel, spesifikasi tabel, dan berkas migrasi SQL golang-migrate.

Bab ini menyatukan Lampiran C (ERD) dan Lampiran D (mockup) dengan model GORM di Bab 5, dan menutup dengan berkas migrasi SQL `golang-migrate`. Penyusunannya menemukan empat penyesuaian terhadap bab dan lampiran lain; semuanya dirinci di **6.7**. Dua yang terpenting: (1) dua *query* di Bab 5 akan ditolak oleh *constraint* basis data, dan perbaikannya sudah diterapkan pada kode Bab 5.6; (2) tipe waktu memakai `DATETIME(3)` dalam UTC, bukan `TIMESTAMP`.

Basis data: **MySQL 8.0.16 atau lebih baru** (diperlukan agar `CHECK` benar-benar ditegakkan). Mesin InnoDB, *charset* `utf8mb4`, *collation* `utf8mb4_unicode_ci`.

## 6.1 Sitemap / Hierarchical Information Architecture

### 6.1.1 Struktur Navigasi dan Rute Next.js

FITE Arsip Digital		
Login	/login	(publik)
Ganti Kata Sandi Wajib	/ganti-sandi	(semua; saat <code>must_change_password</code> )
Dashboard	/dashboard	(semua; isi menyesuaikan peran)
Berkas		
File Explorer	/berkas	(berkas sesuai hak baca + filter)
Pencarian & Filter Lanjutan		
Detail Berkas	/berkas/[id]	
Unduh		
Ubah Metadata		
Ubah Visibilitas (Private/Public)		
Hapus (dan Hapus Permanen khusus Super Admin)		
Unggah Berkas (modal)		
Explorer Publik	/publik	(Public + Approved)
Verifikasi	/verifikasi	(BAAK, Super Admin)
Menunggu		
Disetujui		
Ditolak		
Tinjau & Putuskan (modal)		
Notifikasi	/notifikasi	(semua; dropdown lonceng di header)
Administrasi	/admin/...	(Super Admin)
Manajemen Pengguna	/admin/pengguna	
Manajemen Kategori	/admin/kategori	
Manajemen Program Studi	/admin/prodi	
Log & Pemakaian Penyimpanan	/admin/log	
Profil Saya	/profil	(semua)
Ubah Profil		
Ganti Kata Sandi		
Halaman Status		(403, 404, sesi berakhir, server tidak tersedia)

### 6.1.2 Menu yang Tampil per Peran

Tabel ini dirujuk oleh Lampiran D. ✓ tampil; ✕ tidak tampil.

Menu	SA	DKN	BAK	KPR	DSN	MHS
Dashboard	✓	✓	✓	✓	✓	✓
File Explorer	✓	✓	✓	✓	✓	✓
Explorer Publik	✓	✓	✓	✓	✓	✓
Unggah Berkas	✓	✓	✓	✓	✓	✓
Verifikasi	✓	✕	✓	✕	✕	✕
Notifikasi	✓	✓	✓	✓	✓	✓
Manajemen Pengguna	✓	✕	✕	✕	✕	✕
Manajemen Kategori dan Prodi	✓ <sup>93 / 132</sup>	✕	✕	✕	✕	✕

Menu	SA	DKN	BAK	KPR	DSN	MHS
Log dan Pemakaian Penyimpanan	✓	✗	✗	✗	✗	✗
Profil Saya	✓	✓	✓	✓	✓	✓

“Berkas saya” tidak menjadi menu terpisah. Ia adalah filter cepat di File Explorer. Menu yang disembunyikan hanya untuk kenyamanan pengguna; keamanan tetap ditegakkan *middleware* dan *policy* di server (NFR-SEC-03).

### 6.1.3 Struktur Folder Next.js (App Router)

```
app/
├── (auth)/
│ ├── login/page.tsx
│ └── ganti-sandi/page.tsx
├── (app)/
│ ├── layout.tsx # rute yang butuh sesi; memakai layout sidebar + header
│ ├── dashboard/page.tsx # AuthProvider, guard peran, sidebar, header
│ ├── berkas/page.tsx
│ ├── berkas/[id]/page.tsx
│ ├── publik/page.tsx
│ ├── verifikasi/page.tsx
│ ├── notifikasi/page.tsx
│ ├── admin/{pengguna,kategori,prodi,log}/page.tsx
│ └── profil/page.tsx
├── error.tsx not-found.tsx # S-14 Halaman Status
lib/
├── api.ts # klien fetch: Bearer, refresh otomatis saat 401
├── auth.ts # AuthProvider: access token DI MEMORI
└── permissions.ts # peta menu per peran (6.1.2)
```

**Konsekuensi arsitektur yang perlu diketahui tim frontend:** *access token* disimpan **di memori** (bukan *localStorage* dan bukan *cookie* yang terbaca server Next.js). Akibatnya, server Next.js tidak memegang token sehingga **pengambilan data dilakukan dari sisi klien** (komponen "use client" dengan pustaka seperti TanStack Query), bukan di *server component*. *Middleware* Next.js (*middleware.ts*) tidak dapat memverifikasi sesi, jadi guard rute dilakukan di *(app)/layout.tsx* setelah *refresh* awal berhasil. Pilihan ini sengaja dibuat demi keamanan token (NFR-SEC-04); halaman tetap dapat dirender di server untuk kerangkanya.

### 6.1.4 Hierarki Penyimpanan Objek di GCS

```
gs://fite-arsip-private/
├── documents/
│ ├── IF/2026/10/{uuid}.pdf
│ ├── SI/2026/10/{uuid}.docx
│ ├── TE/2026/10/{uuid}.zip
│ └── fakultas/2026/10/{uuid}.pdf (berkas tanpa prodi, mis. milik Dekan)
```

Nama objek adalah UUID. Nama asli berkas disimpan di *documents.original\_name* dan dipakai saat unduh.

## 6.2 Table Relation

Diagram lengkap dan cara menggambarinya ada di **Lampiran C**. Bagian ini adalah ringkasan normatif yang menjadi acuan skema. Bila ada selisih dengan Lampiran C, **bab ini yang berlaku**.

### 6.2.1 Relasi 1-to-N (dengan aturan FK)

No	Induk (1)	Anak (N)	Kolom FK	ON DELETE	Partisipasi
R1	roles	users	users.role_id	RESTRICT	Pengguna wajib punya satu peran
R2	prodi	users	users.prodi_id (nullable)	RESTRICT	SA, Dekan, BAAK tanpa prodi
R3	prodi	documents	documents.prodi_id (nullable)	RESTRICT	Berkas fakultas tanpa prodi

No	Induk (1)	Anak (N)	Kolom FK	ON DELETE	Partisipasi
R4	categories	documents	documents.category_id	RESTRICT	Kategori terpakai tidak dapat dihapus (API 409)
R5	users	documents	documents.uploader_id	RESTRICT	Pengunggah
R6	users	documents	documents.owner_id	RESTRICT	Pemilik/subjek berkas
R7	users	auth_sessions	auth_sessions.user_id	CASCADE	Sesi tidak bermakna tanpa pengguna
R9	users	document_verifications	requested_by	RESTRICT	Pengaju
R10	users	document_verifications	verified_by (nullable)	SET NULL	Kosong selama <i>pending</i>
R11	users	notifications	notifications.user_id	CASCADE	Penerima
R12	users	storage_logs	storage_logs.user_id (nullable)	SET NULL	Kosong untuk tindakan sistem

### 6.2.2 Relasi 1-to-1 (opsional di sisi anak)

Induk	Anak	Kolom FK	ON DELETE	Keterangan
documents	document_verifications (R8)	document_id ( <b>UNIQUE</b> )	CASCADE	Paling banyak satu baris verifikasi per berkas; berkas Private yang tidak pernah diajukan tidak punya baris

### 6.2.3 Relasi N-to-M

No	Entitas	Realisasi	Keterangan
R13	users ⇌ documents (aktivitas)	Tabel storage_logs sebagai tabel asosiasi; document_id <b>tanpa FK</b>	Satu pengguna melakukan banyak tindakan pada banyak berkas; log bertahan setelah berkas dihapus permanen
R14	categories ⇌ roles	Kolom JSON categories.allowed_roles (daftar roles.key), <b>tanpa FK</b>	Satu kategori diunggah banyak peran dan satu peran mengunggah banyak kategori. Integritas dijaga validasi aplikasi dan CHECK JSON_TYPE = 'ARRAY' . Dinormalisasi menjadi tabel pivot bila kelak perlu pelaporan SQL murni

### 6.2.4 Tabel Framework dan Pustaka

Skema ini **tidak memakai tabel framework** seperti personal\_access\_tokens atau cache (itu milik Laravel). Sesi diwakili tabel auth\_sessions. Tabel schema\_migrations dibuat otomatis oleh golang-migrate dan tidak digambar di ERD.

## 6.3 Table Specifications

**Kunci:** PK kunci primer; FK kunci asing; UK unik; IDX indeks biasa. **NN** = *NOT NULL*. Semua waktu bertipe DATETIME(3) dan **disimpan dalam UTC** (lihat 6.7).

6.3.1 roles

Column	Data Type	Keys	Keterangan
id	BIGINT UNSIGNED NN, AUTO_INCREMENT	PK	
key	VARCHAR(30) NN	UK	super_admin, dekan, baak, kaprodi, dosen, mahasiswa
name	VARCHAR(50) NN		Label tampil
created_at, updated_at	DATETIME(3) NN		

6.3.2 prodi

Column	Data Type	Keys	Keterangan
id	BIGINT UNSIGNED NN, AUTO_INCREMENT	PK	
code	VARCHAR(5) NN	UK	IF, SI, TE
name	VARCHAR(100) NN		
is_active	TINYINT(1) NN, default 1		Prodi nonaktif tidak dapat dipilih saat unggah
created_at, updated_at	DATETIME(3) NN		

6.3.3 users

Column	Data Type	Keys	Keterangan
id	BIGINT UNSIGNED NN, AUTO_INCREMENT	PK	
name	VARCHAR(100) NN		
email	VARCHAR(255) NN	UK	Disimpan huruf kecil oleh aplikasi; <i>collation</i> tidak peka huruf menjaga keunikan
password	VARCHAR(255) NN		Hash bcrypt cost 12
role_id	BIGINT UNSIGNED NN	FK → roles.id; IDX (role_id, is_active)	
prodi_id	BIGINT UNSIGNED NULL	FK → prodi.id	Wajib bagi mahasiswa, dosen, kaprodi (ditegakkan aplikasi)
nim	VARCHAR(30) NULL	UK	Mahasiswa
nip	VARCHAR(30) NULL	UK	Dosen, Kaprodi, BAAK
phone	VARCHAR(20) NULL		
is_active	TINYINT(1) NN, default 1		
must_change_password	TINYINT(1) NN, default 0		
created_at, updated_at	DATETIME(3) NN		
deleted_at	DATETIME(3) NULL	IDX	Soft delete

Email, NIM, dan NIP pengguna yang di-*soft delete* tetap mengunci nilainya (indeks unik tidak melihat deleted\_at ). Ini disengaja demi jejak audit.

6.3.4 auth\_sessions

Satu baris per sesi login. Rotasi *refresh token* memindahkan *hash* lama ke previous\_refresh\_hash (dipakai mendeteksi pemakaian ulang).

Column	Data Type	Keys	Keterangan
id	CHAR(36) NN (ascii)	PK	UUID; nilai klaim <code>sid</code> pada JWT
user_id	BIGINT UNSIGNED NN	FK → <code>users.id</code> (CASCADE); IDX ( <code>user_id</code> , <code>revoked_at</code> )	
refresh_token_hash	CHAR(64) NN (ascii)	UK	SHA-256 heksadesimal dari <i>refresh token</i> aktif
previous_refresh_hash	CHAR(64) NULL (ascii)	IDX	<i>Hash</i> token sebelum rotasi terakhir
user_agent	VARCHAR(255) NN, default ''		
ip	VARCHAR(45) NN, default ''		Cukup untuk IPv6
expires_at	DATETIME(3) NN	IDX	Batas <i>sliding</i> (7 hari sejak pemakaian terakhir)
absolute_expires_at	DATETIME(3) NN		Batas mutlak (30 hari sejak login)
last_used_at	DATETIME(3) NULL		
revoked_at	DATETIME(3) NULL		Terisi saat logout, pencabutan, atau pemakaian ulang token
created_at	DATETIME(3) NN		

6.3.5 categories

Column	Data Type	Keys	Keterangan
id	BIGINT UNSIGNED NN, AUTO_INCREMENT	PK	
name	VARCHAR(100) NN	UK	
allowed_roles	JSON NN	CHECK <code>JSON_TYPE = 'ARRAY'</code>	Daftar <code>roles.key</code> yang boleh mengunggah; selalu memuat <code>super_admin</code>
is_operational	TINYINT(1) NN, default 0		1: BAAK berhak menghapus berkas kategori ini
created_at, updated_at	DATETIME(3) NN		

6.3.6 documents

Column	Data Type	Keys	Keterangan
id	BIGINT UNSIGNED NN, AUTO_INCREMENT	PK	
title	VARCHAR(255) NN		Dicari oleh <code>q</code>
description	TEXT NULL		Maksimal 2000 karakter (validasi aplikasi)
original_name	VARCHAR(255) NN		Sudah disanitasi
extension	VARCHAR(10) NN	IDX; CHECK whitelist	<code>pdf</code> , <code>docx</code> , <code>xlsx</code> , <code>pptx</code> , <code>zip</code> , <code>png</code> , <code>jpg</code>
mime_type	VARCHAR(100) NN		
size_kb	INT UNSIGNED NN	CHECK 1..10240	
storage_path	VARCHAR(500) NN ( <code>utf8mb4_bin</code> )	UK	Lokasi objek di GCS; tidak diekspos ke klien
category_id	BIGINT UNSIGNED NN	FK → <code>categories.id</code>	
prodi_id	BIGINT UNSIGNED NULL	FK → <code>prodi.id</code>	NULL = berkas fakultas
uploader_id	BIGINT UNSIGNED NN	FK → <code>users.id</code>	

Column	Data Type	Keys	Keterangan
owner_id	BIGINT UNSIGNED NN	FK → users.id	Dasar filter NIM/NIP dan cakupan baca
owner_role	VARCHAR(30) NN	IDX (owner_role,prodi_id); CHECK	Salinan roles.key pemilik (denormalisasi)
visibility	ENUM('private','public') NN, default 'private'	IDX (visibility,verification_status,created_at)	
verification_status	ENUM('pending','approved','rejected') NULL	CHECK konsistensi	Lihat invarian di bawah
archived_at	DATETIME(3) NULL	IDX	Acuan retensi 8 tahun
retention_status	ENUM('expired') NULL		expired bila disaring penjadwal
created_at, updated_at	DATETIME(3) NN	IDX (created_at)	
deleted_at	DATETIME(3) NULL	IDX	Soft delete; acuan masa tenggang 90 hari

Invarian yang ditegakkan oleh CHECK (chk\_documents\_visibility):

Kondisi	verification_status yang sah
visibility = 'private'	NULL atau rejected
visibility = 'public'	pending atau approved (tidak boleh NULL)

### 6.3.7 document\_verifications

Column	Data Type	Keys	Keterangan
id	BIGINT UNSIGNED NN, AUTO_INCREMENT	PK	
document_id	BIGINT UNSIGNED NN	UK, FK → documents.id (CASCADE)	Satu baris per berkas
status	ENUM('pending','approved','rejected') NN, default 'pending'	IDX (status,requested_at)	
reason	VARCHAR(500) NULL	CHECK	Wajib bila rejected
requested_by	BIGINT UNSIGNED NN	FK → users.id	
verified_by	BIGINT UNSIGNED NULL	FK → users.id (SET NULL)	
requested_at	DATETIME(3) NN		
verified_at	DATETIME(3) NULL		
created_at, updated_at	DATETIME(3) NN		

### 6.3.8 notifications

Column	Data Type	Keys	Keterangan
id	CHAR(36) NN (ascii)	PK	UUID
user_id	BIGINT UNSIGNED NN	FK → users.id (CASCADE); IDX (user_id,is_read), IDX (user_id,created_at)	Penerima
type	VARCHAR(50) NN	IDX	verification_pending, verification_approved, verification_rejected
title	VARCHAR(150) NN		
message	TEXT NN		
data	JSON NULL		document_id, verification_id

Column	Data Type	Keys	Keterangan
is_read	TINYINT(1) NN, default 0		
created_at, updated_at	DATETIME(3) NN		

### 6.3.9 storage\_logs

Column	Data Type	Keys	Keterangan
id	BIGINT UNSIGNED NN, AUTO_INCREMENT	PK	
document_id	BIGINT UNSIGNED NULL	IDX ( <b>tanpa FK</b> )	Bertahan setelah <i>hard delete</i>
user_id	BIGINT UNSIGNED NULL	FK → users.id (SET NULL); IDX	NULL = tindakan sistem
action	VARCHAR(40) NN	IDX (action, created_at)	Katalog di 6.5
meta	JSON NULL		Ukuran, visibilitas, alasan, judul
created_at	DATETIME(3) NN	IDX	<i>Append-only</i> ; tidak ada updated_at

## 6.4 Migrasi SQL (golang-migrate)

**Prinsip:** satu tabel per migrasi, karena DDL di MySQL tidak transaksional. Bila satu migrasi gagal di tengah, hanya satu tabel yang terdampak.

### 6.4.1 Persiapan dan Cara Menjalankan

```
-- dijalankan sekali oleh administrator DB
CREATE DATABASE fite_arsip CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

-- akun migrasi (boleh DDL) dan akun aplikasi (tanpa DDL)
CREATE USER 'fite_migrate'@ '%' IDENTIFIED BY '<rahasia-1>';
GRANT ALL PRIVILEGES ON fite_arsip.* TO 'fite_migrate'@ '%';

CREATE USER 'fite_app'@ '%' IDENTIFIED BY '<rahasia-2>';
GRANT SELECT, INSERT, UPDATE, DELETE ON fite_arsip.* TO 'fite_app'@ '%';
```

```
menjalankan migrasi naik (multiStatements=true wajib untuk driver MySQL)
migrate -path migrations \
 -database "mysql://fite_migrate:<rahasia-1>@tcp(127.0.0.1:3306)/fite_arsip?multiStatements=true" up

melihat versi, turun satu langkah
migrate -path migrations -database "..." version
migrate -path migrations -database "..." down 1
```

**Pembaruan DB\_DSN aplikasi (wajib):** tambahkan time\_zone agar nilai bawaan CURRENT\_TIMESTAMP(3) berisi UTC:

```
DB_DSN=fite_app:rahasia@tcp(127.0.0.1:3306)/fite_arsip?charset=utf8mb4&parseTime=true&loc=UTC&time_zone=%27%2B00%3A00%27
```

### 6.4.2 Berkas Migrasi

```
-- migrations/000001_create_roles.up.sql
CREATE TABLE roles (
 id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
 `key` VARCHAR(30) NOT NULL,
 name VARCHAR(50) NOT NULL,
 created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
 updated_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
 PRIMARY KEY (id),
 UNIQUE KEY uq_roles_key (`key`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
-- migrations/000002_create_prodi.up.sql
CREATE TABLE prodi (
 id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
 code VARCHAR(5) NOT NULL,
 name VARCHAR(100) NOT NULL,
 is_active TINYINT(1) NOT NULL DEFAULT 1,
 created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
 updated_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
 PRIMARY KEY (id),
 UNIQUE KEY uq_prodi_code (code)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
-- migrations/000003_create_users.up.sql
CREATE TABLE users (
 id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
 name VARCHAR(100) NOT NULL,
 email VARCHAR(255) NOT NULL,
 password VARCHAR(255) NOT NULL,
 role_id BIGINT UNSIGNED NOT NULL,
 prodi_id BIGINT UNSIGNED NULL,
 nim VARCHAR(30) NULL,
 nip VARCHAR(30) NULL,
 phone VARCHAR(20) NULL,
 is_active TINYINT(1) NOT NULL DEFAULT 1,
 must_change_password TINYINT(1) NOT NULL DEFAULT 0,
 created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
 updated_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
 deleted_at DATETIME(3) NULL,
 PRIMARY KEY (id),
 UNIQUE KEY uq_users_email (email),
 UNIQUE KEY uq_users_nim (nim),
 UNIQUE KEY uq_users_nip (nip),
 KEY idx_users_role_active (role_id, is_active),
 KEY idx_users_prodi (prodi_id),
 KEY idx_users_deleted_at (deleted_at),
 CONSTRAINT fk_users_role FOREIGN KEY (role_id) REFERENCES roles (id) ON DELETE RESTRICT ON UPDATE RESTRICT,
 CONSTRAINT fk_users_prodi FOREIGN KEY (prodi_id) REFERENCES prodi (id) ON DELETE RESTRICT ON UPDATE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
-- migrations/000004_create_auth_sessions.up.sql
CREATE TABLE auth_sessions (
 id CHAR(36) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
 user_id BIGINT UNSIGNED NOT NULL,
 refresh_token_hash CHAR(64) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
 previous_refresh_hash CHAR(64) CHARACTER SET ascii COLLATE ascii_bin NULL,
 user_agent VARCHAR(255) NOT NULL DEFAULT '',
 ip VARCHAR(45) NOT NULL DEFAULT '',
 expires_at DATETIME(3) NOT NULL,
 absolute_expires_at DATETIME(3) NOT NULL,
 last_used_at DATETIME(3) NULL,
 revoked_at DATETIME(3) NULL,
 created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
 PRIMARY KEY (id),
 UNIQUE KEY uq_sessions_refresh (refresh_token_hash),
 KEY idx_sessions_prev_refresh (previous_refresh_hash),
 KEY idx_sessions_user_revoked (user_id, revoked_at),
 KEY idx_sessions_expires (expires_at),
 CONSTRAINT fk_sessions_user FOREIGN KEY (user_id) REFERENCES users (id) ON DELETE CASCADE ON UPDATE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
-- migrations/000005_create_categories.up.sql
CREATE TABLE categories (
 id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
 name VARCHAR(100) NOT NULL,
 allowed_roles JSON NOT NULL,
 is_operational TINYINT(1) NOT NULL DEFAULT 0,
 created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
 updated_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
 PRIMARY KEY (id),
 UNIQUE KEY uq_categories_name (name),
 CONSTRAINT chk_categories_roles_array CHECK (JSON_TYPE(allowed_roles) = 'ARRAY')
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
-- migrations/000006_create_documents.up.sql
CREATE TABLE documents (
 id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
 title VARCHAR(255) NOT NULL,
```

```

description TEXT NULL,
original_name VARCHAR(255) NOT NULL,
extension VARCHAR(10) NOT NULL,
mime_type VARCHAR(100) NOT NULL,
size_kb INT UNSIGNED NOT NULL,
storage_path VARCHAR(500) CHARACTER SET utf8mb4 COLLATE utf8mb4_bin NOT NULL,
category_id BIGINT UNSIGNED NOT NULL,
prodi_id BIGINT UNSIGNED NULL,
uploader_id BIGINT UNSIGNED NOT NULL,
owner_id BIGINT UNSIGNED NOT NULL,
owner_role VARCHAR(30) NOT NULL,
visibility ENUM('private','public') NOT NULL DEFAULT 'private',
verification_status ENUM('pending','approved','rejected') NULL,
archived_at DATETIME(3) NULL,
retention_status ENUM('expired') NULL,
created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
updated_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
deleted_at DATETIME(3) NULL,
PRIMARY KEY (id),
UNIQUE KEY uq_documents_storage_path (storage_path),
KEY idx_documents_category (category_id),
KEY idx_documents_prodi (prodi_id),
KEY idx_documents_uploader (uploader_id),
KEY idx_documents_owner (owner_id),
KEY idx_documents_owner_role_prodi (owner_role, prodi_id),
KEY idx_documents_public (visibility, verification_status, created_at),
KEY idx_documents_archived_at (archived_at),
KEY idx_documents_deleted_at (deleted_at),
KEY idx_documents_created_at (created_at),
KEY idx_documents_extension (extension),
CONSTRAINT fk_documents_category FOREIGN KEY (category_id) REFERENCES categories (id) ON DELETE RESTRICT ON UPDATE
RESTRIC,
CONSTRAINT fk_documents_prodi FOREIGN KEY (prodi_id) REFERENCES prodi (id) ON DELETE RESTRICT ON UPDATE
RESTRIC,
CONSTRAINT fk_documents_uploader FOREIGN KEY (uploader_id) REFERENCES users (id) ON DELETE RESTRICT ON UPDATE
RESTRIC,
CONSTRAINT fk_documents_owner FOREIGN KEY (owner_id) REFERENCES users (id) ON DELETE RESTRICT ON UPDATE
RESTRIC,
CONSTRAINT chk_documents_visibility CHECK (
 (visibility = 'private' AND (verification_status IS NULL OR verification_status = 'rejected'))
 OR
 (visibility = 'public' AND verification_status IS NOT NULL AND verification_status IN ('pending','approved'))
),
CONSTRAINT chk_documents_size CHECK (size_kb BETWEEN 1 AND 10240),
CONSTRAINT chk_documents_extension CHECK (extension IN ('pdf','docx','xlsx','pptx','zip','png','jpg')),
CONSTRAINT chk_documents_owner_role CHECK (owner_role IN ('super_admin','dekan','baak','kaprodi','dosen','mahasisw
a'))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

```

-- migrations/000007_create_document_verifications.up.sql
CREATE TABLE document_verifications (
 id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
 document_id BIGINT UNSIGNED NOT NULL,
 status ENUM('pending','approved','rejected') NOT NULL DEFAULT 'pending',
 reason VARCHAR(500) NULL,
 requested_by BIGINT UNSIGNED NOT NULL,
 verified_by BIGINT UNSIGNED NULL,
 requested_at DATETIME(3) NOT NULL,
 verified_at DATETIME(3) NULL,
 created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
 updated_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
 PRIMARY KEY (id),
 UNIQUE KEY uq_verifications_document (document_id),
 KEY idx_verifications_status (status, requested_at),
 KEY idx_verifications_requested_by (requested_by),
 KEY idx_verifications_verified_by (verified_by),
 CONSTRAINT fk_verifications_document FOREIGN KEY (document_id) REFERENCES documents (id) ON DELETE CASCADE ON U
PDATE RESTRICT,
 CONSTRAINT fk_verifications_requester FOREIGN KEY (requested_by) REFERENCES users (id) ON DELETE RESTRICT ON U
PDATE RESTRICT,
 CONSTRAINT fk_verifications_verifier FOREIGN KEY (verified_by) REFERENCES users (id) ON DELETE SET NULL ON U
PDATE RESTRICT,
 CONSTRAINT chk_verifications_rejected_reason CHECK (status <> 'rejected' OR reason IS NOT NULL)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

```

-- migrations/000008_create_notifications.up.sql
CREATE TABLE notifications (
 id CHAR(36) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
 user_id BIGINT UNSIGNED NOT NULL,
 type VARCHAR(50) NOT NULL,

```

```
title VARCHAR(150) NOT NULL,
message TEXT NOT NULL,
data JSON NULL,
is_read TINYINT(1) NOT NULL DEFAULT 0,
created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
updated_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
PRIMARY KEY (id),
KEY idx_notifications_user_read (user_id, is_read),
KEY idx_notifications_user_created (user_id, created_at),
KEY idx_notifications_type (type),
CONSTRAINT fk_notifications_user FOREIGN KEY (user_id) REFERENCES users (id) ON DELETE CASCADE ON UPDATE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
-- migrations/000009_create_storage_logs.up.sql
CREATE TABLE storage_logs (
 id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
 document_id BIGINT UNSIGNED NULL, -- sengaja tanpa FK: log bertahan setelah hard delete
 user_id BIGINT UNSIGNED NULL,
 action VARCHAR(40) NOT NULL,
 meta JSON NULL,
 created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
 PRIMARY KEY (id),
 KEY idx_logs_document (document_id),
 KEY idx_logs_user (user_id),
 KEY idx_logs_action_created (action, created_at),
 KEY idx_logs_created (created_at),
 CONSTRAINT fk_logs_user FOREIGN KEY (user_id) REFERENCES users (id) ON DELETE SET NULL ON UPDATE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
-- migrations/000010_seed_reference_data.up.sql
INSERT INTO roles (`key`, name) VALUES
('super_admin', 'Super Admin'),
('dekan', 'Dekan'),
('baak', 'BAAK / Staf'),
('kaprodi', 'Kaprodi'),
('dosen', 'Dosen / TA'),
('mahasiswa', 'Mahasiswa');

INSERT INTO prodi (code, name) VALUES
('IF', 'S1 Informatika'),
('SI', 'S1 Sistem Informasi'),
('TE', 'S1 Teknik Elektro');

INSERT INTO categories (name, allowed_roles, is_operational) VALUES
('SK Keputusan Dekan', JSON_ARRAY('dekan','super_admin'), 0),
('Dokumen Akreditasi Fakultas', JSON_ARRAY('dekan','super_admin'), 0),
('Arsip Khusus', JSON_ARRAY('dekan','super_admin'), 0),
('Legalisir', JSON_ARRAY('baak','super_admin'), 1),
('Transkrip Nilai Akademik', JSON_ARRAY('baak','super_admin'), 1),
('Dokumen BAAK', JSON_ARRAY('baak','super_admin'), 1),
('Berkas Mahasiswa/Dosen', JSON_ARRAY('baak','super_admin'), 1),
('SK Keputusan Prodi', JSON_ARRAY('kaprodi','super_admin'), 0),
('Dokumen Akreditasi Prodi', JSON_ARRAY('kaprodi','super_admin'), 0),
('Kurikulum', JSON_ARRAY('kaprodi','super_admin'), 0),
('Rencana Pembelajaran Semester (RPS)', JSON_ARRAY('dosen','super_admin'), 0),
('SK Mengajar', JSON_ARRAY('dosen','super_admin'), 0),
('Berita Acara Perkuliahan (BAP)', JSON_ARRAY('dosen','super_admin'), 0),
('Proposal TA/PA', JSON_ARRAY('mahasiswa','super_admin'), 0),
('Berkas Yudisium', JSON_ARRAY('mahasiswa','super_admin'), 0),
('Surat Keterangan', JSON_ARRAY('mahasiswa','super_admin'), 0);
```

**Berkas** `.down.sql` (setiap pasangannya hanya satu pernyataan):

Berkas	Isi
000010_seed_reference_data.down.sql	DELETE FROM categories; DELETE FROM prodi; DELETE FROM roles; (gagal bila sudah dipakai, sesuai FK RESTRICT)
000009_create_storage_logs.down.sql	DROP TABLE IF EXISTS storage_logs;
000008_create_notifications.down.sql	DROP TABLE IF EXISTS notifications;
000007_create_document_verifications.down.sql	DROP TABLE IF EXISTS document_verifications;
000006_create_documents.down.sql	DROP TABLE IF EXISTS documents;
000005_create_categories.down.sql	DROP TABLE IF EXISTS categories;
000004_create_auth_sessions.down.sql	DROP TABLE IF EXISTS auth_sessions;

Berkas	Isi
000003_create_users.down.sql	DROP TABLE IF EXISTS users;
000002_create_prodi.down.sql	DROP TABLE IF EXISTS prodi;
000001_create_roles.down.sql	DROP TABLE IF EXISTS roles;

**Akun Super Admin** (4 akun tim) **tidak** dimasukkan ke migrasi karena menyangkut kata sandi. Akun ini dibuat oleh `cmd/seed` yang membaca email dan kata sandi awal dari variabel lingkungan, memakai `bcrypt` cost 12, dan mengisi `must_change_password = 1`.

**Bila migrasi gagal di tengah:** `golang-migrate` menandai versi sebagai *dirty* dan menolak berjalan. Perbaiki penyebabnya (hapus tabel yang setengah jadi bila ada), lalu `migrate ... force <versi-terakhir-yang-bersih>`, kemudian `up` lagi.

## 6.5 Kamus Nilai

**documents.verification\_status dan document\_verifications.status:**

Nilai	Arti
pending	Menunggu keputusan BAAK/SA
approved	Disetujui; berkas tampil di explorer publik
rejected	Ditolak; berkas tetap Private; alasan wajib

**notifications.type:** `verification_pending` (ke seluruh BAAK dan SA aktif), `verification_approved` dan `verification_rejected` (ke pengunggah).

**storage\_logs.action** (nilai ini dipakai oleh Bab 5 dan filter `GET /dashboard/storage-logs`):

Nilai	Pemicu
upload, download, update, visibility_change	Operasi berkas oleh pengguna
verify_approved, verify_rejected	Keputusan verifikator
soft_delete, hard_delete	Penghapusan oleh pengguna
expired_soft_delete, expired_hard_delete	Retensi 8 tahun
deleted_hard_delete	Pembersihan permanen berkas yang dihapus pengguna setelah 90 hari

## 6.6 Pemetaan Query ke Indeks

Query (sumber)	Indeks yang dipakai
Daftar berkas per peran (VisibleTo)	<code>idx_documents_uploader</code> , <code>idx_documents_owner</code> , <code>idx_documents_owner_role_prodi</code>
Explorer publik ( <code>visibility='public' AND verification_status='approved'</code> ,urut terbaru)	<code>idx_documents_public</code>
Filter prodi dan kategori dan ekstensi	<code>idx_documents_prodi</code> , <code>idx_documents_category</code> , <code>idx_documents_extension</code>
Filter NIM/NIP (subquery ke <code>users</code> )	<code>uq_users_nim</code> , <code>uq_users_nip</code>
Urutan terbaru	<code>idx_documents_created_at</code>
Retensi tahap 1 ( <code>archived_at &lt; cutoff</code> )	<code>idx_documents_archived_at</code>
Retensi tahap 2 ( <code>deleted_at &lt;= grace</code> )	<code>idx_documents_deleted_at</code>
Antrean verifikasi ( <code>status='pending'</code> urut <code>requested_at</code> )	<code>idx_verifications_status</code>
Cegah antrean ganda; cari verifikasi per berkas	<code>uq_verifications_document</code>

Query (sumber)	Indeks yang dipakai
Notifikasi pengguna (daftar, jumlah belum dibaca)	idx_notifications_user_created, idx_notifications_user_read
Pembersihan notifikasi antrean (type='verification_pending')	idx_notifications_type
Autentikasi: sesi aktif per id dan user_id	PK auth_sessions.id
Rotasi dan deteksi pemakaian ulang token	uq_sessions_refresh, idx_sessions_prev_refresh
Pembersihan sesi lama	idx_sessions_expires
Log penyimpanan (filter aksi, berkas, pelaku, tanggal)	idx_logs_action_created, idx_logs_document, idx_logs_user, idx_logs_created

**Pencarian kata kunci (q):** memakai LIKE '%kata%' yang tidak dapat memakai indeks. Pada target 100.000 baris (SLA-RT-02, TC-SRC-20) biasanya masih di bawah 1 detik karena dipersempit filter berindeks, tetapi **ini harus dibuktikan** dengan EXPLAIN dan uji beban. Bila gagal, tambahkan migrasi FULLTEXT ... WITH PARSER ngram pada title dan original\_name, lalu ubah query applyFilter di Bab 5 menjadi MATCH ... AGAINST.

**Pembersihan notifikasi antrean** memakai JSON\_EXTRACT(data, '\$.document\_id') tanpa indeks. Dibatasi oleh idx\_notifications\_type, dan volume notifikasi pending kecil. Bila membesar, tambahkan kolom virtual terindeks ref\_document\_id.

## 6.7 Penyesuaian yang Wajib pada Bab dan Lampiran Lain

### 1. Dua query di Bab 5 melanggar chk\_documents\_visibility (sudah diterapkan di Bab 5)

Saat berkas **Public/Approved** dihapus, kode Bab 5 hanya mengosongkan verification\_status, sehingga baris menjadi visibility = 'public' dengan status NULL. Kondisi itu ditolak oleh CHECK. Perbaikannya adalah ikut mengembalikan visibility ke private. Perbaikan ini **sudah tercantum pada kode Bab 5.6.4 dan 5.6.6;** potongan di bawah dicatat sebagai riwayat perubahan.

internal/service/document.go, di dalam Delete (cabang soft delete), ganti:

```
if err := tx.Model(&domain.Document{}).Where("id = ?", d.ID).Update("verification_status", nil).Error; err != nil {
 return err
}
```

menjadi:

```
if err := tx.Model(&domain.Document{}).Where("id = ?", d.ID).
 Updates(map[string]any{"verification_status": nil, "visibility": "private"}).Error; err != nil {
 return err
}
```

internal/service/retention.go, di dalam expireSoft, ganti:

```
Updates(map[string]any{"retention_status": "expired", "verification_status": nil}).Error
```

menjadi:

```
Updates(map[string]any{"retention_status": "expired", "verification_status": nil, "visibility": "private"}).Error
```

Jalur lain di Bab 5 (unggah, ChangeVisibility, Decide) sudah mengubah visibility dan verification\_status dalam satu pernyataan UPDATE, sehingga lolos CHECK. CHECK ini layak dipertahankan: ia menangkap bug seperti di atas pada uji integrasi, bukan setelah data kotor masuk.

### 2. Tipe waktu: DATETIME(3) dalam UTC, bukan TIMESTAMP

TIMESTAMP MySQL berhenti di 19 Januari 2038 dan dikonversi menurut zona waktu sesi. Untuk sistem arsip yang menyimpan data bertahun-tahun, dua sifat ini berisiko. **Lampiran C** (tabel atribut C.2) sudah memakai

3. Urutan indeks documents

Lampiran C dan draf sebelumnya menyebut indeks (prodi\_id, owner\_role). Bab ini memakai (owner\_role, prodi\_id) karena query VisibleTo untuk Kaprodi mencari owner\_role = 'dosen' tanpa prodi. Indeks terpisah idx\_documents\_prodi melayani filter prodi saja.

4. Lampiran C: tambahan auth\_sessions

Tabel auth\_sessions memakai satu baris per sesi dengan kolom previous\_refresh\_hash, bukan rantai family\_id/replaced\_by seperti rencana awal di Bab 4. Atribut di Lampiran C.2 sudah sesuai. ON DELETE untuk R7 adalah CASCADE.

5. Batasan yang perlu diketahui

1. Peran dan prodi tidak ditegakkan lintas tabel oleh CHECK (MySQL tidak mengizinkan CHECK menunjuk tabel lain). Aturan seperti “mahasiswa wajib punya NIM dan prodi” ditegakkan oleh aplikasi dan diuji, bukan oleh basis data.
2. documents.owner\_role harus disinkronkan setiap kali peran pengguna berubah. CHECK hanya menjamin nilainya salah satu dari enam peran, bukan bahwa nilainya sama dengan peran pemilik saat ini. Sinkronisasi ada pada layanan manajemen pengguna (aturan di Bab 5.10).
3. storage\_logs terus bertambah. Tidak ada kebijakan pemangkasan. Rekomendasi: arsipkan atau pangkas log lebih dari 2 tahun dengan tugas terjadwal terpisah, setelah disepakati dengan pihak kampus.
4. Pencadangan mengikuti SLA-DI-02: cadangan harian dengan mysqldump --single-transaction, RPO ≤ 24 jam. Pencadangan basis data tidak mencakup objek GCS; aktifkan object versioning atau retention policy pada bucket bila kampus menghendaki pemulihan berkas.

6.8 Checklist Bab 6

No	Item	Selesai bila
1	Migrasi berjalan bersih pada MySQL 8.0.16+ kosong	migrate up sukses, down lalu up ulang sukses
2	Seed referensi	6 peran, 3 prodi, 16 kategori terisi
3	Uji CHECK	Mencoba INSERT berkas Public tanpa status ditolak; Private + approved ditolak
4	Penyesuaian 6.7 butir 1	Dua patch kode diterapkan; hapus berkas Public/Approved lolos
5	EXPLAIN	Bukti penggunaan indeks untuk daftar berkas, explorer publik, retensi, dan antrean verifikasi
6	Uji beban pencarian	100.000 baris, persentil ke-95 ≤ 1 detik (TC-SRC-20)
7	Lampiran C diperbarui	Tipe waktu dan keterangan R7 sesuai 6.7

Ketik “lanjut” untuk Bab 7 (Screen Design & Wireframe Architecture). Bab ini merinci 14 layar di Lampiran D menjadi spesifikasi komponen per layar (elemen filter, tabel, modal, tombol aksi, dan perilaku Next.js), tanpa mengulang daftar layar.

Lampiran A sampai E ditulis dalam bentuk **deskripsi pengerjaan**, bukan gambar. Setiap diagram dijabarkan sampai tingkat elemen (apa yang digambar, di jalur mana, disambung ke mana) sehingga dapat digambar langsung di draw.io, Camunda Modeler, atau Figma. ERD dirujuk di Bab 6 dan mockup dirujuk di Bab 7.

Catatan:

- 1. **Proses As-Is (A.1) adalah asumsi penyusun**, berdasarkan gambaran proyek (berkas tersebar di folder lokal, aplikasi pesan, dan surel) dan praktik umum arsip kampus. A.1 wajib divalidasi dengan staf BAAK sebelum diserahkan; daftar pertanyaan ada di A.1.4.
- 2. Proses To-Be, use case, ERD, dan mockup konsisten dengan Bab 1 sampai 6 (ID FR/NFR, nama tabel, dan nama *endpoint* sama).

Alat yang disarankan:

Dokumen	Alat	Format ekspor
BPMN	Camunda Modeler atau draw.io (pustaka BPMN 2.0)	PNG/SVG, plus berkas <code>.bpmn</code>
Use case	draw.io (pustaka UML) atau StarUML	PNG/SVG
ERD	draw.io, dbdiagram.io, atau MySQL Workbench	PNG/SVG
Mockup	Figma	Tautan prototipe dan PNG per layar

A.0 Ketentuan Pemodelan

Aspek	Ketentuan
Notasi	BPMN 2.0, level kolaborasi ( <i>pool</i> dan <i>lane</i> ), satu diagram per proses
Penamaan tugas	Kata kerja + objek (“Memeriksa ukuran berkas”)
Penamaan gateway	Berbentuk pertanyaan (“Ukuran ≤ 10 MB?”); setiap cabang keluar diberi label (“Ya”/“Tidak”)
Penamaan event	Awal: kondisi pemicu (“Berkas perlu diarsipkan”); akhir: hasil (“Berkas tersimpan”)
Jumlah elemen	Maksimal sekitar 15–20 elemen per diagram; pecah menjadi <i>sub-process</i> bila lebih
Jenis tugas	<i>User Task</i> (aksi manusia di layar), <i>Service Task</i> (diproses sistem), <i>Send/Receive Task</i> (notifikasi), <i>Manual Task</i> (tindakan fisik, hanya pada As-Is)
Garis	<i>Sequence flow</i> (garis tegas) di dalam satu <i>pool</i> ; <i>message flow</i> (garis putus-putus) antar- <i>pool</i>
Penyimpanan data	<i>Data Store</i> untuk MySQL dan GCS (To-Be), folder lokal dan spreadsheet (As-Is)
Penanda masalah (As-Is)	Anotasi bernomor merah (P1, P2, ...) yang ditautkan ke tugas bermasalah
Penanda otomatisasi (To-Be)	<i>Service Task</i> berwarna biru muda; <i>User Task</i> putih
Berkas	BPMN-AsIs-CP01.bpmn, ... BPMN-ToBe-TP05.bpmn; setiap diagram diberi judul dan versi

A.1 Proses Saat Ini (Current / As-Is)

A.1.1 Peta Proses As-Is

ID	Proses	Pemicu	Hasil
CP-01	Pengarsipan berkas	Mahasiswa/dosen perlu menyerahkan berkas	Berkas tersimpan di folder dan tercatat di register
CP-02	Permintaan dan pencarian berkas	Seseorang membutuhkan berkas tertentu	Berkas diterima peminta atau permintaan ditolak/gagal
CP-03	Pemusnahan arsip lama	Staf menyadari ada arsip lama atau penyimpanan penuh	Arsip kedaluwarsa dihapus dan register diperbarui

A.1.2 Deskripsi Diagram As-Is

CP-01: Pengarsipan Berkas (As-Is)

**Pool:** “Civitas dan Staf FITE”. **Lane (atas ke bawah):** (1) Mahasiswa/Dosen, (2) Staf BAAK. **Data store:** “Folder Komputer Lokal/Drive Bersama”, “Spreadsheet Register Arsip”.

No	Elemen	Tipe	Lane	Keterangan	Lanjut ke
1	Berkas perlu diarsipkan	Start Event (None)	Mahasiswa/Dosen		2
2	Menyiapkan berkas	Manual Task	Mahasiswa/Dosen	Memindai atau menyiapkan dokumen	3

No	Elemen	Tipe	Lane	Keterangan	Lanjut ke
3	Mengirim berkas ke staf	Send Task	Mahasiswa/Dosen	Melalui surel, aplikasi pesan, atau flashdisk/cetak. <i>Message flow</i> ke langkah 4	4
4	Menerima berkas	Receive Task	Staf BAAK		5
5	Berkas lengkap dan sesuai?	Exclusive Gateway	Staf BAAK	Dinilai manual tanpa standar tertulis	Ya → 8; Tidak → 6
6	Meminta perbaikan	Send Task	Staf BAAK	<i>Message flow</i> kembali ke lane Mahasiswa/Dosen	7
7	Memperbaiki dan mengirim ulang	Manual Task	Mahasiswa/Dosen	Kembali ke langkah 3 (siklus ulang)	3
8	Menyimpan berkas ke folder	Manual Task	Staf BAAK	Terhubung ke <i>data store</i> “Folder Komputer Lokal”	9
9	Mencatat di register	Manual Task	Staf BAAK	Terhubung ke <i>data store</i> “Spreadsheet Register”	10
10	Berkas tersimpan	End Event	Staf BAAK		

Masalah (anotasi merah):

Kode	Ditautkan ke	Masalah
P1	Langkah 3	Tidak ada standar format dan ukuran berkas; kanal pengiriman beragam (surel, pesan, flashdisk)
P2	Langkah 5	Pemeriksaan kelengkapan subjektif dan tidak terdokumentasi
P3	Langkah 8	Berkas tersebar di banyak komputer/folder; penamaan tidak konsisten
P4	Langkah 8	Bergantung pada satu staf atau satu komputer; risiko hilang atau rusak
P5	Langkah 8	Tidak ada kontrol akses; siapa pun yang punya akses folder dapat melihat
P6	Langkah 9	Register manual rawan salah dan tidak sinkron dengan isi folder

CP-02: Permintaan dan Pencarian Berkas (As-Is)

**Pool:** “Civitas dan Staf FITE”. **Lane:** (1) Peminta (Mahasiswa/Dosen/Kaprodi/Dekan), (2) Staf BAAK.

No	Elemen	Tipe	Lane	Keterangan	Lanjut ke
1	Berkas dibutuhkan	Start Event	Peminta		2
2	Mengajukan permintaan	Send Task	Peminta	Via aplikasi pesan, surel, atau datang langsung	3
3	Menerima permintaan	Receive Task	Staf BAAK		4
4	Menelusuri folder dan register secara manual	Manual Task	Staf BAAK	Memakai <i>data store</i> folder dan register	5
5	Berkas ditemukan?	Exclusive Gateway	Staf BAAK		Ya → 6; Tidak → 9

No	Elemen	Tipe	Lane	Keterangan	Lanjut ke
6	Peminta berhak mengakses?	Exclusive Gateway	Staf BAAK	Dinilai staf berdasarkan kebiasaan, bukan aturan tertulis	Ya → 7; Tidak → 10
7	Mengirim berkas	Send Task	Staf BAAK	Surel, pesan, atau cetak	8
8	Menerima berkas	Receive Task	Peminta		End “Permintaan terpenuhi”
9	Memberi tahu berkas tidak ditemukan	Send Task	Staf BAAK		End “Permintaan gagal”
10	Menolak permintaan	Send Task	Staf BAAK		End “Permintaan ditolak”

Masalah:

Kode	Ditautkan ke	Masalah
P7	Langkah 4	Pencarian lambat (menelusuri banyak folder); tidak ada pencarian berdasarkan NIM, NIP, kategori
P8	Langkah 2-3	Waktu penyelesaian bergantung pada ketersediaan staf (bisa berhari-hari)
P9	Langkah 6	Keputusan hak akses subjektif dan tidak konsisten
P10	Langkah 7	Berkas terkirim lewat kanal tidak aman dan tidak tercatat (tidak ada jejak audit)

CP-03: Pemusnahan Arsip Lama (As-Is)

**Pool:** “Civitas dan Staf FITE”. **Lane:** (1) Staf BAAK, (2) Kaprodi/Dekan.

No	Elemen	Tipe	Lane	Keterangan	Lanjut ke
1	Penyimpanan penuh atau ada instruksi pembersihan	Start Event	Staf BAAK	Tidak terjadwal	2
2	Memeriksa tanggal arsip secara manual	Manual Task	Staf BAAK	Membuka folder satu per satu	3
3	Ada arsip lebih dari 8 tahun?	Exclusive Gateway	Staf BAAK		Ya → 4; Tidak → End “Tidak ada pemusnahan”
4	Menyusun daftar arsip kedaluwarsa	Manual Task	Staf BAAK		5
5	Meminta persetujuan pemusnahan	Send Task	Staf BAAK	<i>Message flow</i> ke lane Kaprodi/Dekan	6
6	Menyetujui pemusnahan	User Task	Kaprodi/Dekan		7
7	Menghapus atau memusnahkan berkas	Manual Task	Staf BAAK	Langsung permanen	8
8	Memperbarui register	Manual Task	Staf BAAK		End “Arsip dimusnahkan”

Masalah:

Kode	Ditautkan ke	Masalah
P11	Langkah 1	Retensi tidak dijalankan konsisten (bergantung ingatan atau penyimpanan penuh)
P12	Langkah 2	Pemeriksaan tanggal manual dan rawan terlewat
P13	Langkah 7	Tidak ada masa tenggang; penghapusan langsung permanen dan tidak dapat dipulihkan
P14	Langkah 8	Tidak ada jejak audit siapa menghapus apa dan kapan

### A.1.3 Cara Menggambar As-Is

1. Buat satu berkas diagram per proses (CP-01 sampai CP-03). Mulai dengan *pool* dan *lane*, lalu letakkan *start event* di kiri atas.
2. Tempatkan elemen dari kiri ke kanan mengikuti nomor urut pada tabel; geser ke *lane* yang tertera pada kolom Lane.
3. Untuk setiap *message flow*, gambar garis putus-putus dengan ujung lingkaran di pengirim dan panah terbuka di penerima.
4. Tambahkan *data store* dan sambungkan dengan *data association* (garis putus-putus berpanah) ke tugas yang membaca/menulis.
5. Tambahkan anotasi merah P1–P14 pada tugas yang bersangkutan, lalu buat tabel masalah (seperti di atas) di bawah diagram.

### A.1.4 Pertanyaan Validasi ke Staf BAAK (wajib sebelum finalisasi)

1. Saat ini berkas diterima lewat kanal apa saja (surel, WhatsApp, flashdisk, fisik)?
2. Di mana berkas disimpan (komputer siapa, Google Drive mana) dan siapa yang berwenang mengaksesnya?
3. Apakah ada register (buku atau spreadsheet) dan apa isi kolomnya?
4. Siapa yang memutuskan seseorang boleh meminta berkas milik orang lain, dan bagaimana aturannya?
5. Apakah pemusnahan arsip pernah dilakukan, siapa yang menyetujui, dan apakah ada berita acara?
6. Berapa lama rata-rata menemukan satu berkas yang diminta?

Jawaban nomor 6 sebaiknya dipakai sebagai angka pembanding di tabel A.3 (mis. “rata-rata 2 hari kerja” menjadi “dalam hitungan detik”).

## A.2 Proses Target (Target / To-Be)

### A.2.1 Peta Proses To-Be

ID	Proses	Padanan As-Is	FR/NFR
TP-00	Autentikasi dan sesi (sub-proses rujukan)	(baru)	FR-AUTH-01..04, NFR-SEC-04
TP-01	Unggah berkas	CP-01	FR-DOC-01..03, NFR-FIL-01/02
TP-02	Verifikasi berkas publik	(baru)	FR-BAK-03/04, FR-SA-02, FR-SYS-02/03
TP-03	Pencarian dan unduh berkas	CP-02	FR-SRC-01/02, FR-DOC-04
TP-04	Hapus berkas	(bagian CP-03)	FR-DOC-05, FR-SA-01
TP-05	Retensi otomatis	CP-03	FR-SYS-01, NFR-RET-01..03

Semua diagram To-Be memakai *pool* yang sama agar konsisten:

Pool	Isi
Pengguna (Peramban/Next.js)	<i>Lane</i> sesuai peran yang terlibat pada proses tersebut
Sistem Arsip FITE (API Go)	<i>Lane</i> “Layanan API” (semua <i>service task</i> ) dan, khusus TP-05, <i>lane</i> “Worker Penjadwal”
Google Cloud Storage	<i>Pool</i> terlipat ( <i>collapsed</i> ) sebagai peserta eksternal, atau <i>data store</i>
MySQL	<i>Data store</i>

A.2.3 Deskripsi Diagram To-Be

TP-00: Autentikasi dan Sesi (sub-proses yang dipanggil dengan *Call Activity*)

**Lane:** Pengguna; Layanan API.

No	Elemen	Tipe	Lane	Keterangan	Lanjut ke
1	Pengguna perlu masuk	Start Event	Pengguna		2
2	Mengisi email dan kata sandi	User Task	Pengguna		3
3	Memeriksa batas percobaan	Service Task	Layanan API	Pembatas laju: 5 gagal per menit	4
4	Dalam batas?	Exclusive Gateway	Layanan API		Ya → 5; Tidak → End “Ditolak (429)”
5	Memverifikasi kredensial	Service Task	Layanan API	Hash bcrypt; waktu respons sama untuk email tak dikenal	6
6	Kredensial benar?	Exclusive Gateway	Layanan API		Ya → 7; Tidak → End “Ditolak (401)”
7	Akun aktif?	Exclusive Gateway	Layanan API		Ya → 8; Tidak → End “Ditolak (403)”
8	Menerbitkan access token dan refresh token	Service Task	Layanan API	Access 15 menit; refresh sebagai <i>cookie</i> httpOnly; sesi dicatat di <code>auth_sessions</code>	9
9	Wajib ganti sandi?	Exclusive Gateway	Layanan API	<code>must_change_passwd</code>	Ya → 10; Tidak → End “Sesi aktif”
10	Mengganti kata sandi	User Task	Pengguna	Rute lain dikunci sampai selesai	End “Sesi aktif”

Event pelengkap (digambar sebagai *boundary event* pada tugas yang memakai token):

Pemicu	Tipe	Penanganan
Access token kedaluwarsa (API membalas 401)	Boundary Error Event	Service Task “Memperbarui token melalui refresh”; sukses → ulangi permintaan; gagal → kembali ke langkah 2
Refresh token dipakai ulang atau sesi dicabut	Boundary Error Event	Sesi dicabut; pengguna diarahkan login ulang

TP-01: Unggah Berkas

**Lane:** Pengguna (Pengunggah: semua peran); Layanan API. **Data store:** GCS, MySQL.

No	Elemen	Tipe	Lane	Keterangan	Lanjut ke
1	Berkas perlu diarsipkan	Start Event	Pengunggah		2
2	Masuk ke sistem	Call Activity (TP-00)	Pengunggah		3
3	Mengisi formulir unggah	User Task	Pengunggah	Berkas, judul, kategori (sesuai peran), visibilitas (default Private)	4
4	Memeriksa batas permintaan ( $\leq$ 11 MB)	Service Task	Layanan API	Message flow dari langkah 3	5
5	Ukuran berkas $\leq$ 10 MB?	Exclusive Gateway	Layanan API		Ya $\rightarrow$ 6; Tidak $\rightarrow$ 5a
5a	Menampilkan galat ukuran	Send Task	Layanan API	Message flow ke Pengunggah; Pengunggah memperbaiki lalu kembali ke 3	3
6	Memeriksa ekstensi dan isi berkas	Service Task	Layanan API	Whitelist 7 format; deteksi isi; struktur ZIP Office; ekstensi ganda	7
7	Format dan isi sah?	Exclusive Gateway	Layanan API		Ya $\rightarrow$ 8; Tidak $\rightarrow$ 5a (galat format)
8	Memeriksa hak kategori	Service Task	Layanan API	allowed_roles memuat peran pengunggah	9
9	Kategori diizinkan?	Exclusive Gateway	Layanan API		Ya $\rightarrow$ 10; Tidak $\rightarrow$ End “Ditolak (403)”
10	Menyimpan berkas ke GCS	Service Task	Layanan API	Nama objek UUID; terhubung ke data store GCS	11
11	Menyimpan metadata ke MySQL (transaksi)	Service Task	Layanan API	Dokumen, antrean verifikasi (bila Public), log upload	12
11E	Penyimpanan metadata gagal	Boundary Error Event pada 11	Layanan API	Memicu Compensation: “Menghapus objek GCS yang sudah terunggah”; berakhir di End “Gagal (500)”	
12	Visibilitas Public?	Exclusive Gateway	Layanan API		Ya $\rightarrow$ 13; Tidak $\rightarrow$ End “Berkas tersimpan (Private)”
13	Mengirim notifikasi ke BAAK dan Super Admin	Send Task	Layanan API	Memicu TP-02 (message flow ke pool TP-02)	End “Berkas menunggu verifikasi”

TP-02: Verifikasi Berkas Publik

**Lane:** Verifikator (BAAK/Super Admin); Layanan API; Pengunggah.

No	Elemen	Tipe	Lane	Keterangan	Lanjut ke
1	Berkas publik masuk antrean	Message Start Event	Layanan API	Dari TP-01 langkah 13	2
2	Menampilkan notifikasi antrean	Send Task	Layanan API	<i>Badge</i> notifikasi pada BAAK dan SA	3
3	Membuka antrean verifikasi	User Task	Verifikator	Daftar <i>pending</i> (terlama dulu)	4
4	Meninjau berkas	User Task	Verifikator	Melihat detail dan mengunduh	5
5	Layak dipublikasikan?	Exclusive Gateway	Verifikator		Ya → 6; Tidak → 8
6	Menyetujui berkas	User Task	Verifikator		7
7	Menetapkan status Approved dan Public	Service Task	Layanan API	Transaksi dengan penguncian baris; log <code>verify_approved</code>	7a
7a	Mengirim notifikasi disetujui	Send Task	Layanan API	<i>Message flow</i> ke lane Pengunggah	10
8	Mengisi alasan penolakan (5–500 karakter)	User Task	Verifikator		9
9	Menetapkan status Rejected dan Private	Service Task	Layanan API	Log <code>verify_rejected</code> ; notifikasi berisi alasan	9a
9a	Mengirim notifikasi ditolak	Send Task	Layanan API	<i>Message flow</i> ke Pengunggah	11
10	Menerima notifikasi disetujui	Receive Task	Pengunggah		End “Berkas tampil di explorer publik”
11	Menerima notifikasi ditolak	Receive Task	Pengunggah		Pengunggah dapat memperbaiki dan mengajukan ulang (kembali ke TP-01 atau ubah visibilitas) → End “Menunggu keputusan akhir”

Event pelengkap:

Elemen	Tipe	Keterangan
Keputusan bersamaan (BAAK dan SA memutuskan bersamaan)	Boundary Error Event pada 7/9	Salah satu menerima 409; Verifikator memuat ulang antrean (Service Task “Menolak keputusan ganda”)
Pengunggah membatalkan (kembali ke Private)	Message Boundary Event ( <i>interrupting</i> ) pada rentang langkah 3–5	Antrean dan notifikasi dibersihkan; proses berakhir di End “Dibatalkan oleh pengunggah”

TP-03: Pencarian dan Unduh Berkas

**Lane:** Pengguna; Layanan API. **Data store:** MySQL, GCS.

No	Elemen	Tipe	Lane	Keterangan	Lanjut ke
1	Berkas dibutuhkan	Start Event	Pengguna		2
2	Masuk ke sistem	Call Activity (TP-00)	Pengguna		3

No	Elemen	Tipe	Lane	Keterangan	Lanjut ke
3	Memasukkan kata kunci dan filter	User Task	Pengguna	Prodi, kategori, NIM/NIP, ekstensi, visibilitas	4
4	Menyaring berkas sesuai cakupan baca peran	Service Task	Layanan API	Aturan “VisibleTo”; memakai <i>data store</i> MySQL	5
5	Menampilkan hasil terpaginasi	Send Task	Layanan API		6
6	Berkas ditemukan?	Exclusive Gateway	Pengguna		Ya → 7; Tidak → 6a
6a	Mengubah kata kunci/filter	User Task	Pengguna	Kembali ke 3	3
7	Memilih berkas dan mengunduh	User Task	Pengguna		8
8	Memeriksa hak akses berkas	Service Task	Layanan API		9
9	Berhak?	Exclusive Gateway	Layanan API		Ya → 10; Tidak → End “Ditolak (403)”
10	Mengambil objek dari GCS dan mengalirkannya	Service Task	Layanan API	<i>Data store</i> GCS	11
11	Mencatat log unduhan	Service Task	Layanan API	Log download	12
12	Menerima berkas	Receive Task	Pengguna		End “Berkas diterima”

Jalur alternatif: pada langkah 3, Pengguna dapat memilih “Explorer Publik” yang menyaring hanya berkas Public Approved (tanpa perubahan alur berikutnya).

### TP-04: Hapus Berkas

**Lane:** Pengguna (pemilik/BAAK/Super Admin); Layanan API.

No	Elemen	Tipe	Lane	Keterangan	Lanjut ke
1	Berkas perlu dihapus	Start Event	Pengguna		2
2	Memilih hapus dan mengonfirmasi	User Task	Pengguna	Dialog konfirmasi	3
3	Memeriksa hak hapus	Service Task	Layanan API	Pemilik; BAAK untuk kategori operasional; SA semua	4
4	Berhak?	Exclusive Gateway	Layanan API		Ya → 5; Tidak → End “Ditolak (403)”
5	Hapus permanen diminta?	Exclusive Gateway	Layanan API	Hanya SA yang dapat memilih ini	Tidak → 6; Ya → 8
6	Membatalkan antrean verifikasi dan menandai terhapus ( <i>soft delete</i> )	Service Task	Layanan API	Log <code>soft_delete</code>	7
7	—	End Event		“Berkas terhapus (masa tenggang 90 hari)”	

No	Elemen	Tipe	Lane	Keterangan	Lanjut ke
8	Menghapus metadata dan objek GCS	Service Task	Layanan API	Dalam satu transaksi; objek GCS dihapus terakhir	9
8E	Penghapusan GCS gagal	Boundary Error Event pada 8	Layanan API	<i>Rollback</i> ; End “Gagal (dapat diulang)”	
9	Mencatat log hapus permanen	Service Task	Layanan API	Log <code>hard_delete</code>	End “Berkas terhapus permanen”

TP-05: Retensi Otomatis

**Lane:** Worker Penjadwal (sistem); opsional lane Super Admin untuk peninjauan. **Data store:** MySQL, GCS.

No	Elemen	Tipe	Lane	Keterangan	Lanjut ke
1	Pukul 02:00 WIB setiap hari	Timer Start Event	Worker	Ekspresi cron <code>0 2 * * *</code> , zona Asia/Jakarta	2
2	Mengambil kunci eksekusi tunggal	Service Task	Worker	<code>GET_LOCK</code> MySQL	3
3	Kunci diperoleh?	Exclusive Gateway	Worker		Ya → 4; Tidak → End “Dilewati (instans lain berjalan)”
4	<b>Tahap 1:</b> Mencari berkas dengan <code>archived_at</code> lebih dari 8 tahun	Service Task	Worker		5
5	Menandai kedaluwarsa ( <i>soft delete</i> ) per batch 200	Sub-Process (Multi-instance, <i>loop</i> )	Worker	Isi: batalkan antrean verifikasi, set <code>retention_status = expired</code> , log <code>expired_soft_delete</code>	6
6	<b>Tahap 2:</b> Mencari berkas <i>soft-deleted</i> ≥ 90 hari	Service Task	Worker		7
7	Menghapus permanen per batch 200	Sub-Process (Multi-instance, <i>loop</i> )	Worker	Isi: hapus metadata dan objek GCS dalam transaksi; log <code>expired_hard_delete</code> atau <code>deleted_hard_delete</code> ; kegagalan dicatat dan dicoba lagi pada putaran berikutnya	8
8	Membersihkan sesi kedaluwarsa lebih dari 30 hari	Service Task	Worker		9
9	Mencatat laporan retensi	Service Task	Worker	Jumlah soft, hard, gagal, anomali ( <code>archived_at</code> kosong)	10
10	Melepas kunci	Service Task	Worker		End “Putaran retensi selesai”

No	Elemen	Tipe	Lane	Keterangan	Lanjut ke
11	Meninjau log penyimpanan	User Task	Super Admin	Opsional; membaca log melalui dasbor	

### A.2.4 Cara Menggambar To-Be

1. Buat satu diagram per proses (TP-00 sampai TP-05). Pakai susunan *pool* dari A.2.2 dan letakkan *lane* peran sesuai kolom Lane.
2. Gambar jalur utama lebih dulu (garis lurus kiri ke kanan), lalu cabang gateway ke bawah.
3. TP-01 langkah 4 dan seterusnya: setiap perpindahan dari *lane* Pengunggah ke *lane* Layanan API adalah *message flow* (putus-putus); perpindahan di dalam satu *pool* memakai *sequence flow*.
4. Gambar *boundary event* (11E pada TP-01, 8E pada TP-04) dengan lingkaran ganda di tepi tugas; sambungkan ke *compensation activity* atau *end event* yang tertera.
5. Beri warna biru muda pada semua *Service Task* dan *Send Task* untuk menunjukkan otomatisasi.
6. Tambahkan tabel keterangan elemen (seperti di atas) di bawah setiap diagram.

### A.3 Perbandingan As-Is dan To-Be

Aspek	As-Is	To-Be	Masalah yang ditutup	Rujukan
Penyimpanan	Folder lokal, pesan, surel; tersebar	Satu repositori (GCS) + metadata MySQL	P2, P3, P4	FR-DOC-01
Standar berkas	Tidak ada	Whitelist 7 format, maksimal 10 MB, validasi isi	P1	NFR-FIL-01/02, NFR-SEC-06
Kontrol akses	Subjektif; berbasis kebiasaan	RBAC 6 peran, ditegakkan di server	P5, P9	NFR-SEC-03
Pencarian	Manual (folder dan register)	Pencarian kata kunci dan filter multi-kriteria	P7	FR-SRC-01/02
Waktu layanan	Bergantung staf; bisa berhari-hari	Mandiri; target $\leq 1$ detik pencarian	P8	SLA-RT-01/02
Distribusi berkas	Surel/pesan tanpa catatan	Unduh terotorisasi dan tercatat	P10	FR-SYS-04
Publikasi berkas	Tidak terstruktur	Alur verifikasi Pending → Approved/Rejected	(baru)	FR-DOC-03, FR-BAK-04
Retensi	Manual dan tidak terjadwal; langsung permanen	Otomatis harian; soft delete lalu masa tenggang 90 hari	P11, P12, P13	FR-SYS-01, NFR-RET-01/02
Jejak audit	Tidak ada	<code>storage_logs</code> untuk setiap operasi	P6, P14	FR-SYS-04
Kontrol pusat	Tidak ada	Dasbor dan log untuk Super Admin	P4, P14	FR-SA-05

B.1 Aktor

Aktor	Jenis	Peran dalam diagram
Pengguna Terautentikasi	Primer, <b>abstrak</b>	Induk dari enam aktor di bawah; use case umum dihubungkan ke aktor ini sehingga diwarisi semua peran
Super Admin	Primer	Mewarisi Pengguna Terautentikasi; use case khusus administrasi
Dekan	Primer	Mewarisi Pengguna Terautentikasi
BAAK / Staf	Primer	Mewarisi Pengguna Terautentikasi; use case verifikasi dan unggah atas nama
Kaprodi	Primer	Mewarisi Pengguna Terautentikasi
Dosen / TA	Primer	Mewarisi Pengguna Terautentikasi
Mahasiswa	Primer	Mewarisi Pengguna Terautentikasi
Penjadwal Sistem ( <i>Scheduler</i> )	Primer, non-manusia	Memicu pembersihan arsip otomatis
Google Cloud Storage	Sekunder (sistem eksternal)	Dipanggil oleh use case yang menyimpan, mengambil, atau menghapus objek berkas

**Catatan:** cakupan baca dan hak hapus yang berbeda per peran tidak dapat digambar pada diagram use case. Perbedaan itu dijelaskan oleh matriks akses (Bab 2.1) dan ditulis pada kolom “Catatan aturan” di bawah.

B.2 Daftar Use Case

*Batas sistem:* kotak bertuliskan “Sistem Informasi Arsip Berkas Digital FITE”. Semua use case (kecuali UC-01) mensyaratkan pengguna sudah login; syarat ini ditulis sebagai catatan di bawah diagram, **tidak** digambar sebagai garis *include* agar diagram tidak penuh.

ID	Use Case	Aktor Terhubung	FR	Catatan aturan
UC-01	Login	Semua pengguna (via induk), Mahasiswa dst.	FR-AUTH-01	Sesi memakai access dan refresh token; pembaruan token otomatis menjadi bagian UC ini dan tidak digambar terpisah
UC-02	Logout	Pengguna Terautentikasi	FR-AUTH-02	Mencabut sesi
UC-03	Kelola Profil dan Ganti Kata Sandi	Pengguna Terautentikasi	FR-AUTH-03	Akun baru wajib ganti sandi sebelum memakai fitur lain
UC-04	Unggah Berkas	Pengguna Terautentikasi; GCS (sekunder)	FR-DOC-01, 02	Kategori dibatasi peran; default Private
UC-05	Validasi Berkas	(internal)	NFR-FIL-01/02, NFR-SEC-06	<i>Include</i> dari UC-04; ukuran, format, isi
UC-06	Atur Visibilitas Berkas	Pengguna Terautentikasi	FR-DOC-03	Hanya pengunggah atau Super Admin
UC-07	Ajukan Verifikasi Berkas Publik	(internal)	FR-DOC-03	<i>Extend</i> dari UC-04 dan UC-06; syarat: visibilitas Public

ID	Use Case	Aktor Terhubung	FR	Catatan aturan
UC-08	Cari dan Filter Berkas	Pengguna Terautentikasi	FR-SRC-01, 02	Hasil dibatasi cakupan baca peran
UC-09	Lihat Detail Berkas	Pengguna Terautentikasi	FR-DOC-04	
UC-10	Unduh Berkas	Pengguna Terautentikasi; GCS	FR-DOC-04	Dicatat pada log
UC-11	Ubah Metadata Berkas	Pengguna Terautentikasi	FR-BAK-01	Pengunggah, BAAK (berkas berprodi), Super Admin
UC-12	Hapus Berkas	Pengguna Terautentikasi	FR-DOC-05	Soft delete; hak per peran sesuai matriks
UC-13	Hapus Permanen Berkas	Super Admin; GCS	FR-SA-01	<i>Extend</i> dari UC-12
UC-14	Telusuri Explorer Publik	Pengguna Terautentikasi	FR-DOC-03	Hanya Public + Approved
UC-15	Verifikasi Berkas Publik	BAAK, Super Admin	FR-BAK-03/04, FR-SA-02	Alasan wajib saat ditolak
UC-16	Kirim Notifikasi	(internal)	FR-SYS-02, 03	<i>Include</i> dari UC-07 dan UC-15
UC-17	Kelola Notifikasi	Pengguna Terautentikasi	FR-NTF-01	Lihat, tandai dibaca, hapus
UC-18	Lihat Dasbor Statistik	Pengguna Terautentikasi	FR-DKN-04	Isi bergantung peran
UC-19	Kelola Pengguna dan Peran	Super Admin	FR-SA-03	Buat, ubah, ubah peran, nonaktifkan, reset sandi, hapus
UC-20	Kelola Kategori	Super Admin	FR-SA-04	
UC-21	Kelola Program Studi	Super Admin	FR-SA-04	
UC-22	Lihat Log Penyimpanan	Super Admin	FR-SA-05	
UC-23	Unggah Berkas atas Nama Pengguna Lain	BAAK, Super Admin	FR-BAK-02	<i>Extend</i> dari UC-04
UC-24	Bersihkan Arsip Kedaluwarsa Otomatis	Penjadwal Sistem; GCS	FR-SYS-01	Soft delete lalu hard delete setelah 90 hari

### B.3 Relasi

Jenis	Dari	Ke	Kondisi/Keterangan
Generalisasi aktor	Super Admin, Dekan, BAAK, Kaprodi, Dosen/TA, Mahasiswa	Pengguna Terautentikasi	Panah segitiga kosong; use case yang dihubungkan ke induk berlaku untuk semua peran
<<include>>	UC-04 Unggah Berkas	UC-05 Validasi Berkas	Selalu dilakukan
<<extend>>	UC-07 Ajukan Verifikasi Berkas Publik	UC-04 Unggah Berkas	Titik perluasan “Visibilitas = Public”
<<extend>>	UC-07 Ajukan Verifikasi Berkas Publik	UC-06 Atur Visibilitas Berkas	Kondisi: diubah ke Public
<<include>>	UC-07	UC-16 Kirim Notifikasi	Memberi tahu BAAK dan Super Admin
<<include>>	UC-15 Verifikasi Berkas Publik	UC-16 Kirim Notifikasi	Memberi tahu pengunggah
<<extend>>	UC-23 Unggah atas Nama Pengguna Lain	UC-04 Unggah Berkas	Kondisi: aktor BAAK/Super Admin mengisi pemilik berkas
<<extend>>	UC-13 Hapus Permanen	UC-12 Hapus Berkas	Kondisi: aktor Super Admin memilih hapus permanen

Jenis	Dari	Ke	Kondisi/Keterangan
Asosiasi (sekunder)	GCS	UC-04, UC-10, UC-13, UC-24	Garis tanpa panah dari use case ke aktor sekunder (sisi kanan diagram)

## B.4 Tata Letak Diagram

1. **Kiri:** aktor “Pengguna Terautentikasi” di bagian atas; di bawahnya enam aktor peran (urutan: Super Admin, Dekan, BAAK, Kaprodi, Dosen/TA, Mahasiswa) dengan panah generalisasi ke induknya.
2. **Tengah:** kotak batas sistem. Kelompokkan use case dengan baris/bagian visual: *Akun* (UC-01 sampai 03), *Berkas* (UC-04 sampai 14), *Verifikasi dan Notifikasi* (UC-15 sampai 17), *Pemantauan* (UC-18), *Administrasi* (UC-19 sampai 22), *Otomatis* (UC-24).
3. **Kanan:** aktor sekunder “Google Cloud Storage” dan aktor “Penjadwal Sistem”.
4. Garis asosiasi dari Super Admin ke UC-13, UC-19 sampai UC-22, dan UC-15; dari BAAK ke UC-15 dan UC-23.
5. Beri catatan teks di bawah diagram: “Seluruh use case selain UC-01 mensyaratkan login. Cakupan akses per peran mengikuti matriks Bab 2.1.”

## B.5 Spesifikasi Use Case (empat yang paling penting)

### UC-01 Login

Atribut	Isi
Aktor	Pengguna (semua peran)
Deskripsi	Pengguna masuk untuk memperoleh sesi
Prakondisi	Akun terdaftar dan aktif
Pascakondisi sukses	Access token diterbitkan; <i>cookie</i> refresh terpasang; sesi tercatat
Alur utama	1. Pengguna membuka halaman login. 2. Mengisi email dan kata sandi. 3. Sistem memeriksa batas percobaan. 4. Sistem memverifikasi kredensial. 5. Sistem memeriksa akun aktif. 6. Sistem menerbitkan token dan membuka sesi. 7. Bila <code>must_change_password</code> , sistem mengarahkan ke halaman ganti sandi; selain itu ke dasbor
Alur alternatif	A1: Kredensial salah → pesan generik (401), percobaan dihitung. A2: Percobaan > 5 per menit → ditolak 429 dengan waktu tunggu. A3: Akun nonaktif → ditolak 403. A4: Field kosong atau format email salah → 422
Aturan	NFR-SEC-02, 04, 05; TC-AUTH-01..07

### UC-04 Unggah Berkas (dengan UC-05 dan UC-07)

Atribut	Isi
Aktor	Pengguna Terautentikasi; GCS (sekunder)
Prakondisi	Sudah login; kategori yang dipilih diizinkan untuk perannya
Pascakondisi sukses	Berkas tersimpan (Private), atau menunggu verifikasi (Public); log tercatat
Alur utama	1. Pengguna membuka dialog unggah. 2. Memilih berkas, mengisi judul, kategori, dan visibilitas. 3. Sistem memeriksa ukuran, format, dan isi (UC-05). 4. Sistem memeriksa hak kategori. 5. Sistem menyimpan berkas ke GCS. 6. Sistem menyimpan metadata dan mencatat log. 7. Bila Public, sistem membuat antrean verifikasi dan mengirim notifikasi (UC-07)

Atribut	Isi
Alur alternatif	A1: Ukuran > 10 MB → 422; > 11 MB → 413. A2: Format di luar whitelist atau isi tidak sesuai → 422. A3: Kategori bukan hak peran → 403. A4: Penyimpanan metadata gagal → objek GCS dihapus kembali, galat 500. A5: BAAK/SA mengisi pemilik berkas → UC-23
Aturan	FR-DOC-01..03; NFR-FIL-01/02; TC-UPL-01..29

UC-15 Verifikasi Berkas Publik

Atribut	Isi
Aktor	BAAK, Super Admin
Prakondisi	Ada berkas berstatus <i>pending</i>
Pascakondisi sukses	Status berkas Approved (tampil di explorer publik) atau Rejected (tetap Private); pengunggah dinotifikasi
Alur utama	1. Verifikator membuka antrean <i>pending</i> . 2. Memilih satu berkas dan meninjaunya. 3. Memilih Setujui atau Tolak. 4. Bila Tolak, mengisi alasan (5-500 karakter). 5. Sistem menyimpan keputusan dalam transaksi. 6. Sistem mengirim notifikasi ke pengunggah (UC-16)
Alur alternatif	A1: Berkas sudah diputuskan atau verifikator lain memutuskan bersamaan → 409, daftar dimuat ulang. A2: Aktor selain BAAK/SA → 403. A3: Alasan kosong saat menolak → 422
Aturan	FR-BAK-04, FR-SA-02; TC-VER-01..19

UC-24 Bersihkan Arsip Kedaluwarsa Otomatis

Atribut	Isi
Aktor	Penjadwal Sistem; GCS (sekunder)
Prakondisi	Pukul 02:00 WIB; kunci eksekusi tunggal diperoleh
Pascakondisi sukses	Berkas > 8 tahun ditandai terhapus; berkas yang sudah ≥ 90 hari terhapus dihapus permanen; log dan laporan tercatat
Alur utama	1. Penjadwal terpicu. 2. Mengambil kunci. 3. Tahap 1: menandai berkas <code>archived_at</code> > 8 tahun (soft delete) per batch. 4. Tahap 2: menghapus permanen berkas <i>soft-deleted</i> ≥ 90 hari (metadata dan objek GCS). 5. Membersihkan sesi lama. 6. Mencatat laporan dan melepas kunci
Alur alternatif	A1: Kunci tidak diperoleh → proses dilewati. A2: Hapus objek GCS gagal → transaksi dibatalkan, dicoba lagi besok, kegagalan dicatat. A3: <code>archived_at</code> kosong → dicatat sebagai anomali, tidak dihapus
Aturan	FR-SYS-01, NFR-RET-01..03; TC-RET-01..18

# LAMPIRAN C: ENTITY RELATIONSHIP DIAGRAM (ERD)

## C.1 Pengaturan Diagram

Aspek	Ketentuan
Notasi	<i>Crow’s Foot</i>
Tingkat	Fisik (nama tabel dan kolom sesuai basis data), tipe data disingkat
Kode kunci	<b>PK</b> kunci primer (digarisbawahi); <b>FK</b> kunci asing (cetak miring); <b>UK</b> unik
Tabel yang digambar	9 tabel aplikasi. Tabel bawaan <i>framework</i> tidak digambar
Pengelompokan warna	<i>Identitas</i> (roles, prodi, users, auth_sessions); <i>Arsip</i> (categories, documents, document_verifications); <i>Pendukung</i> (notifications, storage_logs)
Relasi logis (tanpa FK)	Digambar dengan garis putus-putus dan diberi label “logis, tanpa FK”

## C.2 Entitas dan Atribut

### roles

Kolom	Tipe	Kunci
id	BIGINT	PK
key	VARCHAR(30)	UK
name	VARCHAR(50)	
created_at, updated_at	DATETIME(3)	

### prodi

Kolom	Tipe	Kunci
id	BIGINT	PK
code	VARCHAR(5)	UK
name	VARCHAR(100)	
is_active	BOOLEAN	
created_at, updated_at	DATETIME(3)	

### users

Kolom	Tipe	Kunci
id	BIGINT	PK
name	VARCHAR(100)	
email	VARCHAR(255)	UK
password	VARCHAR(255)	
role_id	BIGINT	FK → roles
prodi_id	BIGINT (null)	FK → prodi
nim	VARCHAR(30) (null)	UK
nip	VARCHAR(30) (null)	UK
phone	VARCHAR(20) (null)	
is_active	BOOLEAN	

Kolom	Tipe	Kunci
must_change_password	BOOLEAN	
deleted_at	DATETIME(3) (null)	
created_at, updated_at	DATETIME(3)	

## auth\_sessions

Kolom	Tipe	Kunci
id	CHAR(36)	PK
user_id	BIGINT	FK → users
refresh_token_hash	CHAR(64)	UK
previous_refresh_hash	CHAR(64) (null)	
user_agent	VARCHAR(255)	
ip	VARCHAR(45)	
expires_at	DATETIME(3)	
absolute_expires_at	DATETIME(3)	
last_used_at	DATETIME(3) (null)	
revoked_at	DATETIME(3) (null)	
created_at	DATETIME(3)	

## categories

Kolom	Tipe	Kunci
id	BIGINT	PK
name	VARCHAR(100)	UK
allowed_roles	JSON	
is_operational	BOOLEAN	
created_at, updated_at	DATETIME(3)	

## documents

Kolom	Tipe	Kunci
id	BIGINT	PK
title	VARCHAR(255)	
description	TEXT (null)	
original_name	VARCHAR(255)	
extension	VARCHAR(10)	
mime_type	VARCHAR(100)	
size_kb	INT	
storage_path	VARCHAR(500)	UK
category_id	BIGINT	FK → categories
prodi_id	BIGINT (null)	FK → prodi
uploader_id	BIGINT	FK → users
owner_id	BIGINT	FK → users
owner_role	VARCHAR(30)	
visibility	ENUM('private','public')	
verification_status	ENUM('pending','approved','rejected') (null)	
archived_at	DATETIME(3) (null)	

Kolom	Tipe	Kunci
retention_status	ENUM('expired') (null)	
deleted_at	DATETIME(3) (null)	
created_at, updated_at	DATETIME(3)	

document\_verifications

Kolom	Tipe	Kunci
id	BIGINT	PK
document_id	BIGINT	FK → documents, UK
status	ENUM('pending','approved','rejected')	
reason	VARCHAR(500) (null)	
requested_by	BIGINT	FK → users
verified_by	BIGINT (null)	FK → users
requested_at	DATETIME(3)	
verified_at	DATETIME(3) (null)	
created_at, updated_at	DATETIME(3)	

notifications

Kolom	Tipe	Kunci
id	CHAR(36)	PK
user_id	BIGINT	FK → users
type	VARCHAR(50)	
title	VARCHAR(150)	
message	TEXT	
data	JSON (null)	
is_read	BOOLEAN	
created_at, updated_at	DATETIME(3)	

storage\_logs

Kolom	Tipe	Kunci
id	BIGINT	PK
document_id	BIGINT (null)	(bukan FK)
user_id	BIGINT (null)	FK → users
action	VARCHAR(40)	
meta	JSON (null)	
created_at	DATETIME(3)	

C.3 Relasi dan Kardinalitas

Cara baca: simbol di sisi tabel A menunjukkan berapa banyak A yang berelasi dengan satu B, dan sebaliknya.

No	Tabel A	Tabel B	Kardinalitas	Kolom FK	Partisipasi	Label garis
R1	roles	users	1 : N	users.role_id	Peran boleh tanpa pengguna; pengguna wajib punya satu peran	“memiliki”
R2	prodi	users	0..1 : N	users.prodi_id	Pengguna lintas prodi (SA, Dekan, BAAK) tanpa prodi	“menaungi”
R3	prodi	documents	0..1 : N	documents.prodi_id	Berkas fakultas tanpa prodi	“mengelompokkan”
R4	categories	documents	1 : N	documents.category_id	Berkas wajib punya kategori	“mengklasifikasikan”
R5	users	documents	1 : N	documents.uploader_id		“mengunggah”
R6	users	documents	1 : N	documents.owner_id		“memiliki”
R7	users	auth_sessions	1 : N	auth_sessions.user_id		“membuka sesi”
R8	documents	document_verifications	1 : 0..1	document_verifications.document_id (unik)	Berkas Private yang tidak pernah diajukan tidak punya baris verifikasi	“diverifikasi”
R9	users	document_verifications	1 : N	requested_by		“mengajukan”
R10	users	document_verifications	0..1 : N	verified_by	Kosong selama <i>pending</i>	“memutuskan”
R11	users	notifications	1 : N	notifications.user_id		“menerima”
R12	users	storage_logs	0..1 : N	storage_logs.user_id	Kosong untuk tindakan sistem	“melakukan”
R13	documents	storage_logs	0..1 : N	(logis, tanpa FK)	Log bertahan setelah berkas dihapus permanen	“dicatat”
R14	categories	roles	N : M (logis)	categories.allowed_roles (JSON)	Tanpa FK	“boleh diunggah oleh”

R5 dan R6 adalah dua garis terpisah antara `users` dan `documents`; beri label berbeda agar tidak membingungkan.

### C.4 Catatan Perancangan (tuliskan di bawah ERD)

- Normalisasi:** skema berada pada 3NF dengan dua denormalisasi yang disengaja: `documents.owner_role` (menyederhanakan filter cakupan baca) dan `categories.allowed_roles` (JSON, menggantikan tabel pivot untuk 6 peran tetap).
- Integritas hapus:** FK memakai `RESTRICT` (data induk tidak boleh dihapus selama dipakai), kecuali `document_verifications.document_id` dan `notifications.user_id` (`CASCADE`), serta `verified_by` dan `storage_logs.user_id` (`SET NULL`).
- Soft delete:** `users` dan `documents` memakai `deleted_at`; kueri normal mengabaikan baris yang ber-`deleted_at`.

4. **Konsistensi** `owner_role`: harus disinkronkan saat peran pengguna diubah (dijaga oleh layanan manajemen pengguna).
5. `storage_logs.document_id` **tanpa FK**: agar jejak audit tidak hilang ketika berkas dihapus permanen oleh retensi atau Super Admin.

## C.5 Cara Menggambar ERD

1. Letakkan **users** di tengah. Di kirinya: **roles** (atas) dan **prodi** (bawah). Di kanannya: **auth\_sessions** dan **notifications**.
2. Di bawah *users*: **documents**. Di sebelah kiri *documents*: **categories**. Di sebelah kanannya: **document\_verifications**. Di bawahnya: **storage\_logs**.
3. Gambar garis R1 sampai R13 dengan kaki gagak di sisi “banyak” dan simbol “|” (wajib) atau “o” (opsional) di sisi lainnya sesuai kolom Partisipasi.
4. Gambar R13 dan R14 putus-putus dan beri label.
5. Cantumkan semua kolom dengan kode PK/FK/UK sesuai tabel C.2; cukup kolom kunci dan kolom penting bila diagram terlalu padat, dan sertakan tabel lengkap C.2 sebagai pelengkap.

Bab 6.2 dan 6.3 akan mengambil isi bagian ini dan menambahkan indeks serta berkas migrasi SQL.

## D.1 Panduan Pembuatan di Figma

Aspek	Ketentuan
Ukuran <i>frame</i>	Desktop 1440×900 (semua layar); tablet 834×1112 (layar tabel); ponsel 390×844 (prioritas: S-01, S-03 varian Mahasiswa, S-04, S-06, S-09)
Grid	12 kolom desktop (margin 24 px), 4 kolom ponsel (margin 16 px); jarak kelipatan 8 px
Tipografi	Satu keluarga huruf sans-serif (mis. Inter); skala: 12/14/16/20/24/32
Warna	Ikuti identitas institusi bila ada. Usulan netral: <i>primary</i> biru tua, abu-abu netral, hijau (berhasil), kuning (menunggu), merah (galat)
Komponen	Buat komponen yang dapat dipakai ulang: tombol ( <i>primary/secondary/danger</i> ), input, <i>select</i> , tabel, <i>badge</i> status, modal, <i>toast</i> , <i>empty state</i> , <i>skeleton</i>
<i>Badge</i> status berkas	Private: abu-abu; Menunggu: kuning; Disetujui: hijau; Ditolak: merah. Setiap <i>badge</i> wajib berisi teks, tidak hanya warna
Aksesibilitas	Rasio kontras teks minimal 4,5:1; <i>focus state</i> terlihat; label pada setiap input; ukuran sasaran sentuh minimal 44 px
Tiga keadaan wajib per layar	<i>Loading</i> ( <i>skeleton</i> ), <i>empty</i> (pesan + ajakan tindak), <i>error</i> (pesan + coba lagi)
Penamaan	S-04 / File Explorer / Desktop / Default; varian peran: S-03 / Dasbor / Mahasiswa
Prototipe	Hubungkan tiga alur pada D.4

## D.2 Daftar Layar

ID	Layar	Rute Next.js	Peran	Komponen utama	Keadaan yang digambar	API
S-01	Login	/login	Publik	Logo, input email dan kata sandi (tombol tampilkan), tombol Masuk, pesan galat	Default; galat 401 (“Email atau kata sandi salah”); 429 (“Coba lagi dalam N detik”) dengan hitung mundur; akun nonaktif; <i>loading</i> tombol	POST /auth/login
S-02	Ganti Kata Sandi Wajib	/ganti-sandi	Semua (saat <code>must_change_password</code> )	Banner penjelasan, 3 input (lama, baru, konfirmasi), indikator syarat sandi	Default; syarat belum terpenuhi; sandi lama salah; sukses lalu ke dasbor	PUT /auth/password

ID	Layar	Rute Next.js	Peran	Komponen utama	Keadaan yang digambar	API
S-03	Dasbor (6 varian)	/dashboard	Semua	Kartu ringkasan, grafik (batang per prodi; garis tren unggahan), tabel “Berkas terbaru”	Varian per peran: SA (total berkas, pengguna, antrean, penyimpanan); Dekan (berkas fakultas, per prodi, akreditasi); BAAK (antrean verifikasi, unggahan hari ini); Kaprodi (berkas prodi, dosen, mahasiswa); Dosen (berkas pribadi, mahasiswa prodi, pending/rejected ); Mahasiswa (berkas pribadi per status). Plus <i>loading</i> dan <i>empty</i>	GET /dashboard/summary dan lainnya
S-04	File Explorer & Pencarian Lanjutan	/berkas	Semua	Kolom pencarian; panel filter (prodi, kategori, uploader role, NIM/NIP, ekstensi, visibilitas); tabel (ikon format, judul, kategori, prodi, pengunggah, ukuran, <i>badge</i> status, tanggal, aksi); pengurutan; paginasi; tombol Unggah	Default; filter aktif (chip dapat dihapus); <i>empty</i> (“Tidak ada berkas yang cocok”); <i>loading</i> ; galat	GET /documents, GET /documents/categories
S-05	Detail Berkas (panel samping)	/berkas/[id]	Semua (sesuai hak baca)	Metadata, <i>badge</i> status, tombol Unduh, Ubah, Ubah Visibilitas, Hapus (muncul sesuai hak), riwayat verifikasi (alasan penolakan bila ada)	Private; Menunggu; Disetujui; Ditolak (menampilkan alasan); 403	GET /documents/{id}

ID	Layar	Rute Next.js	Peran	Komponen utama	Keadaan yang digambar	API
S-06	Modal Unggah Berkas & Visibilitas	(modal di S-04)	Semua	Area <i>drag-and-drop</i> ; daftar format dan batas 10 MB; input judul, deskripsi, kategori (hanya yang diizinkan perannya); pilihan visibilitas (Private/Public) dengan teks “Berkas Public memerlukan verifikasi BAAK”; untuk BAAK/SA: input pemilik (NIM/NIP) dan prodi; tombol Unggah	Default; berkas dipilih; progres unggah; galat ukuran/format/isi (inline); sukses ( <i>toast</i> + status menunggu bila Public)	POST /documents
S-07	Explorer Publik	/publik	Semua	Pencarian, filter (prodi, kategori, ekstensi), kartu atau tabel berkas Public Approved, tombol Unduh	Default; <i>empty</i> ; <i>loading</i>	GET /documents/public
S-08	Panel Verifikasi	/verifikasi	BAAK, SA	Tab (Menunggu / Disetujui / Ditolak), filter prodi, tabel antrean (judul, pengunggah, prodi, waktu menunggu); modal tinjauan (metadata, unduh, tombol Setujui, Tolak); form alasan penolakan (5–500 karakter, penghitung)	Default; antrean kosong; konflik 409 (“Sudah diputuskan oleh verifikator lain” + muat ulang); sukses	GET /verifications , PATCH /verifications/{id}
S-09	Panel Notifikasi	/notifikasi + dropdown di <i>header</i>	Semua	Ikon lonceng dengan <i>badge</i> jumlah belum dibaca; dropdown 5 terbaru; halaman penuh dengan filter (Semua/Belum dibaca), tombol Tandai semua dibaca, hapus per item, tautan ke berkas terkait	Default; belum dibaca (penebalan); <i>empty</i>	GET /notifications , GET /notifications/unread-count , PATCH ...

ID	Layar	Rute Next.js	Peran	Komponen utama	Keadaan yang digambar	API
S-10	Manajemen Pengguna	/admin/pengguna	SA	Tabel (nama, email, peran, prodi, NIM/NIP, status), filter, tombol Tambah; modal Tambah/Ubah; menu aksi (ubah peran, nonaktifkan, reset sandi, hapus) dengan dialog konfirmasi	Default; galat validasi; konflik 409 (mis. SA terakhir)	GET/POST/PUT/PATCH/DELETE /users...
S-11	Kategori dan Program Studi	/admin/kategori, /admin/prodi	SA	Tabel kategori (nama, peran pengunggah, operasional); form Tambah/Ubah dengan <i>checkbox</i> peran; tabel prodi (kode, nama, aktif)	Default; hapus ditolak 409 (“Kategori masih dipakai”)	/documents/categories..., /users/prodi...
S-12	Log Penyimpanan & Pemakaian	/admin/log	SA	Kartu pemakaian (total berkas, ukuran, hampir kedaluwarsa, masa tenggang); tabel log (waktu, aksi, berkas, pelaku); filter aksi, pelaku, rentang tanggal	Default; <i>empty</i> ; tindakan sistem tampil sebagai “Sistem”	GET /dashboard/storage, GET /dashboard/storage-logs
S-13	Profil Saya	/profil	Semua	Form nama dan telepon; bagian ganti kata sandi; informasi peran dan prodi (tidak dapat diubah)	Default; sukses; galat	PUT /auth/profile, PUT /auth/password
S-14	Halaman Status	(beberapa rute)	Semua	403 “Tidak berwenang”; 404 “Tidak ditemukan”; “Sesi Anda berakhir” dengan tombol Masuk lagi; keadaan <i>offline</i> /server tidak tersedia	Empat keadaan	

**Kerangka halaman (*layout*) untuk S-03 sampai S-13:** *sidebar* kiri (menu sesuai peran, lihat matriks menu di Bab 6.1.2), *header* atas (judul halaman, lonceng notifikasi, menu pengguna dengan Profil dan Keluar). Di ponsel, *sidebar* menjadi menu *hamburger* dan tabel berubah menjadi daftar kartu.

### D.3 Perilaku yang Harus Terlihat pada Mockup

Perilaku	Layar	Catatan
Menu dan tombol aksi mengikuti hak peran	Semua	Mis. tombol Hapus tidak tampil bagi yang tidak berhak (tetapi server tetap memeriksa)
Tombol Hapus Permanen hanya untuk Super Admin	S-05	Dengan peringatan “Tidak dapat dipulihkan”
Pesan galat berbahasa Indonesia dan spesifik	S-01, S-06	Sesuai respons API (Bab 4)
Berkas Public menampilkan status menunggu setelah unggah	S-04, S-05, S-06	<i>Badge</i> kuning “Menunggu verifikasi”
Notifikasi muncul tanpa muat ulang halaman	S-09	<i>Polling</i> berkala atau saat fokus halaman
Sesi kedaluwarsa ditangani mulus	S-14	Pembaruan token otomatis; hanya bila gagal pengguna diarahkan ke login

### D.4 Alur Prototipe yang Dihubungkan di Figma

Alur	Langkah
<b>Alur 1: Masuk</b>	S-01 → (akun baru) S-02 → S-03 sesuai peran
<b>Alur 2: Unggah publik sampai terbit</b>	Mahasiswa: S-04 → S-06 (pilih Public) → <i>toast</i> menunggu → BAAK: S-09 (notifikasi) → S-08 (tinjau, Setujui) → Mahasiswa: S-09 (disetujui) → S-07 (berkas tampil)
<b>Alur 3: Penolakan dan pengajuan ulang</b>	BAAK: S-08 (Tolak + alasan) → Mahasiswa: S-09 → S-05 (melihat alasan) → S-06 atau ubah visibilitas (ajukan ulang)
<b>Alur 4: Sesi berakhir</b>	Layar mana pun → S-14 “Sesi berakhir” → S-01

### E.1 Keterlacakan

Proses To-Be	Use Case	Entitas utama	Layar	FR utama
TP-00 Autentikasi	UC-01, 02, 03	users, roles, auth_sessions	S-01, S-02, S-13, S-14	FR-AUTH-01..04
TP-01 Unggah berkas	UC-04, 05, 06, 07, 23	documents, categories, document_verifications, notifications, storage_logs	S-04, S-06	FR-DOC-01..03
TP-02 Verifikasi	UC-15, 16	document_verifications, documents, notifications	S-08, S-09	FR-BAK-03/04, FR-SA-02
TP-03 Cari dan unduh	UC-08, 09, 10, 14	documents, prodi, categories, storage_logs	S-04, S-05, S-07	FR-SRC-01/02, FR-DOC-04
TP-04 Hapus	UC-11, 12, 13	documents, storage_logs	S-05	FR-DOC-05, FR-SA-01
TP-05 Retensi	UC-24	documents, auth_sessions, storage_logs	S-12	FR-SYS-01, NFR-RET-01..03
(administrasi)	UC-17, 18, 19, 20, 21, 22	users, categories, prodi, notifications, storage_logs	S-03, S-09, S-10, S-11, S-12	FR-SA-03..05, FR-NTF-01

### E.2 Ceklis Penyelesaian

No	Hasil kerja	Selesai bila
1	Validasi As-Is	Pertanyaan A.1.4 dijawab staf BAAK dan diagram CP-01..03 disesuaikan
2	BPMN As-Is	3 diagram, masalah P1-P14 beranotasi
3	BPMN To-Be	6 diagram (TP-00..TP-05) dengan tabel elemen
4	Tabel perbandingan A.3	Memuat angka waktu layanan dari staf BAAK
5	Use case	1 diagram + tabel UC-01..24 + 4 spesifikasi (UC-01, 04, 15, 24)
6	ERD	1 diagram 9 tabel + tabel atribut C.2 + catatan C.4
7	Mockup	14 layar (S-01..S-14), 4 alur prototipe, tautan Figma di dokumen
8	Konsistensi	Nama tabel, endpoint, dan ID FR sama dengan Bab 1-5

### E.3 Keputusan yang Masih Terbuka

1. **Apakah staf BAAK bisa diwawancara** sebelum A.1 final? Bila tidak, tandai A.1 sebagai “kondisi umum berdasarkan observasi” dan jangan menulis angka waktu pada A.3.
2. **Apakah ada identitas visual institusi** (logo dan warna) untuk mockup? Bila ada, panduan warna D.1 disesuaikan.

Ketik **“lanjut”** untuk **Bab 6 (Database Design & Architecture)** versi Go: sitemap, relasi tabel, spesifikasi tabel, dan berkas migrasi SQL `golang-migrate`. Bab 6.2 akan merujuk Lampiran C, dan Bab 7 akan merujuk

